WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

Demonstration Report: FSO System Prototype Based on FPGA

2026.10.23

Prof. Jang Yeong Min

Kookmin University, Seoul, Korea

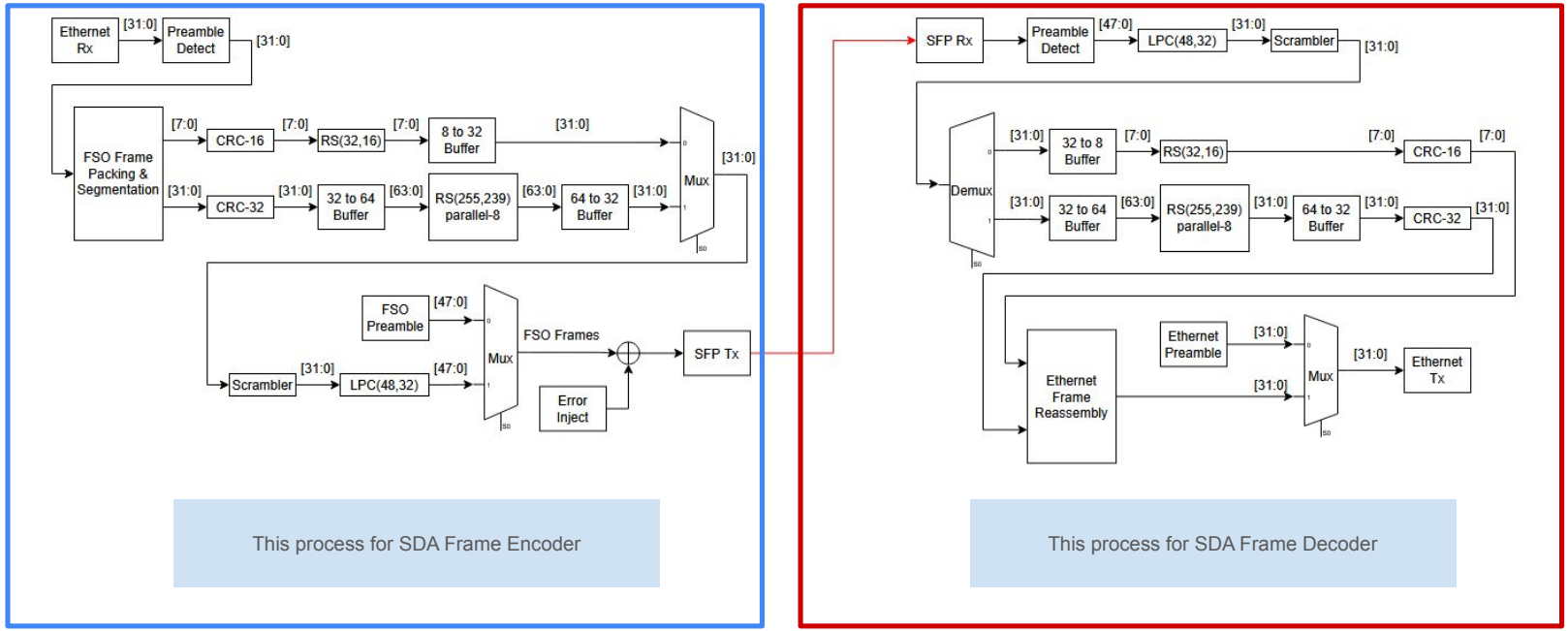
Wireless Communications and Artificial Intelligence Lab.
https://wireless.kookmin.ac.kr

Presentation Overview

1. Explanation of Previous System (14 August 2026) (Slide 3-5)
2. Problem of Previous System (Slide 6)
3. Solution and How to Solve the Problem (Slide 7)
4. New System Architecture (Slide 8 & 9)
5. Result of New System (Simulation Based) (Slide 10-15)
6. Hardware Demo (Slide 16-19)
7. Detailed System Update for System Architecture (Slide 20-24)
8. ARQ Selective Repeat (Slide 25-27)
9. Future Plan of Research and Paper (Slide 28-31)
10. Detailed System Explanation for Each Module (Slide 32-82)

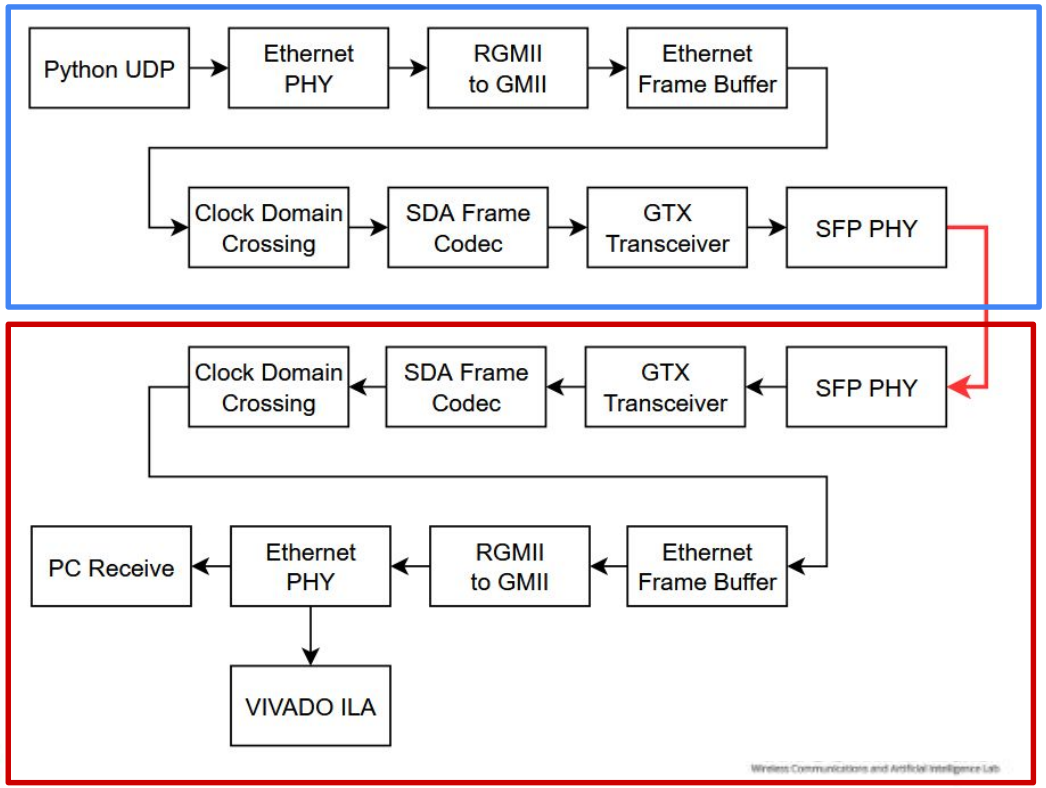
System Architecture (previous system)

System Architecture (14 August 2026)



System Architecture (previous system)

System Architecture (14 August 2026)

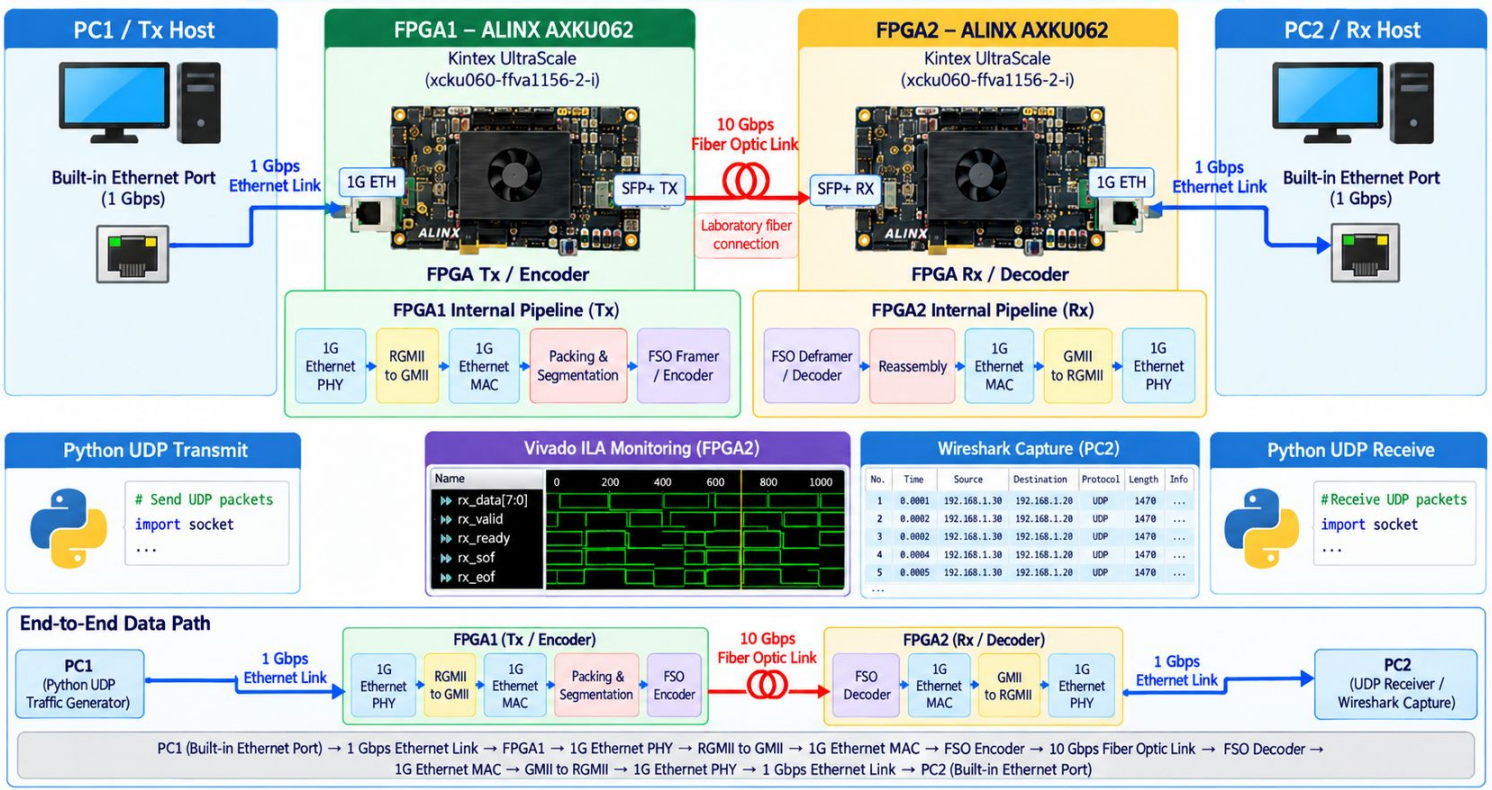


This process for SDA Frame Encoder

This process for SDA Frame Decoder

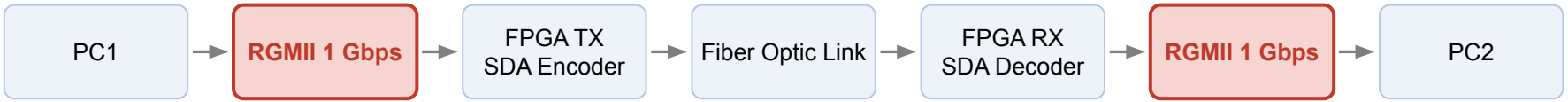
System Architecture (previous system)

System Architecture (14 August 2026)



Previous System Problem (Demo 14 August 2026)

Previous data path



Bottleneck: the 10G optical link is fed by a 1 Gbps RGMII Ethernet interface

1 Gbps data input

PC side uses RGMII (1G Ethernet), far below the 10 Gbps optical line rate.

Throughput instability

Data rate fluctuates between 6.1 Gbps and 8 Gbps; not fixed to SDA OISL Table 2-4.

No multi-frame support

Data longer than one FSO payload is not segmented across consecutive frames.

Frame gap 10 ns

Gap between FSO frames; SDA OISL requires a continuous frame stream.

No error injection

No controlled way to inject bit errors and verify FEC / CRC behaviour.

ARQ not implemented

Lost or corrupted frames are not retransmitted.

How We Resolved the Problem ?

New modules following SDA OISL v2.1.2 · 43 custom modules + 9 IP cores = 52 architectural blocks (TX + RX)

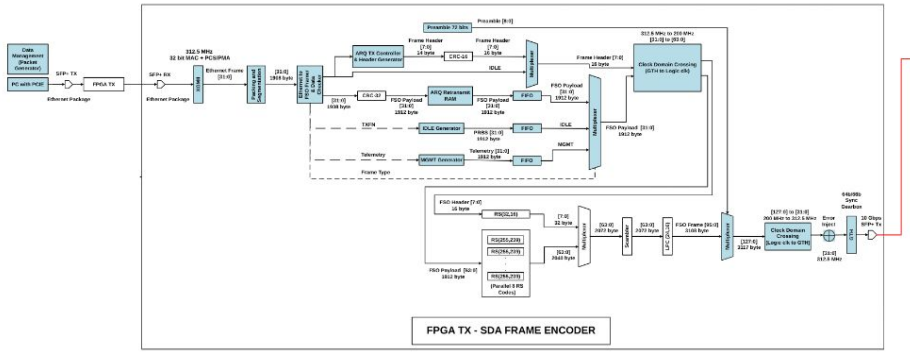
1 Gbps RGMII input	→	10GBASE-R over SFP+ (Intel X520 NIC) → XGMII 32-bit @ 312.5 MHz	Complete
Fluctuated Throughput 6.1 – 8 Gbps	→	Fixed frame timing per OISL Table 2-4: 2.4936 µs / frame and 6.121 Gbps Effective Data Rate	Complete
Frame gap 10 ns	→	By Adding Multiplexer and Fifo Buffer for Arbitration (Multiframe Mux) and transmit idle when No Data / MGMT.	Complete
No multi-frame	→	Adding Multiframe Module (Payload, Idle, MGMT, MUX Arbitration) based on SDA OISL Figure 2-6	Complete
No error injection	→	Error Injector: simulation mode (error.mem XOR) and hardware mode (manual VOA)	Complete
ARQ not implemented	→	ARQ Selective Repeat (SDA OCT method with SDA OISL header parameters)	In progress

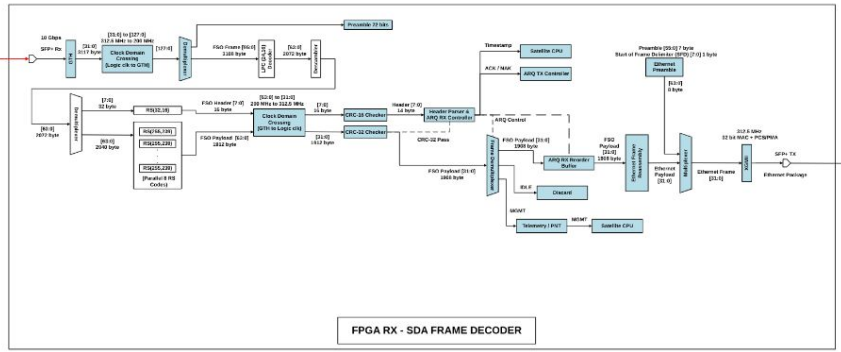
System Architecture (proposed system structure)

Blue color: New Module

Architecture Upgrade Summary Total Update :

- **New Custom Modules:** 43 Modules
- **New IP Cores / Buffers:** 9 IPs
- **Total Architectural Changes:** 52 Blocks





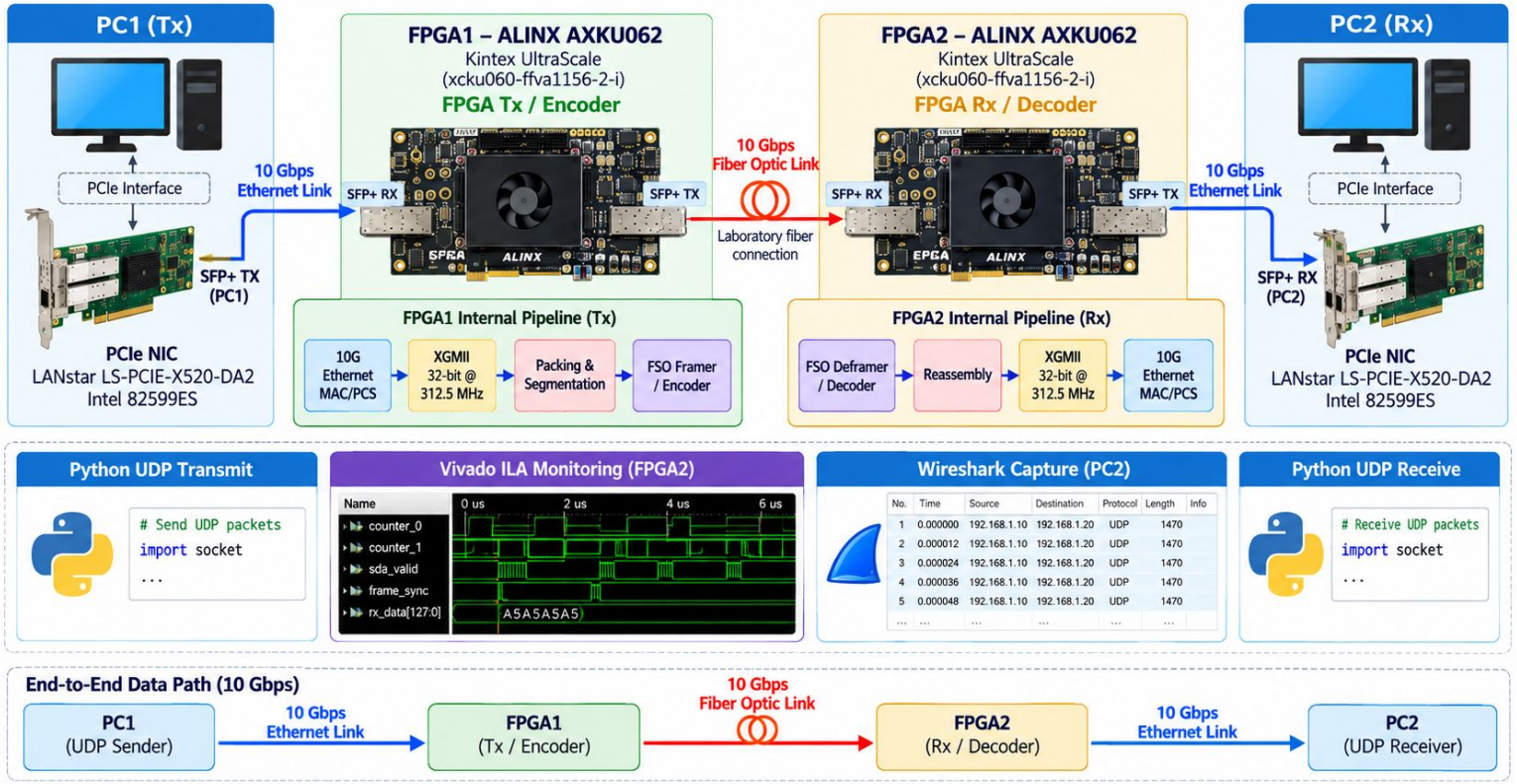
FPGA TX (Encoder) - Architecture Upgrade Summary Total Update (Encoder Only):

- **New Custom Modules:** 25 Modules
- **New IP Cores / Buffers:** 4 IPs
- **Total Architectural Changes:** 29 Blocks

FPGA RX (Decoder) - Architecture Upgrade Summary Total Update (Decoder Only):

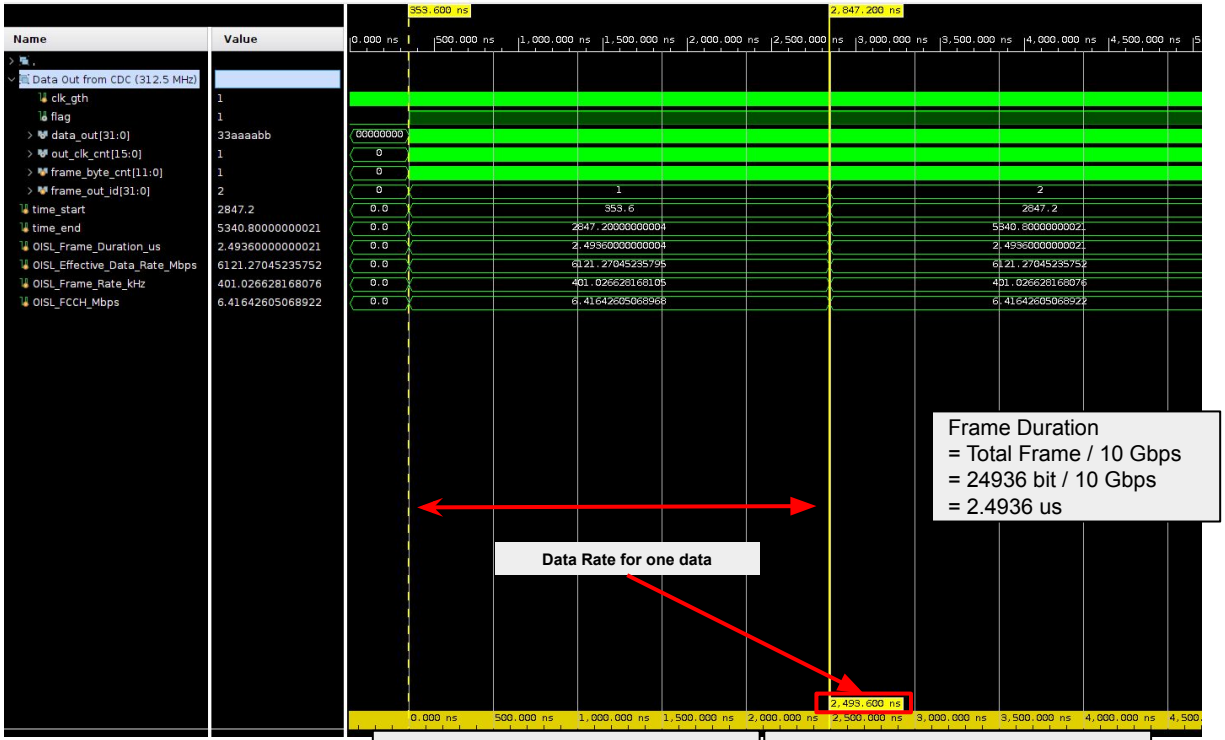
- **New Custom Modules:** 18 Modules
- **New IP Cores / Buffers:** 5 IPs
- **Total Architectural Changes:** 23 Blocks

New Testing Environment



Effective Data Rate

Following SDA OISL Table 2-4: Maximum Effective Data Rates for Frame Configurations



Frame Duration
= Total Frame / 10 Gbps
= 24936 bit / 10 Gbps
= 2.4936 us

2.4 Effective Data Rate

The effective data rates for the primary OISL modem frame configurations are summarized in Table 2-4 as a function of the OISL baud rate. The last column of the table indicates the required support for each baud rate.

Table 2-4: Maximum Effective Data Rates for Frame Configurations

Baud Rate (Mbaud)	Frame Duration (microseconds)	Effective Data Rate (Mbps)	Frame Rate (kHz)	Fast Control Channel (Mbps)	Requirement
312.5	79.795	191.290	12.532	0.201	Shall be supported
625	39.898	382.579	25.064	0.401	Should be supported
1250	19.949	765.159	50.128	0.802	Should be supported
2500	9.974	1530.318	100.257	1.604	May be supported
5000	4.987	3060.635	200.513	3.208	May be supported
10000	2.494	6121.270	401.027	6.416	May be supported

Baud Rate	Frame Duration	Effective Data Rate	Frame Rate	Fast Control Channel
10000 Mbaud (312.5 MHz x 32 Bit = 10 Gbps)	2.4936 us (In this slide show 2,493.600 ns)	6121.270 Mbps (6,121 Gbps)	401.02 66 kHz	6.416 Mbps

FCCH (Fast Control Channel 16-bit)

$$R_{FCCH} = \frac{16\text{ bit}}{2.493600\ \mu s} = \mathbf{6.416426}\ \text{Mbps}$$

Frame Rate

$$f_{frame} = \frac{1}{2.493600\ \mu s} = \mathbf{401.026628}\ \text{kHz}$$

Effective Payload Data Rate
Payload = 1908 Byte = 15,264 bit

$$R_{eff} = \frac{15,264\text{ bit}}{2.493600\ \mu s} = \mathbf{6121.270452}\ \text{Mbps}$$

Task Completed

0 ns

Frame gap

3117-byte frames sent back-to-back

2.4936 µs

Frame duration

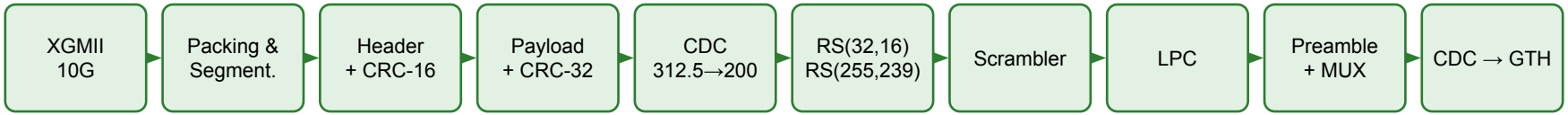
24,936 bits at 10,000 Mbaud

6121.27

Mbps effective data rate

Matches SDA OISL Table 2-4

TX encoder pipeline following SDA OISL (simulation verified)



- Frame gap solved
- All TX modules following SDA OISL
- Data rate following SDA OISL
- Error injector (simulation + hardware mode)

ARQ – still in development

Memory visibility · RAM usage · Retransmit & reorder

Power Consumption & Resource Utilization

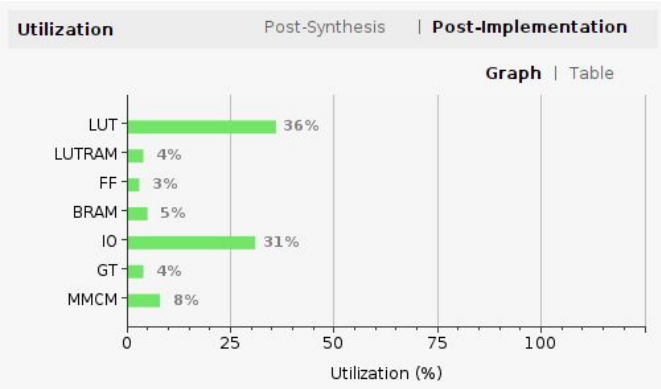
Vivado post-implementation report · Kintex UltraScale XCKU060-FFVA1156

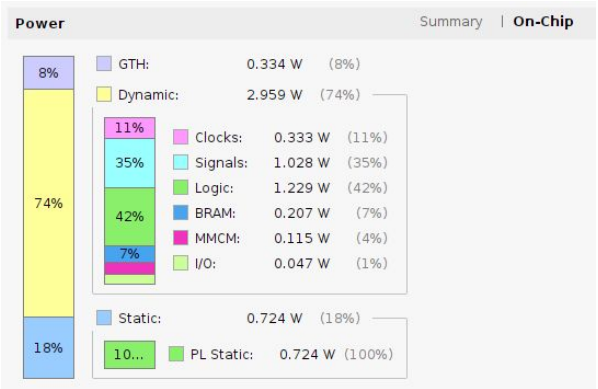
4.02 W
Total on-chip power
Power to off-chip devices: 0 W

30.6 °C
Junction temperature
Thermal margin 69.4 °C (47.4 W)

2.96 W
Dynamic power (74%)
Mostly logic and signal switching

0.72 W
Static power (18%)
Leakage, independent of activity





Logic + signals = 2.26 W (56%) of the total; the 10G optical interface (GTH) needs only 0.33 W. Impact on the project: next slide.
Confidence: Low (vectorless estimate) – to be confirmed by on-board measurement.

Impact on the FPGA Project

What the Vivado post-implementation report means for the Encoder + Decoder design

36% LUT used

64% LUT capacity remains

5% BRAM used

External DDR still needed for ARQ

4.016 W

Vivado on-chip power estimate

1 RESOURCE HEADROOM

Space for additional FPGA functions

- LUT 36% and BRAM 5% used in this design.
- Low utilization alone does not guarantee timing closure.

2 POWER & THERMAL

An initial baseline, not a hardware measurement

- 4.016 W on-chip; estimated junction temperature 30.6°C.
- 74% dynamic power → real traffic activity matters.

3 HIGH-SPEED TIMING

Timing closure is the next critical gate

- Check WNS/TNS and critical paths at the selected clocks.
- If needed: 64-bit @ 156.25 MHz + FIFO/CDC redesign.

4 SYSTEM-LEVEL SCALING

ARQ and net payload rate need separate validation

- BRAM 5% does not include a validated DDR ARQ budget.
- 10-Gb/s OISL signaling ≠ 10-Gb/s Ethernet payload.

NEXT VALIDATION **1. Measure board power & temperature** **2. Test ARQ/DDR + end-to-end UDP**

Vivado power confidence: LOW. Values describe the implemented design; they do not establish total board power or end-to-end system compliance.

This Year Plan (2026)

- **Half-Duplex → Full-Duplex**
- **ARQ Implementation**
- **Hardware Implementation & Demo**
- **Final Report**

12 – 23 Oct

Sprint 5 *Current*

RX Pipeline, Simulation & ETRI Demo

GTH RX, ARQ RX, end-to-end simplex sim, ILA setup, live demo

26 Oct – 6 Nov

Sprint 6 *Planned*

Hardware Implementation, Python BER Ecosystem & Validation

Python automation, RS(255,239) limit validation

9 – 20 Nov

Sprint 7 *Planned*

Full-Duplex Prototyping

Full-duplex integration, stress test

23 Nov – 1 Dec

Sprint 8 *Planned*

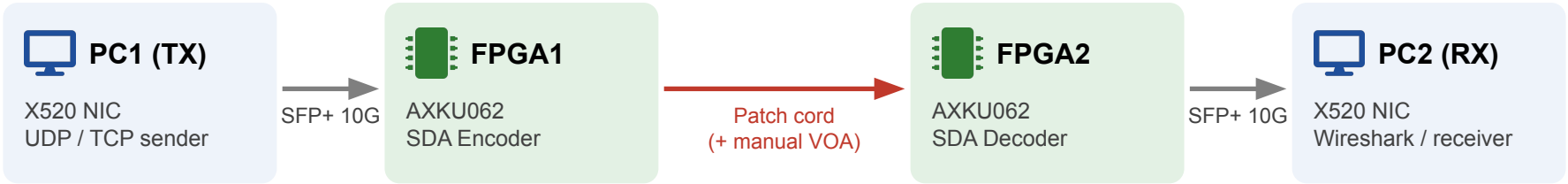
Final Submission

Architecture / schematic doc, ARQ viability report, ETRI handoff

Module Reference

XGMII / 10G Ethernet	**IEEE 802.3**	XGMII (Clause 46), 10GBASE-R PCS (Clause 49): IDLE 0x07, IPG ≥ 12 bytes, preamble / SFD
SDA Encoder & Decoder	**SDA OISL v2.1.2**	Frame format, CRC-16 / CRC-32, RS(32,16) header, RS(255,239) payload, scrambler, LPC, preamble (Sec. 2.6.3, Table 2-4)
GTH Transceiver	**AMD Xilinx PG182 & UG576**	UltraScale Transceivers Wizard (PG182) and GTH Transceivers User Guide (UG576)
ARQ	**SDA OCT v4.0.0 + SDA OISL v2.1.2**	Technical Selective Repeat procedure from SDA OCT, using SDA OISL header parameters

Hardware Demo



Demo procedure

1 Calibration

Connect PC1 ↔ PC2 with a patch cord; check SFP+ link up.

2 Send traffic

PC1 sends UDP / TCP packets through the PCIe X520 NIC at 10 Gbps.

3 Monitor

Vivado ILA on FPGA2 shows frame sync, CRC status and recovered data.

4 Verify

Wireshark on PC2 captures the packets; compare sent vs received count and throughput.

Hardware Preparation – Demo on 23 October 2026

Full Hardware Demo on 01 December 2026

Hardware Demo – Calibration Setup

Step 1: before the FPGAs go into the path, the two PCs are linked directly with a fiber patch cord to measure the best-case 10G link (baseline).

PC1 TX

192.168.10.1 · PCIe X520 NIC (SFP+)

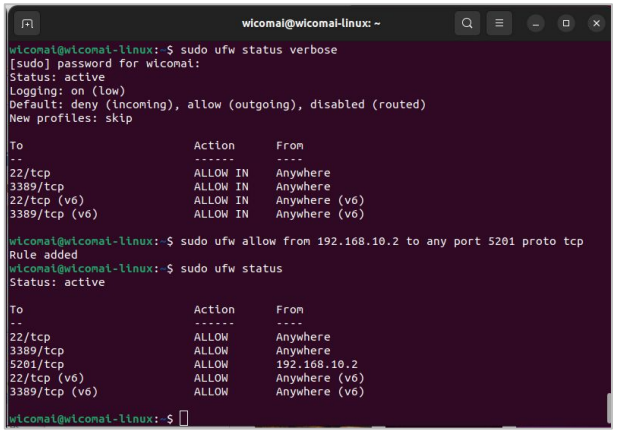
Fiber patch cord · SFP+ 10 Gbps

No FPGA in the path (calibration)

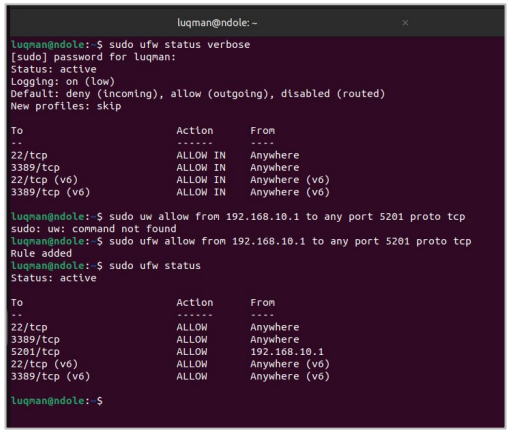
PC2 RX

192.168.10.2 · PCIe X520 NIC (SFP+)

Firewall on PC 1



Firewall on PC 2



1 Connect

SFP+ patch cord between the two X520 NICs, static IPs 192.168.10.1 (PC 1) and 192.168.10.2(PC 2).

2 Open port 5201

ufw allows TCP 5201 (iperf3) only from the peer PC.

3 Measure

ping both ways, then iperf3 TCP in both directions (next slide).

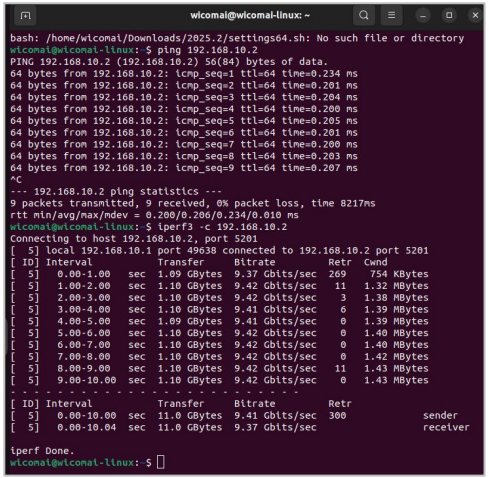
Hardware Demo – Calibration Result

This is testing calibration between PC1 and PC2 ping and iperf3 (TCP, 10 seconds) in both directions over the patch cord.

(PC 2) → (PC 1)

```
luqman@ndole:~$ ping 192.168.10.1
PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data.
64 bytes from 192.168.10.1: icmp_seq=1 ttl=64 time=0.255 ms
64 bytes from 192.168.10.1: icmp_seq=2 ttl=64 time=0.227 ms
64 bytes from 192.168.10.1: icmp_seq=3 ttl=64 time=0.217 ms
^C
--- 192.168.10.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2059ms
rtt min/avg/max/mdev = 0.217/0.233/0.255/0.016 ms
luqman@ndole:~$ iperf3 -c 192.168.10.1
Connecting to host 192.168.10.1, port 5201
[  5] local 192.168.10.2 port 45778 connected to 192.168.10.1 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec  1.10 GBytes  9.44 Gbits/sec   49   1.43 MBytes
[  5]   1.00-2.00   sec  1.09 GBytes  9.41 Gbits/sec    0   1.43 MBytes
[  5]   2.00-3.00   sec  1.10 GBytes  9.42 Gbits/sec    3   1.43 MBytes
[  5]   3.00-4.00   sec  1.10 GBytes  9.42 Gbits/sec    0   1.43 MBytes
[  5]   4.00-5.00   sec  1.10 GBytes  9.42 Gbits/sec    0   1.43 MBytes
[  5]   5.00-6.00   sec  1.10 GBytes  9.42 Gbits/sec    0   1.43 MBytes
[  5]   6.00-7.00   sec  1.10 GBytes  9.42 Gbits/sec    0   1.43 MBytes
[  5]   7.00-8.00   sec  1.10 GBytes  9.42 Gbits/sec    0   1.43 MBytes
[  5]   8.00-9.00   sec  1.09 GBytes  9.41 Gbits/sec    0   1.43 MBytes
[  5]   9.00-10.00  sec  1.10 GBytes  9.42 Gbits/sec    0   1.43 MBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec  11.0 GBytes  9.42 Gbits/sec   52             sender
[  5]   0.00-10.04  sec  11.0 GBytes  9.38 Gbits/sec                  receiver
```

(PC 1) → (PC 2)



≈ 0.2 ms

ping round trip, 0% packet loss

9.42 Gbps

TCP sender (9.37–9.38 at receiver), both directions

52 / 300

TCP retransmissions, mostly in the first second while TCP ramps up

Why 9.42 and not 10 Gbps? Each 1448-byte TCP payload uses a 1538-byte slot on the wire (Ethernet, IP and TCP headers, preamble, IPG): 10 Gbps × 1448 / 1538 ≈ 9.41 Gbps. So PCs, NICs and patch cord already run at full 10G. **Later with the FPGA OISL link in the path, the ceiling becomes 6.12 Gbps (OISL Table 2-4).**

Hardware Demo – Calibration Result

This is testing calibration between PC1 and PC2 ping and iperf3 (UDP, 10 seconds) in both directions over the patch cord.

(PC 1) → (PC 2) Ping Test

```
gio@Gio: ~
gio@Gio:~$ ping 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data.
64 bytes from 192.168.10.2: icmp_seq=1 ttl=64 time=0.255 ms
64 bytes from 192.168.10.2: icmp_seq=2 ttl=64 time=0.213 ms
64 bytes from 192.168.10.2: icmp_seq=3 ttl=64 time=0.202 ms
64 bytes from 192.168.10.2: icmp_seq=4 ttl=64 time=0.211 ms
^C
--- 192.168.10.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3100ms
rtt min/avg/max/mdev = 0.202/0.220/0.255/0.020 ms
gio@Gio:~$
```

(PC 1) → (PC 2) UDP Packet Test

```
gio@Gio: ~
gio@Gio:~$ iperf3 -c 192.168.10.2 -B 192.168.10.1 -u -b 10G -l 1972 -w 4M -t 10
warning: UDP block size 1972 exceeds TCP MSS 1948, may result in fragmentation / drops
Connecting to host 192.168.10.2, port 5201
[  5] local 192.168.10.1 port 47325 connected to 192.168.10.2 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   sec  1.13 GBytes  9.69 Gbits/sec  614086
[  5]   1.00-2.00   sec  1.13 GBytes  9.68 Gbits/sec  613325
[  5]   2.00-3.00   sec  1.13 GBytes  9.68 Gbits/sec  613330
[  5]   3.00-4.00   sec  1.13 GBytes  9.68 Gbits/sec  613336
[  5]   4.00-5.00   sec  1.13 GBytes  9.68 Gbits/sec  613330
[  5]   5.00-6.00   sec  1.13 GBytes  9.68 Gbits/sec  613328
[  5]   6.00-7.00   sec  1.13 GBytes  9.68 Gbits/sec  613335
[  5]   7.00-8.00   sec  1.13 GBytes  9.68 Gbits/sec  613326
[  5]   8.00-9.00   sec  1.13 GBytes  9.68 Gbits/sec  613334
[  5]   9.00-10.00  sec  1.13 GBytes  9.68 Gbits/sec  613337
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-10.00  sec  11.3 GBytes  9.68 Gbits/sec  0.000 ms  0/6134067 (0%)  sender
[  5]   0.00-10.04  sec  11.3 GBytes  9.64 Gbits/sec  0.002 ms  0/6133317 (0%)  receiver

iperf Done.
gio@Gio:~$
```

≈ 0.22 ms

ping round trip, 0% packet loss

9.68 Gbps

UDP sender (9.37–9.38 at receiver), both directions

0% loss

0 of 6,133,317 datagrams lost, jitter 0.002 ms

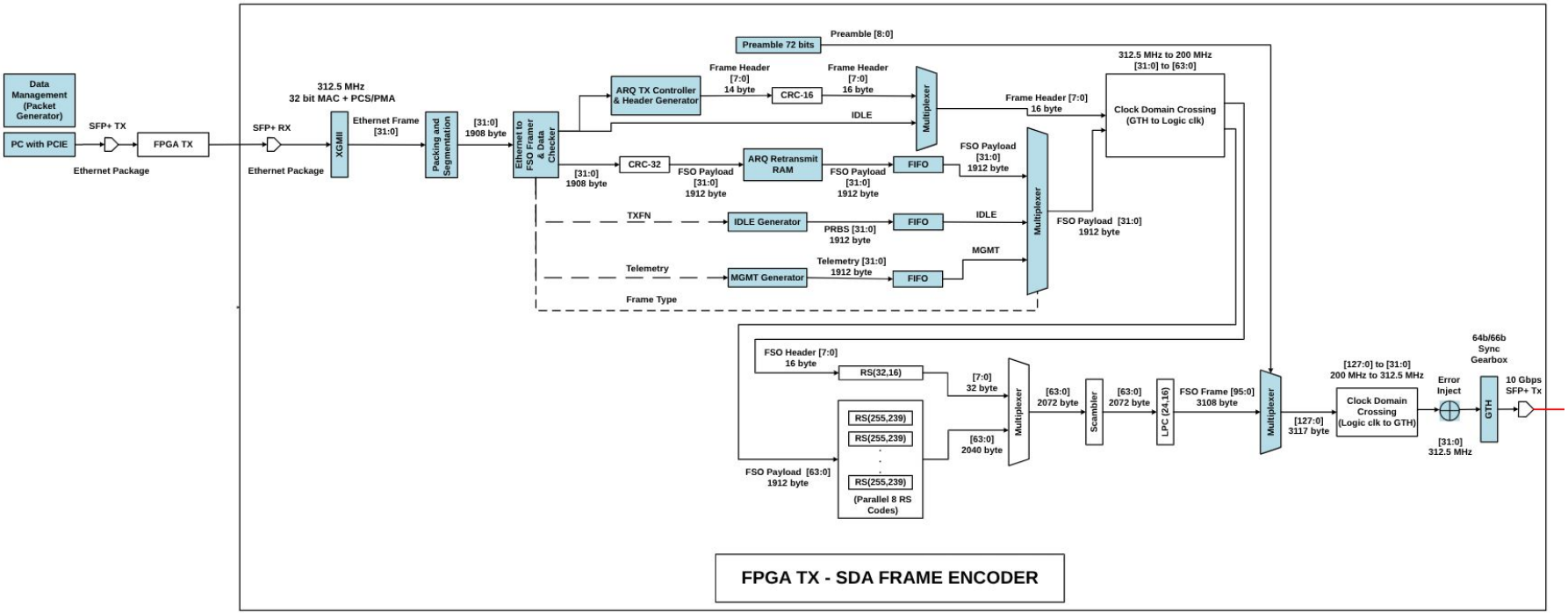
Why 9.68 and not 10 Gbps? **Each 1972-byte UDP payload takes a 2038-byte slot on the wire (28 B UDP/IP header + 38 B Ethernet header, FCS, preamble and gap): 10 Gbps × 1972 / 2038 ≈ 9.68 Gbps. So the PCs, NICs and patch cord already carry full 10G with no loss. With the FPGA OISL link in the path, the ceiling becomes 6.12 Gbps (OISL Table 2-4).**

Detailed System Update

Detail Explanation for New System Architecture

System Architecture (proposed system structure)

Blue color: New Module



System Architecture (proposed system structure)

FPGA TX (Encoder) - Architecture Upgrade Summary Total Update (Encoder Only):

- **New Custom Modules:** 25 Modules
- **New IP Cores / Buffers:** 4 IPs
- **Total Architectural Changes:** 29 Blocks

1. System Input & XGMII Front-End (Total: 4 Modules, 1 IP)

- **Create Module** – Data Management / Packet Generator (PCIe/NIC Simulator)
- **Create Module** – XGMII Parser
- **Create Module** – Ethernet Preamble Remover
- **Create Module** – Packing and Segmentation
- **Create IP** – 10G Ethernet PCS/PMA (MAC + PHY)

2. SDA Framer & Payload Multiplexing (Total: 15 Modules, 1 IPs)

- **Create Module** – Ethernet to FSO Framer & Data Checker
- **Create Module** – FSO Header Generator
- **Create Module** – CRC 16
- **Create Module** – FSO Header MUX
- **Create Module** – FSO Payload
- **Create Module** – 3x CRC 32 (DATA & IDLE & MGMT)
- **Create Module** – IDLE Payload Generator
- **Create Module** – Telemetry / MGMT Generator
- **Create Module** – Header Multiplexer (Frame Header & IDLE)
- **Create Module** – Payload Multiplexer (DATA & IDLE & MGMT)
- **Create Module** – 3x FIFO Buffers (for Payload, IDLE, and MGMT synchronization)
- **Update IP** – Clock Domain Crossing Async FIFO (312.5 MHz to 200 MHz)

3. ARQ TX Management (Total: 2 Module)

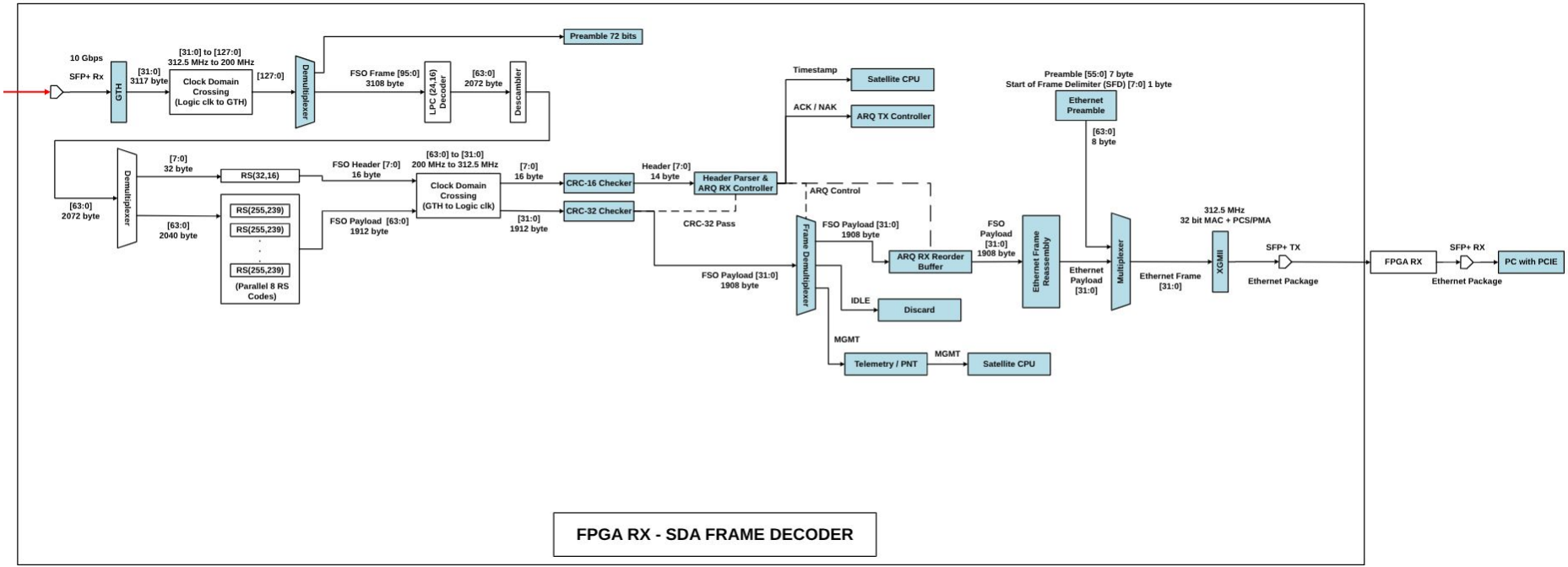
- **Create Module** – ARQ TX Controller
- **Create Module** – ARQ Retransmit RAM (Memory/Buffer Controller)

4. Post-FEC & Egress Pipeline (Total: 3 Modules, 2 IPs)

- **Create Module** – Preamble 72-bit Generator
- **Create Module** – Multiplexer for Preamble and LPC
- **Create Module** – Error Injector
- **Update IP** – Clock Domain Crossing Async FIFO (200 MHz to 312.5 MHz)
- **Create Module** – 64b/66b Sync Gearbox TX: Verilog code to encode and align 32-bit payloads into 64b/66b blocks with sync headers.
- **Create IP** – GTH Transceiver (10 Gbps SFP+ TX with 64b/66b Gearbox)

System Architecture (proposed system structure)

Blue color: New Module



System Architecture (proposed system structure)

FPGA RX (Decoder) - Architecture Upgrade Summary Total Update (Decoder Only):

- **New Custom Modules:** 18 Modules
- **New IP Cores / Buffers:** 5 IPs
- **Total Architectural Changes:** 23 Blocks

1. Physical Ingress & Synchronization (Total: 3 Modules, 2 IPs)

- **Create IP** – GTH Transceiver RX (Receives the 10 Gbps optical signal via SFP+).
- **Create Module** – 64b/66b Sync Gearbox RX: Verilog code to achieve block lock and decode 64b/66b blocks back to 32-bit data.
- **Update IP** – Clock Domain Crossing / CDC (Bridges 312.5 MHz GTH clock to 200 MHz Logic clock for baseband processing).
- **Create Module** – Demultiplexer (Separates the 3108-byte FSO frame and Preamble).
- **Create Module** – Preamble Extractor (Extracts the 72-bit Preamble to maintain continuous data stream synchronization).

2. Data Validation & Post-FEC CDC (Total: 3 Modules, 1 IP)

- **Create Module** – Header & Payload Demultiplexer (Splits the decoded stream into a 32-byte header and a 2040-byte payload for error checking).
- **Update IP** – Clock Domain Crossing / CDC (Bridges the 200 MHz baseband data back to the 312.5 MHz domain after FEC decoding).
- **Update Module** – CRC-16 Checker (Validates the integrity of the 14-byte Frame Header).
- **Update Module** – CRC-32 Checker (Validates the integrity of the 1908-byte FSO Payload).

3. ARQ & Frame Routing Management (Total: 7 Modules, 1 IP)

- **Create Module** – CRC 16 Checker
- **Create Module** – Header Parser
- **Create Module** – ARQ RX Controller (Evaluates CRC status and generates ACK/NAK signals for the ARQ TX Controller).
- **Create Module** – CRC 32 Checker
- **Create Module** – Frame Demultiplexer (Dynamically routes the 1908-byte payload based on its Frame Type indicator).
- **Create Module** – MGMT Extractor (Extracts timestamp data for the Satellite CPU).
- **Create Module** – IDLE Discard (Safely drops dummy IDLE frames from the pipeline).
- **Create IP** – ARQ RX Reorder Buffer (Buffers and re-sequences out-of-order 1908-byte payloads before reassembly).

4. Payload Reassembly & Egress Back-End (Total: 5 Modules, 1 IPs)

- **Create Module** – Ethernet Frame Reassembly (Stitches the 1908-byte FSO payloads back into their original, dynamic-sized Ethernet frames).
- **Create Module** – Ethernet Preamble Generator (Generates the standard 7-byte Preamble and 1-byte SFD).
- **Create Module** – Ethernet Multiplexer (Combines the standard Ethernet Preamble with the reassembled Ethernet Payload).
- **Create Module** – XGMII TX Formatter & IPG: Verilog code to insert Terminate (0xFD) and 12-byte Inter-Packet Gap.
- **Create Module** – Ethernet MAC Packing (Dynamic Ethernet MAC Transmit Stream Packer)
- **Create IP** – XGMII Formatter & 10G PCS/PMA TX (Formats the 32-bit data and transmits the rebuilt Ethernet packages via SFP+ TX to the Receiver PC)

ARQ Selective Repeat

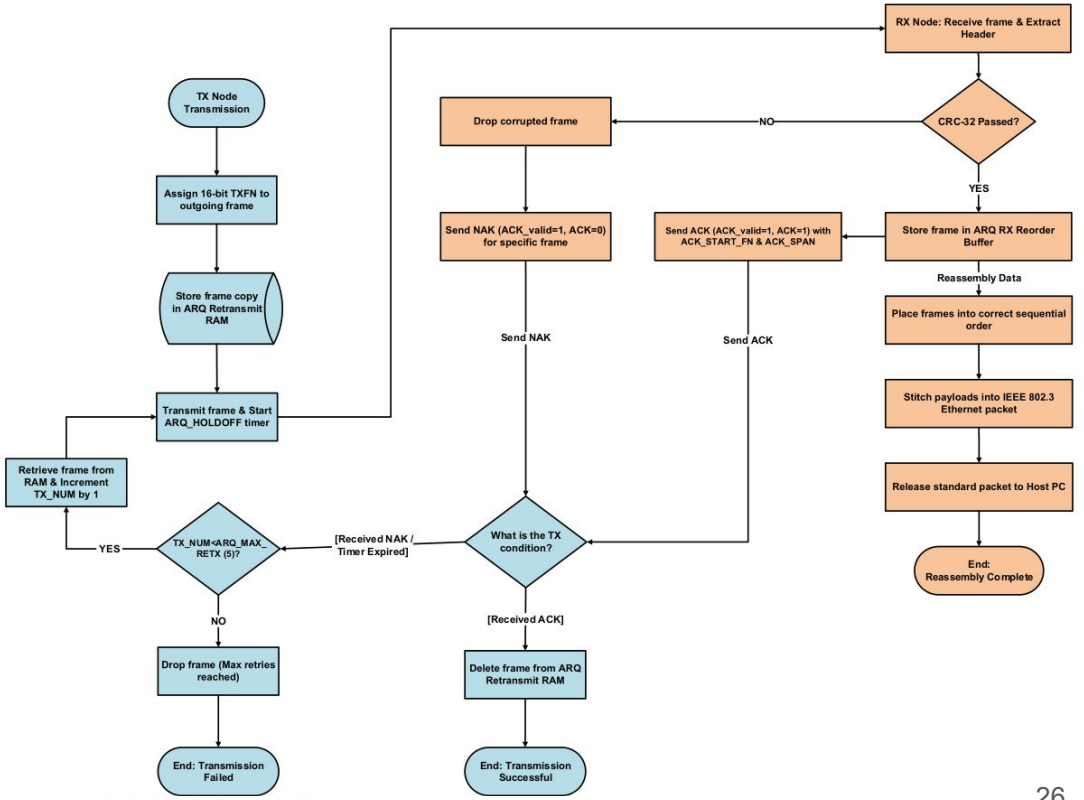
Detailed Explanation for ARQ

ARQ Selective Repeat

The initial development of the Automatic Repeat Request (ARQ) system was based on **SDA OISL Standard v2.1.2**, particularly **for the frame header structure, ARQ configuration parameters, and acknowledgement (ACK/NAK) signaling.**

During further development, **SDA OCT Standard v4.0.0 was adopted as an additional technical reference to clarify the operational aspects of ARQ**, including frame identification, retransmission control, buffer management, and in-order frame delivery.

Reference -> SDA OCT with SDA OISL Parameter



ARQ Selective Repeat (2)

2.6.3.3.3 Automatic Repeat Request (ARQ)

Header fields supporting implementation of the Automatic Repeat Request modem feature are grouped as the "ARQ" fields in Table 2-9. The ARQ configuration parameters shall remain static for the duration of a session. The ARQ configuration shall only be changed prior to the onset of acquisition. ARQ operation shall be commanded from the ground. ARQ is expected to be used for Space-to-Ground links and may be used for other links.

Only DATA and MGMT frames are be subject to ARQ. IDLE frames shall not be subject to ARQ. The receive terminal is only required to ACK frames received correctly. Successful reception of a frame is defined as the payload CRC-32 passing. Terminals shall treat frames for which an ACK is not explicitly received as NAK'd.

The automatic repeat request scheme shall be configured with the set of parameters in Table 2-11. When operating with ARQ "on," the receive terminal shall support these configurable ARQ parameters.

Table 2-11: ARQ Parameters

Parameter	Valid Range	Number of bits	Description
ARQ_HOLDOFF_NFRAMES	0,16,32,...4080 N=16 *(0...2^8-1)	8	Window size of ARQ shall be from 0 to (2^8)-1. The maximum value is 16 times 2^8-1=4080. The value of ARQ_HOLDOFF_NFRAMES determines the hold off time: e.g. $256 \times 80\ \mu s$ = 20 ms ARQ HOLDOFF TIME for 312.5 Mbps. Window size and hold-off time are not set independently.
ARQ_MAX_RETX	0-5	3	Maximum number of retransmission attempts.

For ARQ_HOLDOFF_NFRAMES, the range of values is the range required by the interface and does not represent the range of values required to be implemented.

ARQ shall be disabled by setting the number of re-transmission attempts to zero.

The ACK field in the Frame Header (Table 2-9) shall be ignored unless ACK-valid is set to one. When ACK-valid is one, the ACK shall apply to the frame with sequence number RXFN.

The implementation shall buffer received frames such that, after the maximum number of retransmission attempts has been exhausted or all frames have been ACK'd, correctly received FSO frames shall be released to the Ethernet reassembly block (Section 2.6.3.7.7) in order. FSO frames for which an ACK was never received shall be dropped. This scheme produces in-order delivery of Ethernet packets to the space vehicle for all values of ARQ (including settings where ARQ is disabled).

3.1 Theoretical Capacity and Memory Calculation

To maintain a continuous 10 Gbps transmission rate without stalling (Stop-and-Wait), the TX ARQ Retransmit RAM must be capable of buffering all FSO frames currently "in flight" until their respective ACK or NAK responses are received from the far-end terminal.

For a target satellite **Line-of-Sight (LoS) distance of 5,000 km,** the physical memory requirement is dictated by the speed of light and the frame duration:

- **Propagation Delay (1-Way):**

$$\begin{aligned} Propagation\ Delay &= Target\ distance\ /\ Speed\ of\ light\ in\ vacuum \\ &= 5,000\ km\ /\ 300,000\ km/s. \\ &= 16.67\ ms \end{aligned}$$

- **Round-Trip Time (RTT):**

$$\begin{aligned} Round\ Trip\ Time\ (RTT) &= Propagation\ Delay\ x\ 2 \\ &= 16.67\ ms\ \times\ 2 \\ &\approx 33.34\ ms \end{aligned}$$

Accounting for internal FPGA processing latency ($1.66\ ms$), the total turnaround time is rounded to **35 ms**.

- **Frame Duration:** According to SDA OISL Table 2-4 and SDA OCT V4.0.0 standard (Table 3-20), at a 10 Gbps line rate, a single FSO frame is transmitted in $2.494\ \mu s$.

- **Frames in Flight:**

$$\begin{aligned} Frames\ in\ Flight &= RTT\ /\ Frame\ Duration \\ &= 35\ ms\ /\ 2.494\ \mu s \\ &\approx 14,033\ frames \end{aligned}$$

Since each frame payload contains 1912 bytes, the **minimum required buffer size** is:

- **Required RAM Capacity:**

$$\begin{aligned} Required\ RAM\ Capacity &= Frames\ in\ Flight\ \times\ Size\ Payload \\ &= 14,033\ frames\ \times\ 1912\ bytes \\ &\approx 26.8\ Megabytes\ (MB) \text{ (or roughly 214.4 Megabits).} \end{aligned}$$

Project Plan

Sprint	Reporting Date	Focus
Sprint 1	August 28, 2026	Architecture Baseline & Initial Modules (System Arch, RS BRAM, Preamble, XGMII Base, Packaging)
Sprint 2	September 11, 2026	XGMII IPG & GTH Transceiver Foundation (IPG Handling, Packing Finale, GTH Clocking, Reset, 64b/66b Sync)
Sprint 3	September 25, 2026	Documentation & Theory Spike (Arch Refinement, Project Guideline Doc, 10ns Gap Theory, ARQ Conceptual FSM)
Sprint 4	October 9, 2026	TX Pipeline & Encoder (XGMII TX & FSO Framer Int., ARQ TX Controller, TX CDC/FEC/LPC, GTH TX)
Sprint 5	October 23, 2026	RX Pipeline, Simulation & ETRI Demo (GTH RX, ARQ RX, End-to-End Simplex Sim, ILA Setup, Live Demo)
Sprint 6	November 6, 2026	Python BER Ecosystem & Validation (Python Automation, RS(255,239) Limit Validation)
Sprint 7	November 20, 2026	Full-Duplex Prototyping (Full-Duplex Integration, 24H Stress Test)
Sprint 8	December 1, 2026	Final Submission (Arch/Schematic Doc, ARQ Viability Report, ETRI Handoff)

Sprint	Focus (Summary)	Status	Start	End	August		September				October					November				December
					W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W5	W1	W2	W3	W4	W1
Sprint 1	Architecture Baseline & Initial Modules (System Arch, RS BRAM, Preamble, XGMII Base, Packaging)	Completed	Aug 17	Aug 28																
Sprint 2	XGMII IPG & GTH Transceiver Foundation (IPG Handling, Packing Finale, GTH Clocking, Reset, 64b/66b Sync)	Completed	Aug 31	Sep 11																
Sprint 3	Documentation & Theory Spike (Arch Refinement, Project Guideline Doc, 10ns Gap Theory, ARQ Conceptual FSM)	Completed	Sep 14	Sep 25																
Sprint 4	TX Pipeline & Encoder (XGMII TX & FSO Framer Int., ARQ TX Controller, TX CDC/FEC/LPC, GTH TX)	Completed	Sep 28	Oct 9																
Sprint 5	RX Pipeline, Simulation & ETRI Demo (GTH RX, ARQ RX, End-to-End Simplex Sim, ILA Setup, Live Demo)	Completed	Oct 12	Oct 23																
Sprint 6	Python BER Ecosystem & Validation (Python Automation, RS(255,239) Limit Validation)	Planned	Oct 26	Nov 6																
Sprint 7	Full-Duplex Prototyping (Full-Duplex Integration, 24H Stress Test)	Planned	Nov 9	Nov 20																
Sprint 8	Final Submission (Arch/Schematic Doc, ARQ Viability Report, ETRI Handoff)	Planned	Nov 23	Dec 1																

Future Plan (28 September 2026)

Feature / Metric	SDA OISL V2.1.2 Standard (Current)	SDA OCT V4.0.0 Standard (Next-Gen. US Standard)	ESA ESTOL v3.1 Standard (European Standard)
Physical Optical Signaling Rate	10 Gbps (10000 Mbaud physical optical link). (Ref: SDA OISL Sec. 2.6.3)	10 Gbps (10000 Mbaud physical optical link via GTH). (Ref: SDA OCT V4.0.0 Sec. 3.3)	10 Gbps (11.3 Gbaud physical optical link). (Ref: ESTOL v3.1 Sec. 3.5.1)
Net Effective Data Rate (Throughput)	6.1 Gbps max net effective throughput due to LPC and RS overhead. (Ref: SDA OISL Sec. 2.4, Table 2-4)	Up to 1.17 Gbps (PL_RATE 4 high protection) / 1.73 Gbps (PL_RATE 2). (Trade-off: High redundancy/coding gain for 5000 km ultra-long distance S2S links) LDPC provides a very massive additional coding gain, about 3 to 4 dB higher than Reed-Solomon, approaching the Shannon Limit (maximum physical limit of communication) (Ref: SDA OCT V4.0.0 Sec. 3.4.9, Table 3-20)	Up to 10.0 Gbps net user data rate upper bound (leaner frame without LPC). (Ref: ESTOL v3.1 Sec. 3.5.1)
Payload FEC Algorithm	Reed-Solomon RS(255, 239) with interleaving depth 8. Ref: SDA OISL Sec. 2.6.3.6.3)	5G NR LDPC (Base Graph 1) with variable code rates. (Ref: SDA OCT V4.0.0 Sec. 3.4.5.3)	Reed-Solomon RS(255, 239) with interleaving depth 8. (Ref: ESTOL v3.1 Sec. 3.5.3, ESTOL-OOK-43110)
Header FEC	Reed-Solomon RS(32, 16). (Ref: SDA OISL Sec. 2.6.3.6.4)	Convolutional Code 1/6. (Ref: SDA OCT V4.0.0 Sec. 3.4.5.2)	Convolutional Code 1/6. (Ref: ESTOL v3.1 Sec. 3.5.3, ESTOL-OOK-43120)

Future Plan for SCIE Paper (28 September 2026)

Tentative Title	Journal Name	IF	JCR IF %	Rating	Submission Date	WiComAI Lab Team
FPGA-Based Edge AI Acceleration Across Kintex, Alinx, and Zynq Platforms: A Comprehensive Survey and Application Case Study on Free Space Optics	IEEE Wireless communications	13.4	4 %	Q1	March 2027	FPGA
AM-Tone-Assisted Beam Acquisition for Optical Inter-Satellite Links Under Pointing Uncertainty	IEEE Internet of Things Journal	8.7	5.8 %	Q1	April 2027	PAT
Reliable FPGA Architectures for Free-Space Optical and Inter-Satellite Communication: A Survey of Error Control, Fault Tolerance, and Link Recovery	IEEE Internet of Things Journal	8.7	5.8 %	Q1	Sept. 2027	FPGA
Disturbance-Aware Predictive Fine-Pointing Control for Optical Inter-Satellite Communication Under Spacecraft Jitter	Alexandria Engineering Journal	6.4	7 %	Q1	Dec. 2027	PAT

Future Plan (28 September 2026)

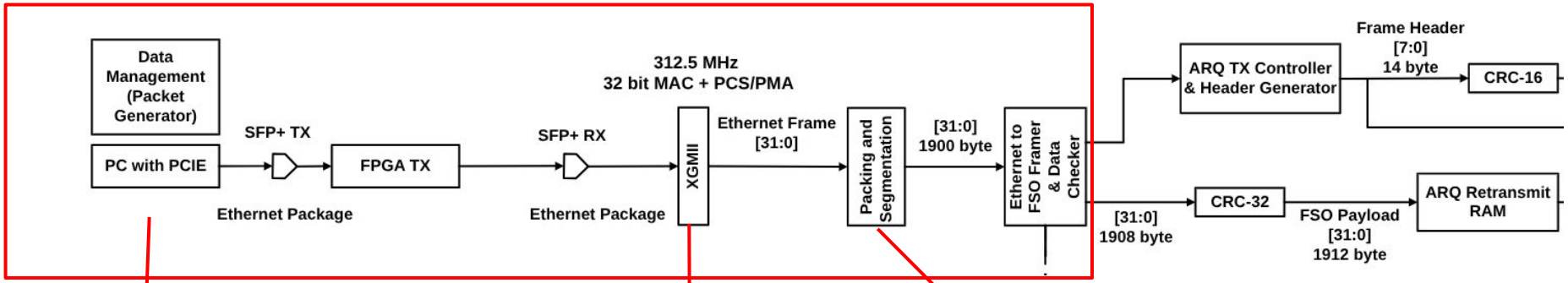
Phase	Development Content & Objectives	Theoretical Foundation	WiComAI Lab Team
Year 1 (2027) (Months 1 - 12)	**Baseband Extension Following SDA OCT:** Benchmarking alternative following the SDA OISL as the baseline with other Standards that are SDA OCT, and ESTOL v3.1 over wired SFP+.	Error Control Coding, FEC, SDA OISL, SDA OCT, ESA ESTOL	FPGA Team
Year 2 (2028) (Months 13 - 24)	**Baseband Extension Following ESA ESTOL:** Benchmarking alternative following the SDA OISL as the baseline with other Standards that are SDA OCT, and ESTOL v3.1 over wired SFP+.	Error Control Coding, FEC, SDA OISL, SDA OCT, ESA ESTOL	
Year 1 (2027) (Months 1 - 6)	**AM Tone Modulation Modeling:** Establishing PAT baseline via SDA OISL V2.1.2 before developing simulation models for the advanced 40/50 kHz AM Tone generator with dynamic MI scaling.	Amplitude Modulation, SDA OISL V.2.1.2 , SDA OCT V4.0 Standard	PAT Team
Year 1 (2027) (Months 7 - 12)	**AM Tone Demodulation Algorithms:** Formulating signal processing algorithms to accurately extract AM Tone magnitude and phase from simulated receiver data.	Signal Demodulation, Digital Signal Processing, SDA OISL V.2.1.2 , SDA OCT V4.0 Standard	
Year 2 (2028) (Months 13 - 18)	**PAT State Machine Integration:** Synthesizing modulation and demodulation models to build the autonomous spatial acquisition and tracking sequence.	Control Theory, SDA OISL V.2.1.2 PAT Protocol, SDA OCT V4.0 Standard PAT Protocol	
Year 2 (2028) (Months 19 - 24)	**System Simulation & Evaluation:** Evaluating the end-to-end tracking lock performance under simulated S2S link conditions and generating final reports.	Optical Beam Tracking, System Optimization, SDA OISL V.2.1.2 , SDA OCT V4.0 Standard	

Detailed System Explanation

Explanation of all the system by the module with detail

New Module for XGMII TX –
XGMII System Architecture

XGMII - 32 bit Testing



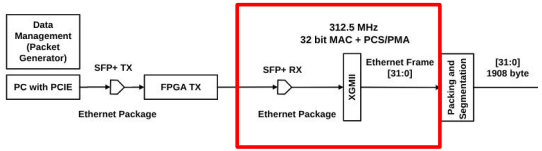
We used packet generator to send ethernet data with jumbo frame (2000 byte data)

XGMII 32-bit Interface @ 312.5 MHz
- Receives 10-Gbps Ethernet data from MAC/PCS-PMA
- Transfers **32 bits (4 bytes) per clock cycle** at 312.5 MHz -> 10 Gbps
- Detects and removes **Ethernet Preamble (7 bytes) and SFD (1 byte)**
- Extracts the valid Ethernet frame for the

Packaging and segmentation of ethernet frames to conform to the FSO frame format following SDA OISL. Ethernet Frame will send 1900 Byte + 8 Byte FSO Payload Header (Following table 2-18)

Data :
Packaging threshold -> 1900 byte
Segmentation threshold -> 1900 byte

New Module for XGMII TX –
Creating IP for 10G Data Transfer on FPGA TX and RX (2 IP)



Base-R Using 64b/66b Line Encoding, Following IEEE Ethernet 802.3 Standard

IEEE Std 802.3-2022, IEEE Standard for Ethernet
SECTION FOUR

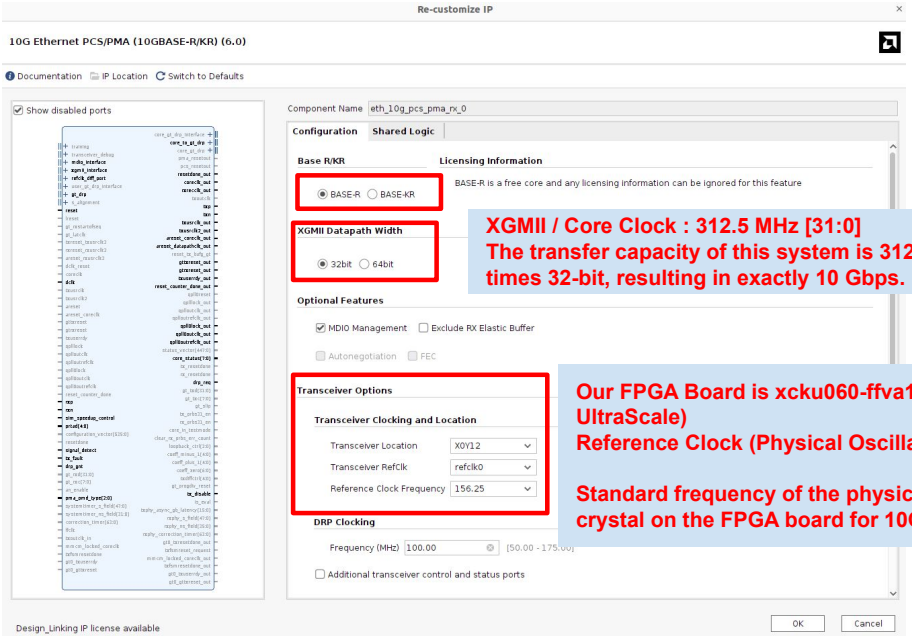
49. Physical Coding Sublayer (PCS) for 64B/66B, type 10GBASE-R

49.1 Overview

49.1.1 Scope

This clause specifies the Physical Coding Sublayer (PCS) that is used in the family of 10GBASE-R Physical Layer devices (PHYs): 10GBASE-SR, 10GBASE-LR, 10GBASE-ER, 10GBASE-LRM, and 10GBASE-KR. Alternatively, this PCS can connect to a WAN Interface Sublayer (WIS), which produces the 10GBASE-W encoding (10GBASE-R encoded data stream encapsulated into frames compatible with SONET and SDH networks) for transport by the 10GBASE-W Physical Layer devices: 10GBASE-SW, 10GBASE-LW, and 10GBASE-EW. The term 10GBASE-R is used when referring generally to Physical Layers using the PCS defined here.

The 10GBASE-R is based on a 64B/66B code. The 64B/66B code supports data and control characters, while maintaining robust error detection.



XGMII / Core Clock : 312.5 MHz [31:0]
The transfer capacity of this system is 312.5 MHz times 32-bit, resulting in exactly 10 Gbps.

Our FPGA Board is xcku060-ffva1156-2-i (Kintex UltraScale)
Reference Clock (Physical Oscillator) : 156.25 MHz

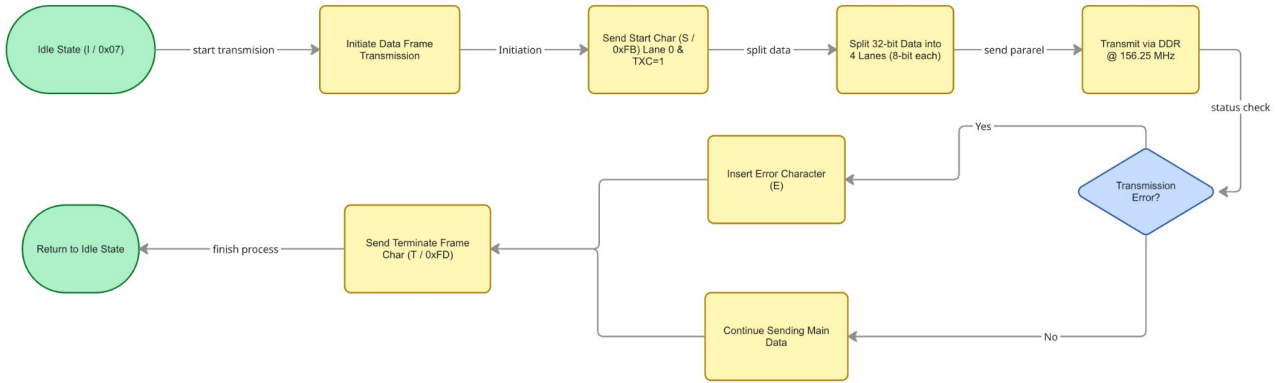
Standard frequency of the physical oscillator crystal on the FPGA board for 10G network

New Module for XGMII TX –
Creating Packet Generator Based on IEEE 802.3

Data Frame Standard Based on IEEE 802.3

Preamble (7 bytes)	Start Frame Delimiter (1 byte)	Dest. MAC Address (6 bytes)	Source MAC Address (6 bytes)	Type/Lenght (2 bytes)	Payload Data (46-1500 bytes)	Frame Check Sequence (4 bytes)

Flowchart XGMII



New Module for XGMII TX –
Creating Packet Generator Based on IEEE 802.3

XGMII Module - Trial Packet Generator Process - 2000 byte (Jumbo Frame)

eth_axis_packet_gen.v

/home/williams/MYFPGA/FPGA Final/SDAOISL/sfp_int/sfp_int_0/gt_ex/gt_ex.srcs/sources_1/new/eth_axis_packet_gen.v

```
`timescale 1ns / 1ps
/*
==========================================================================
Company      : WiComAI Lab, Kookmin University
Engineers    : Williamsyah Baginda Miraj & Giovanni Dwi
Description  : Dynamic AXI4-Stream Ethernet Packet Generator.
               Features:
               - Dynamically adjustable frame size via input port.
               - Generates standard Ethernet Headers (MAC, IPv4, UDP).
               - Appends custom payload and simulated FCS (0xDEADBEEF).
==========================================================================
*/

module eth_axis_packet_gen #(
    // --- Header Configuration ---
    parameter [47:0] DEST_MAC  = 48'hAA_BB_CC_DD_EE_FF,
    parameter [47:0] SRC_MAC   = 48'h11_22_33_44_55_66,
    parameter [31:0] SRC_IP    = {8'd192, 8'd168, 8'd10, 8'd1},
    parameter [31:0] DEST_IP   = {8'd192, 8'd168, 8'd10, 8'd2},

    // --- Custom Marker Payload ---
    parameter [31:0] PAYLOAD_WORD_1 = 32'h4B_6F_6F_6B, // ASCII "Kook"
    parameter [31:0] PAYLOAD_WORD_2 = 32'h6D_69_6E_20, // ASCII "min "
    parameter [31:0] PAYLOAD_WORD_3 = 32'h46_53_4F_20  // ASCII "FSO "
)(
    input  wire        clk,
    input  wire        rst_n,
    input  wire        trigger,          // Trigger 1 cycle HIGH to fire
    input  wire [15:0] frame_size_bytes, // Target size (e.g., 2000 for Jumbo)

    // --- AXI4-Stream Output ---
    output reg [31:0]  m_axis_tdata,
    output reg [3:0]   m_axis_tkeep,
    output reg         m_axis_tvalid,
    output reg         m_axis_tlast,
    input  wire        m_axis_tready
);

    localparam S_IDLE = 2'd0, S_DATA = 2'd1, S_DONE = 2'd2;

    reg [1:0]  state;
    reg [15:0] word_count;
    reg [15:0] total_words;

    always @(posedge clk or negedge rst_n) begin
        if (~rst_n) begin
            state        <= S_IDLE;
            m_axis_tdata  <= 32'd0;
            m_axis_tkeep  <= 4'd0;
            m_axis_tvalid <= 1'b0;
            m_axis_tlast  <= 1'b0;
            word_count    <= 16'd0;
            total_words   <= 16'd0;
        end else begin
            case (state)
                S_IDLE: begin
                    m_axis_tvalid <= 1'b0;
                    m_axis_tlast  <= 1'b0;
                    word_count    <= 16'd0;
```

Core Module for Packet Generator

Reason why we choose send jumbo frame

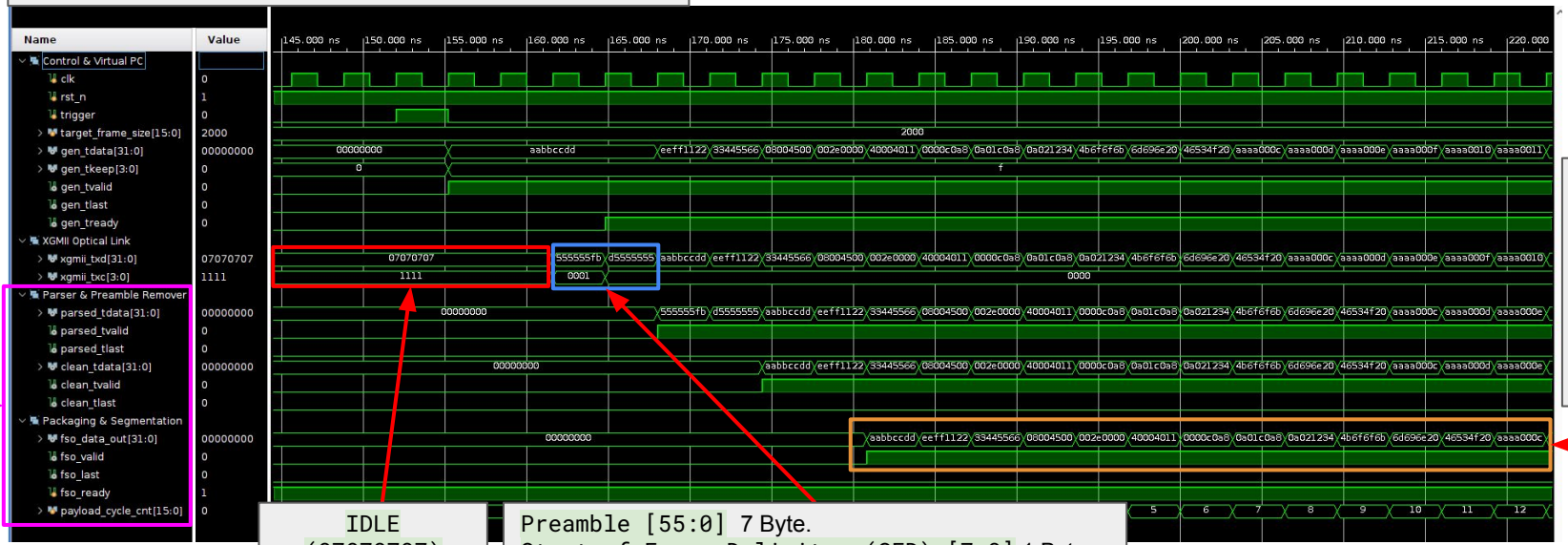
- It tests the truncation of a complete block (the first 1900 bytes).
- The remaining data (100 bytes) behaves exactly like a Mini Frame, testing the entry of Zero Padding mode.
- Back-to-back transmissions force the electrical system (XGMII) to physically create an Inter-Packet Gap (IPG), while the Segmenter is busy padding. This demonstrates the effectiveness of Elastic FIFO.

New Module - Creating XGMII Module for 312.5 MHz 32 bit for FPGA TX

XGMII Module - Trial Packet Generator Process - 2000 byte (Jumbo Frame)

(Packet Generator -> XGMII -> Parser -> Preamble Remover -> Packaging and Segmentation)

(Throughput) 312.5 * 32 bit = 10 Gbps



Frame Data – 32-bit/clock
`fso_data_out [31:0]` outputs 4 bytes of packet data every clock cycle.

IDLE (07070707)

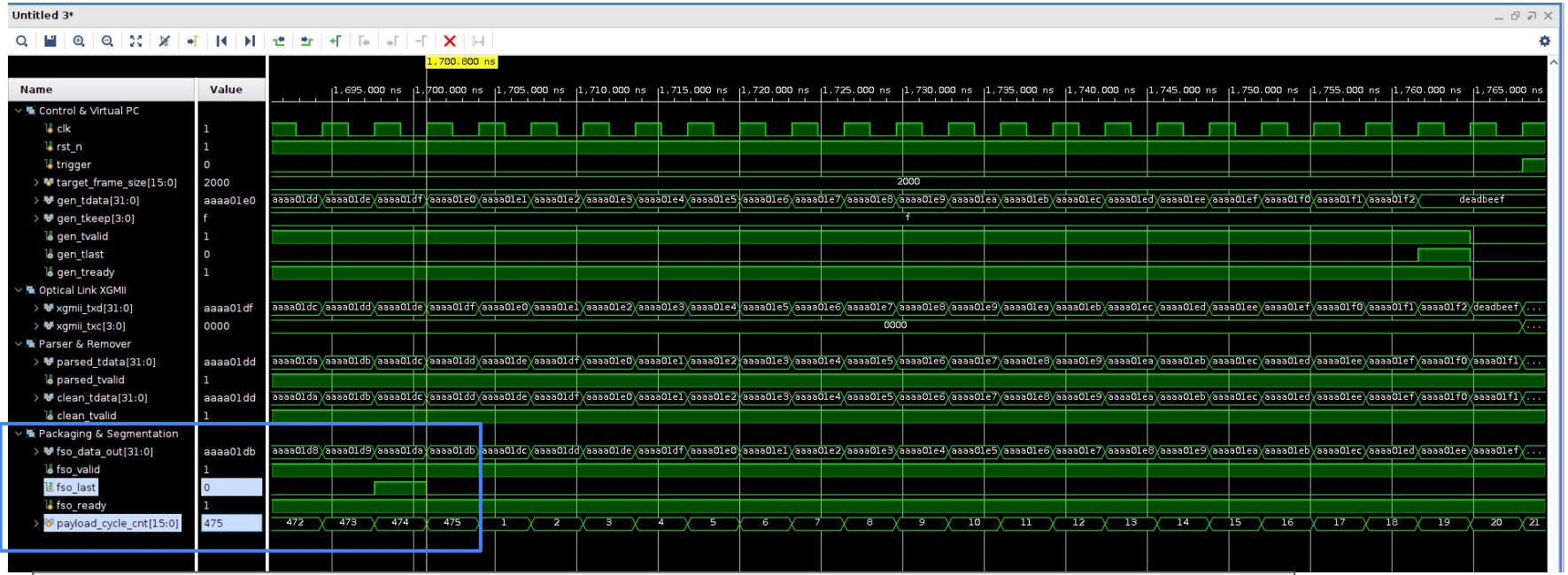
`Preamble [55:0]` 7 Byte.
`Start of Frame Delimiter (SFD) [7:0]` 1 Byte

`Parsed Data` -> Raw Data and Electrical Translator to AXI-Stream
`Clean Data` -> Discard the first 8 bytes (Preamble and SFD)
`FSO Data Out` -> Payload Data

New Module - Creating XGMII Module for 312.5 MHz 32 bit for FPGA TX

XGMII Module - Trial Packet Generator Process - 2000 byte (Jumbo Frame)

(Packet Generator -> XGMII -> Parser -> Preamble Remover -> Packaging and Segmentation)

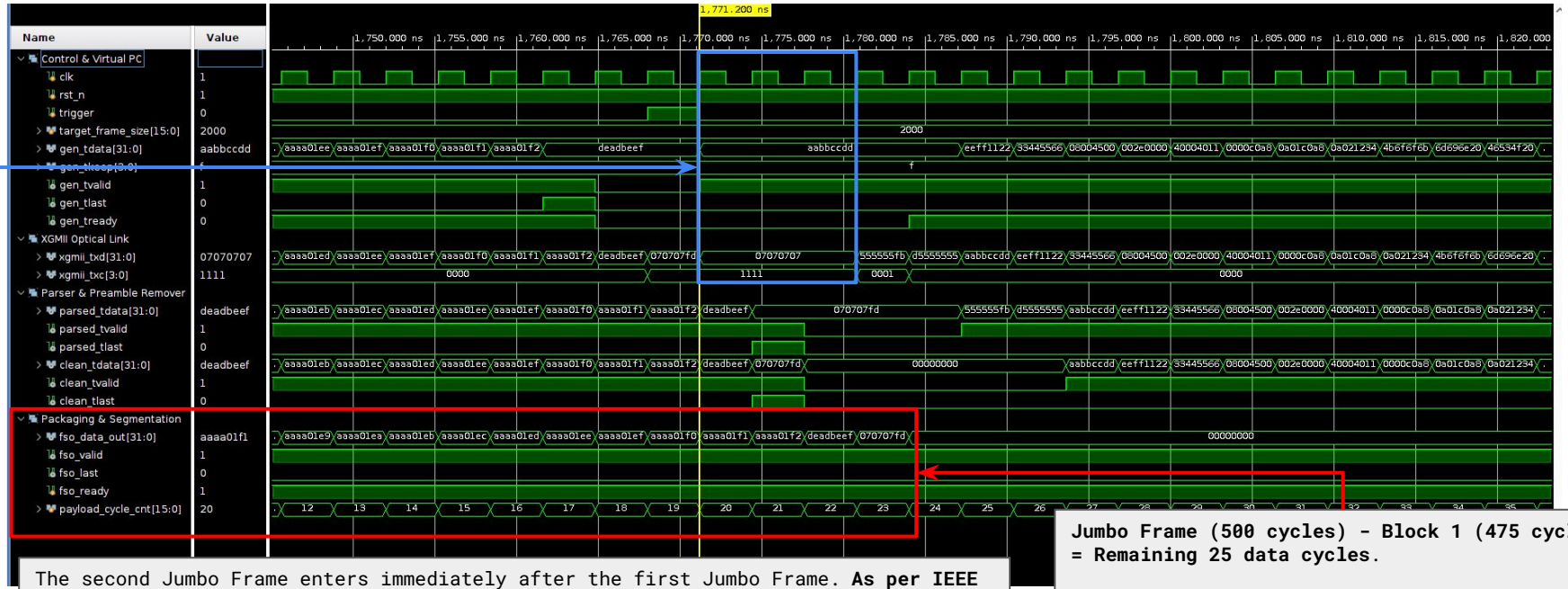


From a 2000-byte Jumbo Frame, Packaging & Segmentation Module counts exactly 475 cycles (get by 475x4=1908).
At exactly the 475th cycle, the fso_last flag is raised to notify the downstream Framer block that a 1908-byte FSO block has been completed and is ready to be packed.

New Module - Creating XGMII Module for 312.5 MHz 32 bit for FPGA TX

XGMII Module - Trial Packet Generator Process - 2000 byte (Jumbo Frame)

(Packet Generator -> XGMII -> Parser -> Preamble Remover -> Packaging and Segmentation)



The second Jumbo Frame enters immediately after the first Jumbo Frame. **As per IEEE 802.3 standards, our electrical transmitter forcibly inserts exactly 3 pure IPG clock cycles (07070707).** Our Elastic FIFO in the Segmenter absorbs this second Jumbo Frame without killing gen_tready (without backpressure), proving the system is immune to extreme asynchronous traffic.

Jumbo Frame (500 cycles) - Block 1 (475 cycles) = Remaining 25 data cycles.

The segmenter needs 475-25= 450 Zero Padding cycles to complete the second FSO Block.

New Module - Creating XGMII Module for 312.5 MHz 32 bit – Inter Packet Gap for FPGA TX and RX

The XGMII standard defines that this pause (IPG) must be filled with an Idle control character.

The Idle character in XGMII is represented by a Data value of 0x07 and a Control Bit of 1.

46.2.1 Inter-frame <inter-frame>

The inter-frame <inter-frame> period on an XGMII transmit or receive path is an interval during which no frame data activity occurs. The <inter-frame> corresponding to the MAC interpacket gap begins with the Terminate control character, continues with Idle control characters and ends with the Idle control character prior to a Start control character. The length of the interpacket gap may be changed between the transmitting MAC and receiving MAC by one or more functions (e.g., RS lane alignment, PHY clock rate compensation, or 10GBASE-W data rate adaptation functions). The minimum IPG at the XGMII of the receiving RS is five octets.

The signaling of link status information logically occurs in the <inter-frame> period (see 46.3.4). Subclause 46.3.3 describes frame processing when signaling of link status information is initiated or terminated.

Our XGMII Code

```
IPG_WAIT: begin
    xgmii_txd <= 32'h07070707; // Inter-Packet Gap (Idles)
    xgmii_txc <= 4'b1111;
    // Wait for 3 cycles of Idles (12 bytes)
    if (count == 3) state <= IDLE;
    else            count <= count + 1;
end
```

The MAC standard defines that the minimum distance between Ethernet packets (after the termination character to the start of the next packet) must be 96 bit times.

Our XGMII data line has a width of 32-bits (4 Bytes) per clock cycle.

Inter Packet Gap 12 Byte = 12 / 4 = 3 Clock Cycle.

IEEE Std 802.3-2022, IEEE Standard for Ethernet
SECTION ONE

4.4.2 MAC parameters

The parameter values shown in Table 4–2 shall be used for their corresponding MAC data rate.

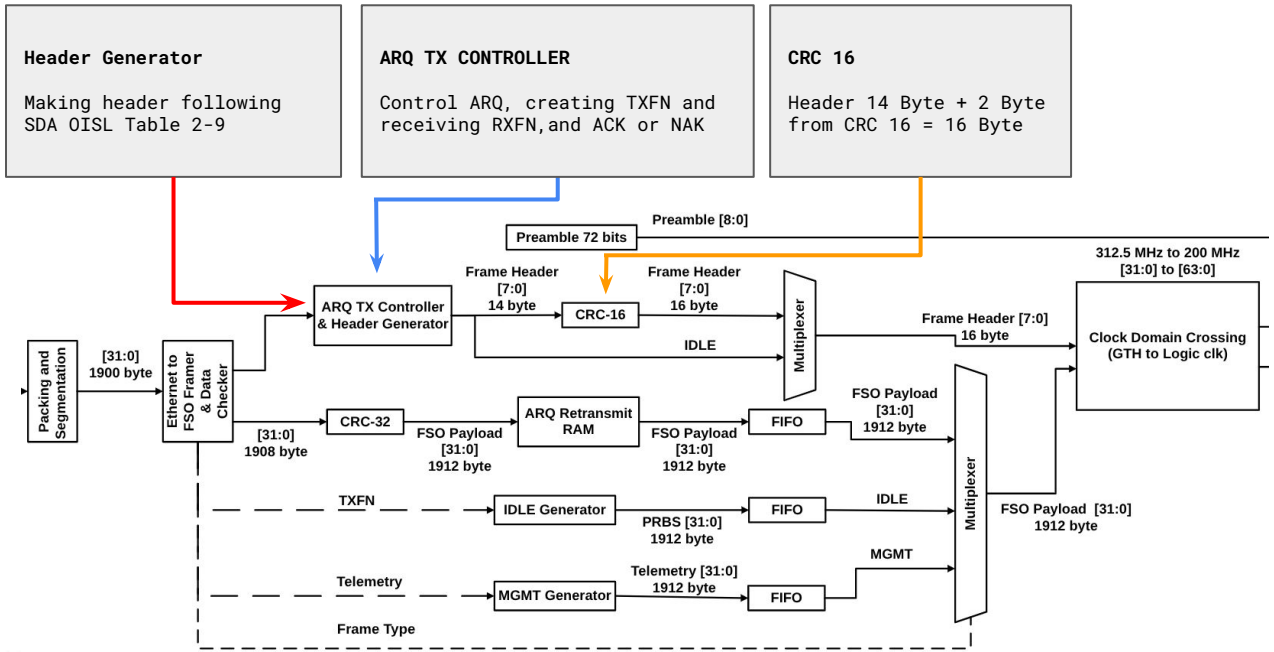
Table 4–2—MAC parameters

Parameters	MAC data rate			
	Up to and including 100 Mb/s	1 Gb/s	2.5 Gb/s, 5 Gb/s, 25 Gb/s, 40 Gb/s, 50 Gb/s, 100 Gb/s, 200 Gb/s, and 400 Gb/s	10 Gb/s
slotTime	512 bit times	4096 bit times	not applicable	not applicable
interPacketGap[a]	96 bits	96 bits	96 bits	96 bits
attemptLimit	16	16	not applicable	not applicable
backoffLimit	10	10	not applicable	not applicable
jamSize	32 bits	32 bits	not applicable	not applicable
maxBasicFrameSize	1518 octets	1518 octets	1518 octets	1518 octets
maxEnvelopeFrameSize	2000 octets	2000 octets	2000 octets	2000 octets
minFrameSize	512 bits (64 octets)	512 bits (64 octets)	512 bits (64 octets)	512 bits (64 octets)
burstLimit	not applicable	65 536 bits	not applicable	not applicable
ipgStretchRatio	not applicable	not applicable	not applicable	104 bits

[a]References to interFrameGap or interFrameSpacing in other clauses (e.g., Clause 13, Clause 35, and Clause 42) shall be interpreted as interPacketGap.

New Module for FSO Payload –
Creating Header Generator, CRC 16 and Header MUX

Core Engine for Header Module (FSO Header Frame following SDA OISL & ARQ following SDA OCT)



2.6.3.6.9 Frame Generation Summary

Construction of the OISL frame and its integration with the line product code is summarized in Figure 2-15.

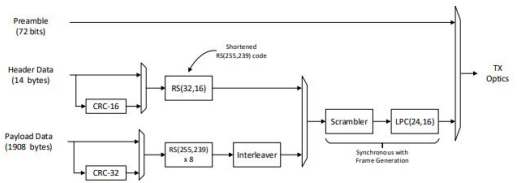


Figure 2-15: Frame Encoding Summary

2.6.3.3.1 Header Fields

The frame Header fields are listed in Table 2-9. Exactly one Header shall be present in every modem frame.

Table 2-9: Frame Header Fields

Function	Field	Bits	Description
ARQ	TXFN	16	Sequence number of this (outgoing) TX frame
	RXFN	16	Sequence number of ACK
	ACK	1	ACK (1) or NAK (0) for RXFN
	ACK-valid	1	ACK field is only valid if ACK-valid=1
MAC	FRAME_TYPE	2	00: IDLE
			01: DATA
			10: MGMT
			11: reserved
pseudo-range	TX_TS	40	TX time-stamp (frame egress)
	TOD_SECONDS	6	number of seconds in time-of-day epoch : 0-59
	TS-applies	3	TX_ TS applies to current frame (0), preceding frames (1-7)
Fast Control Channel	FCCH_OPCODE	4	time-multiplexed control signaling
	FCCH_PL	16	Payload contents depends on FCCH_TYPE
reserved		7	header size divisible by 16
CRC	CRC-16	16	See Section 2.6.3.6.1.
Total		128	

New Module for FSO Frame –
Creating Header Generator, CRC 16 and Header MUX

Mathematical Proof (Setup Phase, following table 2-9 and table 2-10)

header_generator.v

/home/williams/MYFPGA/FPGA Final/SDAOISL/sfp_int/sfp_int_0/gt_ex/gt_ex.srcs/sources_1/new/header_generator.v

```
// Assemble the 112-bit header based on Table 2-10 mapping.
// Critical: All reserved bits are forced to 1 per SDA OISL standard.
full_header_reg[7:0]     <= txfn[7:0];                                    // d0
full_header_reg[15:8]    <= txfn[15:8];                                   // d1
full_header_reg[23:16]   <= rxfn[7:0];                                    // d2
full_header_reg[31:24]   <= rxfn[15:8];                                   // d3
full_header_reg[39:32]   <= {2'b11, 2'b11, ack, ack_valid, frame_type}; // d4 (Reserved=1)
full_header_reg[47:40]   <= tx_ts[7:0];                                   // d5
full_header_reg[55:48]   <= tx_ts[15:8];                                  // d6
full_header_reg[63:56]   <= tx_ts[23:16];                                 // d7
full_header_reg[71:64]   <= tx_ts[31:24];                                 // d8
full_header_reg[79:72]   <= tx_ts[39:32];                                 // d9
full_header_reg[87:80]   <= {2'b11, tod_seconds};                         // d10 (Reserved=1)
full_header_reg[95:88]   <= {1'b1, ts_applies, fcch_opcode};              // d11 (Reserved=1)
full_header_reg[103:96]  <= fcch_pl[7:0];                                 // d12
full_header_reg[111:104] <= fcch_pl[15:8];                                // d13

is_generating <= 1'b1;
byte_counter  <= 4'd0;
header_en_out <= 1'b0; // Wait 1 cycle for stability
```

Tcl Console | Messages | Log

```
Time resolution is 1 ps
=====================================================
STARTING OISL HEADER GENERATION (SDA Table 2-10)
=====================================================
d0  | TXFN[7:0]          | 0x34 | Expected: 0x34
d1  | TXFN[15:8]         | 0x12 | Expected: 0x12
d2  | RXFN[7:0]          | 0xcd | Expected: 0xCD
d3  | RXFN[15:8]         | 0xab | Expected: 0xAB
d4  | Control & TYPE     | 0xfd | Expected: 0xFD (DATA, ACK=1)
d5  | TX_TS[7:0]         | 0x55 | Expected: 0x55
d6  | TX_TS[15:8]        | 0x44 | Expected: 0x44
d7  | TX_TS[23:16]       | 0x33 | Expected: 0x33
d8  | TX_TS[31:24]       | 0x22 | Expected: 0x22
d9  | TX_TS[39:32]       | 0x11 | Expected: 0x11
d10 | TOD_SECONDS        | 0xf2 | Expected: 0xF2 (TOD=50)
d11 | FCCH Opcode        | 0x8a | Expected: 0x8A (Opcode=A)
d12 | FCCH_PL[7:0]       | 0x88 | Expected: 0x88
d13 | FCCH_PL[15:8]      | 0x99 | Expected: 0x99
d14 | CRC-16[15:8]       | 0x0f | (Hardware MUX Appended)
d15 | CRC-16[7:0]        | 0x28 | (Hardware MUX Appended)
-----------------------------------------------------
```

Basic Architecture Rules

- **Reserved Bits Mandate:** The SDA OISL standard requires all reserved bits to be set to a logical 1 (not 0).
- **LSB-First (Little-Endian)** Transmission Architecture: For multi-byte data (such as 16-bit TXFN or 40-bit TX_TS), the least significant byte **(Least Significant Byte)** must be transmitted first.
- **Simulation Test Parameters:** TXFN = 0x1234, RXFN = 0xABCD, FRAME_TYPE = 01 (DATA), ACK/ACK_val = 1, TOD_SECONDS = 50, FCCH_OPCODE = 0xA.

d0 & d1 (TXFN Transmission 0x1234):
d0 = LSB of TXFN -> 0x34.
d1 = MSB of TXFN -> 0x12.

d2 & d3 (RXFN Transmission 0xABCD):
d2 = LSB of RXFN -> 0xCD.
d3 = MSB of RXFN -> 0xAB.

d5 to d9 (TX_TS Transmission 0x1122334455):
Reverse order from the lowest byte: d5=0x55, d6=0x44, d7=0x33, d8=0x22, d9=0x11.

d12 & d13 (FCCH Payload Transmission 0x9988):
d12 = LSB -> 0x88.
d13 = MSB -> 0x99.

2.6.3.3.1 Header Fields

The frame Header fields are listed in Table 2-9. Exactly one Header shall be present in every modem frame.

Table 2-9: Frame Header Fields

Function	Field	Bits	Description
ARQ	TXFN	16	Sequence number of this (outgoing) TX frame
	RXFN	16	Sequence number of ACK
	ACK	1	ACK (1) or NAK (0) for RXFN
	ACK-valid	1	ACK field is only valid if ACK-valid=1
MAC	FRAME_TYPE	2	00: IDLE
			01: DATA
			10: MGMT
			11: reserved
pseudo-range	TX_TS	40	TX time-stamp (frame egress)
	TOD_SECONDS	6	number of seconds in time-of-day epoch : 0-59
	TS-applies	3	TX_ TS applies to current frame (0), preceding frames (1-7)
Fast Control Channel	FCCH_OPCODE	4	time-multiplexed control signaling
	FCCH_PL	16	Payload contents depends on FCCH_TYPE
reserved		7	header size divisible by 16
CRC	CRC-16	16	See Section 2.6.3.6.1.
Total		128	

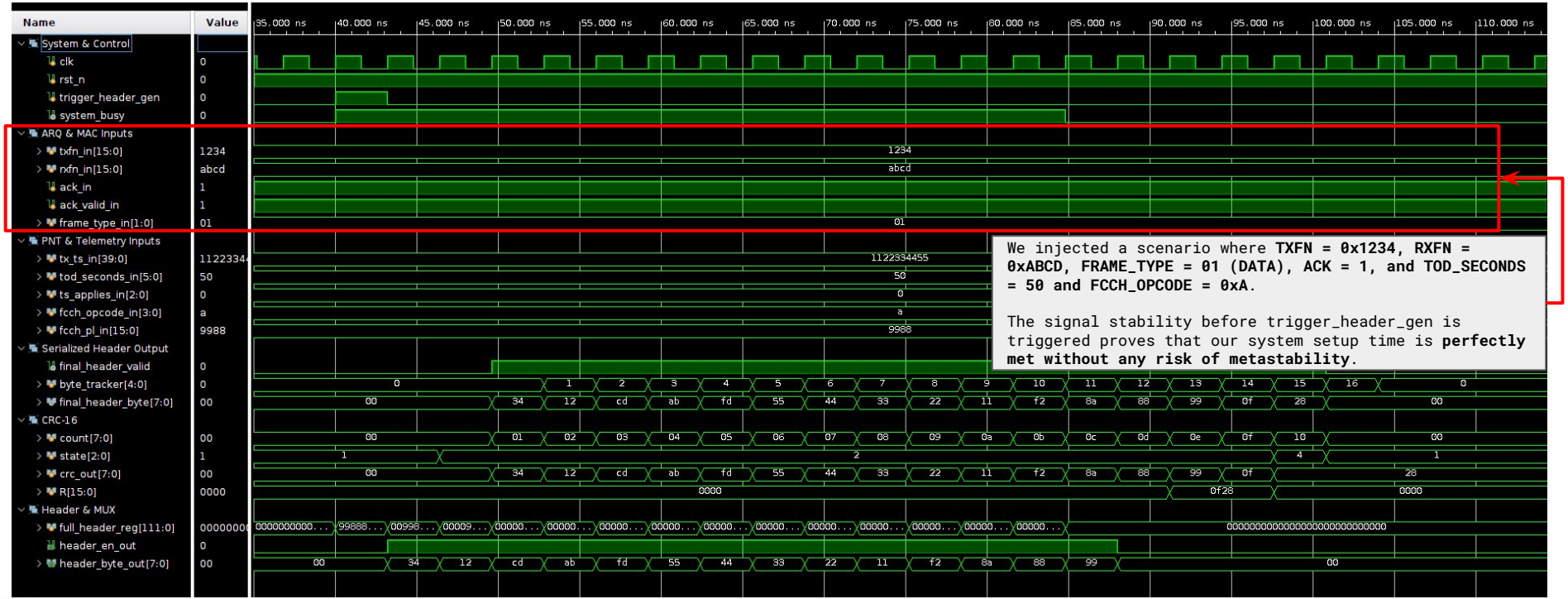
The mapping of the fields in Table 2-9 to physical locations in the modem frame is shown in Table 2-10. Here, the column headers indicate the bit number of the byte (MSB: b_7, LSB: b_0) and the row labels indicate the byte number in the header Reed-Solomon codeword. The value 1 shall be assigned to all bits indicated as reserved. A CRC-16 shall be attached to the end of the header with the bits entering the CRC circuit in natural order (byte d_0 to byte d_{13}, MSB first on each byte) and attached to the header in natural order (bytes d_{14} and d_{15}) as illustrated in Table 2-10.

Table 2-10: Mapping of Header Fields to Byte and Bit-locations in the Transmitted Header

	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
d_0	TXFN[7]	TXFN[6]	TXFN[5]	TXFN[4]	TXFN[3]	TXFN[2]	TXFN[1]	TXFN[0]
d_1	TXFN[15]	TXFN[14]	TXFN[13]	TXFN[12]	TXFN[11]	TXFN[10]	TXFN[9]	TXFN[8]
d_2	RXFN[7]	RXFN[6]	RXFN[5]	RXFN[4]	RXFN[3]	RXFN[2]	RXFN[1]	RXFN[0]
d_3	RXFN[15]	RXFN[14]	RXFN[13]	RXFN[12]	RXFN[11]	RXFN[10]	RXFN[9]	RXFN[8]
d_4	1	1	1	1	ACK	ACK-valid	FRAME_TYPE[1]	FRAME_TYPE[0]
d_5	TX_TS[7]	TX_TS[6]	TX_TS[5]	TX_TS[4]	TX_TS[3]	TX_TS[2]	TX_TS[1]	TX_TS[0]
d_6	TX_TS[15]	TX_TS[14]	TX_TS[13]	TX_TS[12]	TX_TS[11]	TX_TS[10]	TX_TS[9]	TX_TS[8]
d_7	TX_TS[23]	TX_TS[22]	TX_TS[21]	TX_TS[20]	TX_TS[19]	TX_TS[18]	TX_TS[17]	TX_TS[16]
d_8	TX_TS[31]	TX_TS[30]	TX_TS[29]	TX_TS[28]	TX_TS[27]	TX_TS[26]	TX_TS[25]	TX_TS[24]
d_9	TX_TS[39]	TX_TS[38]	TX_TS[37]	TX_TS[36]	TX_TS[35]	TX_TS[34]	TX_TS[33]	TX_TS[32]
d_{10}	1	1	TOD_SECONDS[5]	TOD_SECONDS[4]	TOD_SECONDS[3]	TOD_SECONDS[2]	TOD_SECONDS[1]	TOD_SECONDS[0]
d_{11}	1	TS-applies[2]	TS-applies[1]	TS-applies[0]	FCCH_OPCODE[3]	FCCH_OPCODE[2]	FCCH_OPCODE[1]	FCCH_OPCODE[0]
d_{12}	FCCH_PL[7]	FCCH_PL[6]	FCCH_PL[5]	FCCH_PL[4]	FCCH_PL[3]	FCCH_PL[2]	FCCH_PL[1]	FCCH_PL[0]
d_{13}	FCCH_PL[15]	FCCH_PL[14]	FCCH_PL[13]	FCCH_PL[12]	FCCH_PL[11]	FCCH_PL[10]	FCCH_PL[9]	FCCH_PL[8]
d_{14}	CRC-16[15]	CRC-16[14]	CRC-16[13]	CRC-16[12]	CRC-16[11]	CRC-16[10]	CRC-16[9]	CRC-16[8]
d_{15}	CRC-16[7]	CRC-16[6]	CRC-16[5]	CRC-16[4]	CRC-16[3]	CRC-16[2]	CRC-16[1]	CRC-16[0]

New Module for FSO Frame –
Creating Header Generator, CRC 16 and Header MUX

Asynchronous Parameter Assimilation (Setup Phase, following table 2-9 and table 2-10)



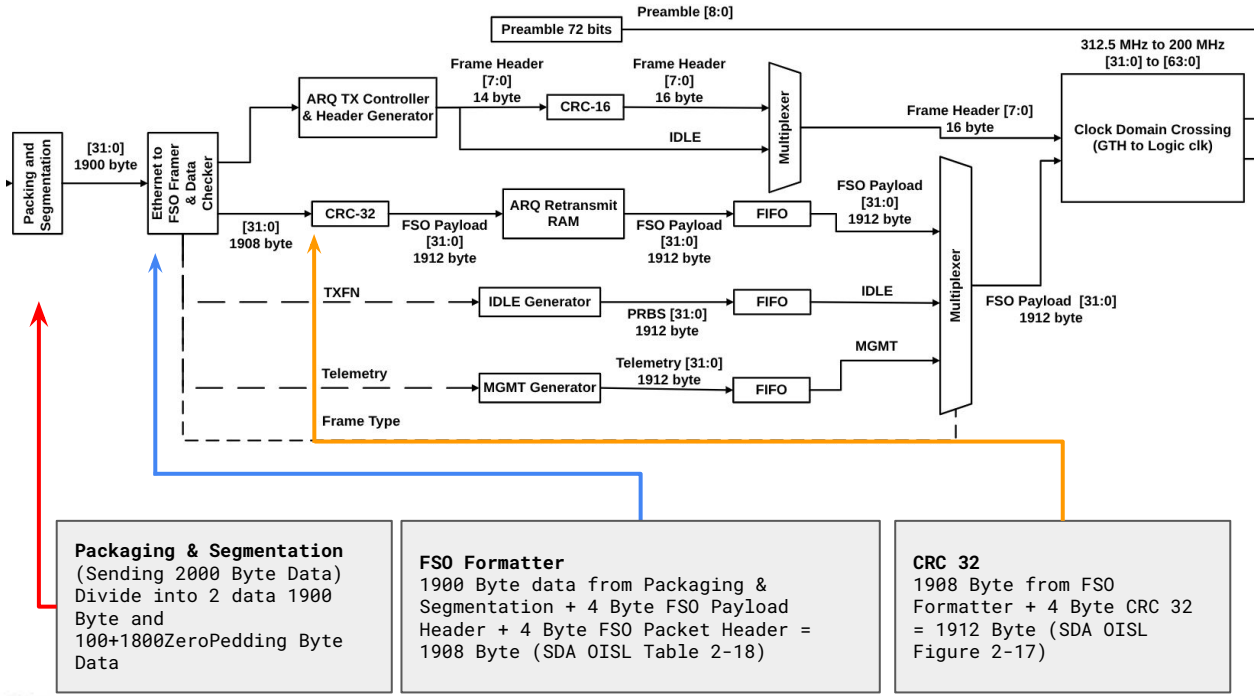
New Module for FSO Frame –
Creating Header Generator, CRC 16 and Header MUX

Core Engine for Header (Setup Phase, following table 2-9 and table 2-10)



New Module for FSO Frame –
Creating Payload Module, CRC 32, ARQ, FIFO & MUX

Core Engine for FSO Payload (FSO Payload Frame following SDA OISL & ARQ following SDA OCT)
Testing Sending 2000 Byte data



Packaging & Segmentation
(Sending 2000 Byte Data)
Divide into 2 data 1900 Byte and 100+1800ZeroPedding Byte Data

FSO Formatter
1900 Byte data from Packaging & Segmentation + 4 Byte FSO Payload Header + 4 Byte FSO Packet Header = 1908 Byte (SDA OISL Table 2-18)

CRC 32
1908 Byte from FSO Formatter + 4 Byte CRC 32 = 1912 Byte (SDA OISL Figure 2-17)

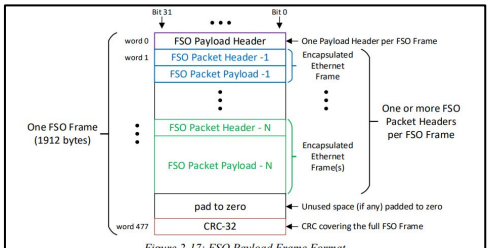


Figure 2-17: FSO Payload Frame Format

The FSO Payload and FSO Frame headers split Ethernet frame payloads across multiple frames, enabling efficient packing of Ethernet frames into FSO frames.

2.6.3.7.5 FSO Payload and Packet Headers

Every FSO frame shall start with an FSO Payload Header. There shall be exactly one FSO Payload Header (32 bits) per FSO frame. The contents of the FSO frame and packet headers shall follow the definition presented in Table 2-18.

Table 2-18: FSO Payload and Packet Headers

	Field	bit numbers	#bits
FSO Payload Header (32 bits)	0xAB	31:24	8
	seq_num	23:14	10
	Length	13:0	14
FSO Packet Header (32 bits)	0xCDEF	31:16	16
	reserved	15:14	2
	Length	13:0	14

2.6.3.5.3 DATA/IDLE/MGMT Frame Handling

2.6.3.5.3.1 Multiplexing

The modem shall multiplex the modem frame types (FRAME_TYPE, Table 2-9) for transmission as illustrated in Figure 2-6.

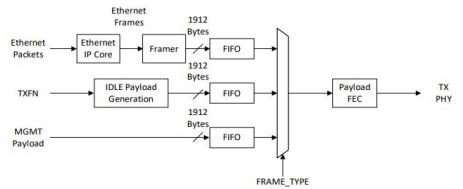
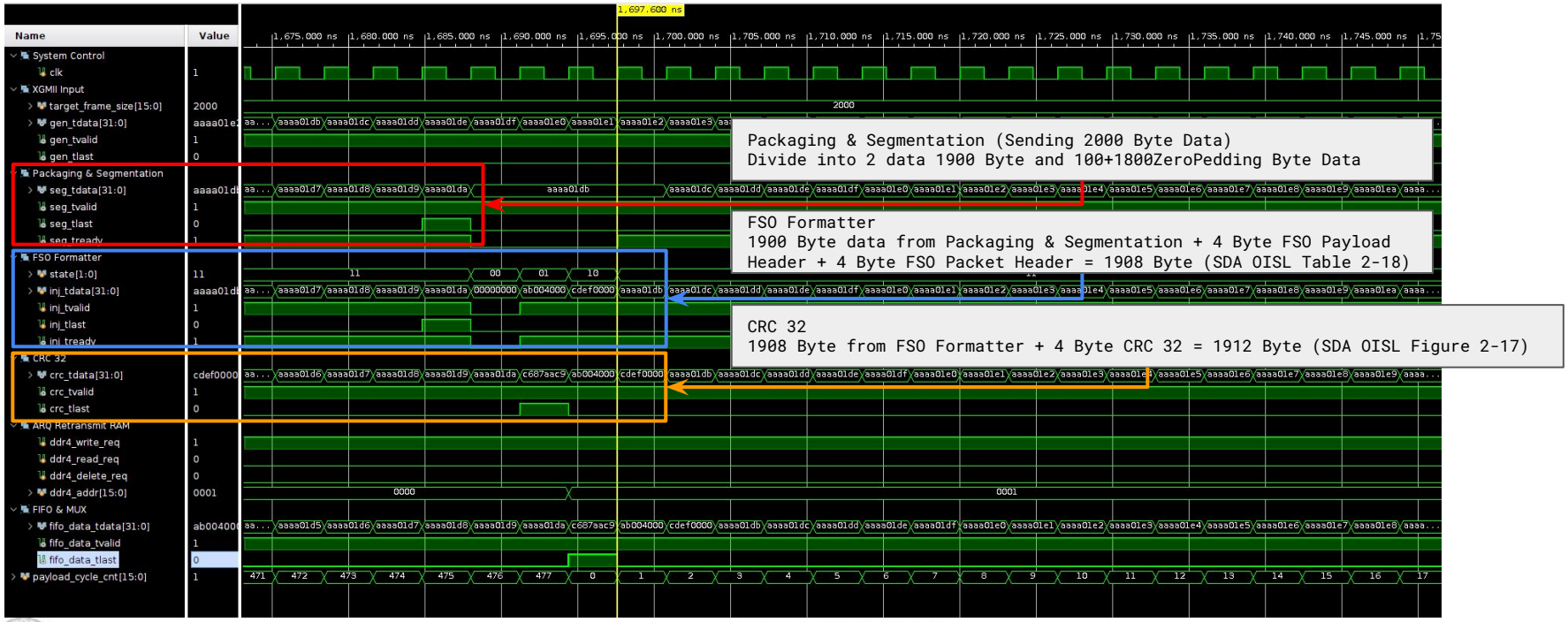


Figure 2-6: Multiplexing Frame Types

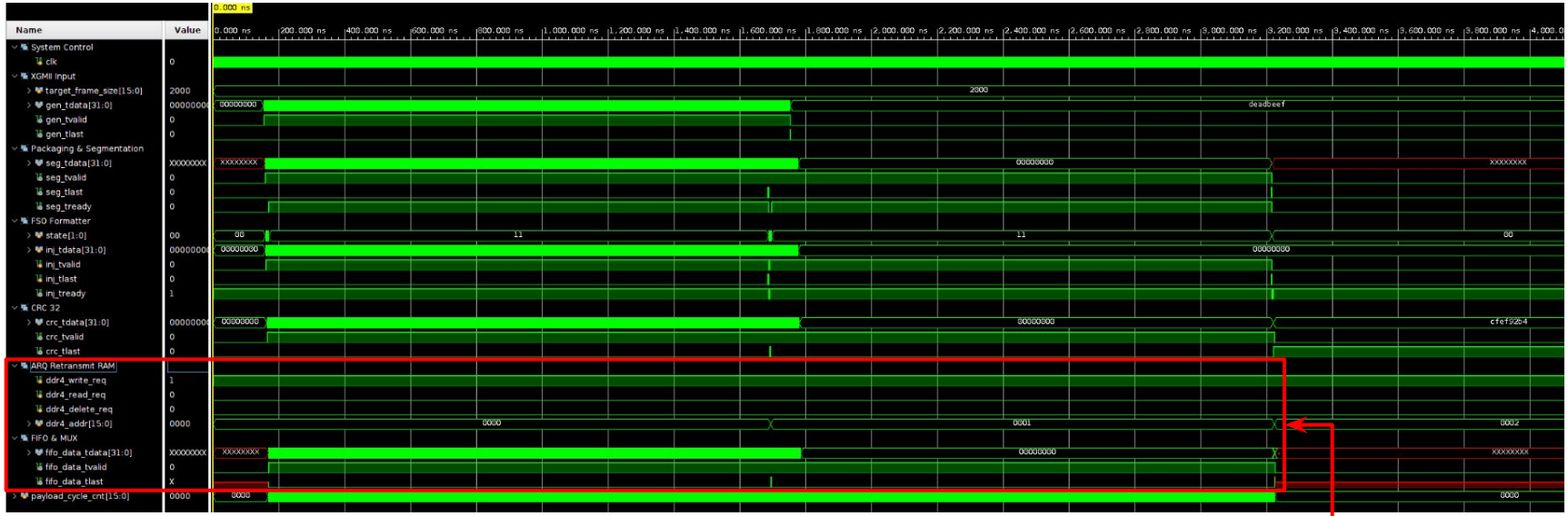
New Module for FSO Frame –
Creating Payload Module, CRC 32, ARQ, FIFO & MUX

Core Engine for FSO Payload (FSO Payload Frame following SDA OISL & ARQ following SDA OCT)
Testing Sending 2000 Byte data



New Module for FSO Frame –
Creating Payload Module, CRC 32, ARQ, FIFO & MUX

Core Engine for FSO Payload (FSO Payload Frame following SDA OISL & ARQ following SDA OCT)
Testing Sending 2000 Byte data
(Divide into 2 data 1900 Byte and 100+1800ZeroPedding Byte Data)

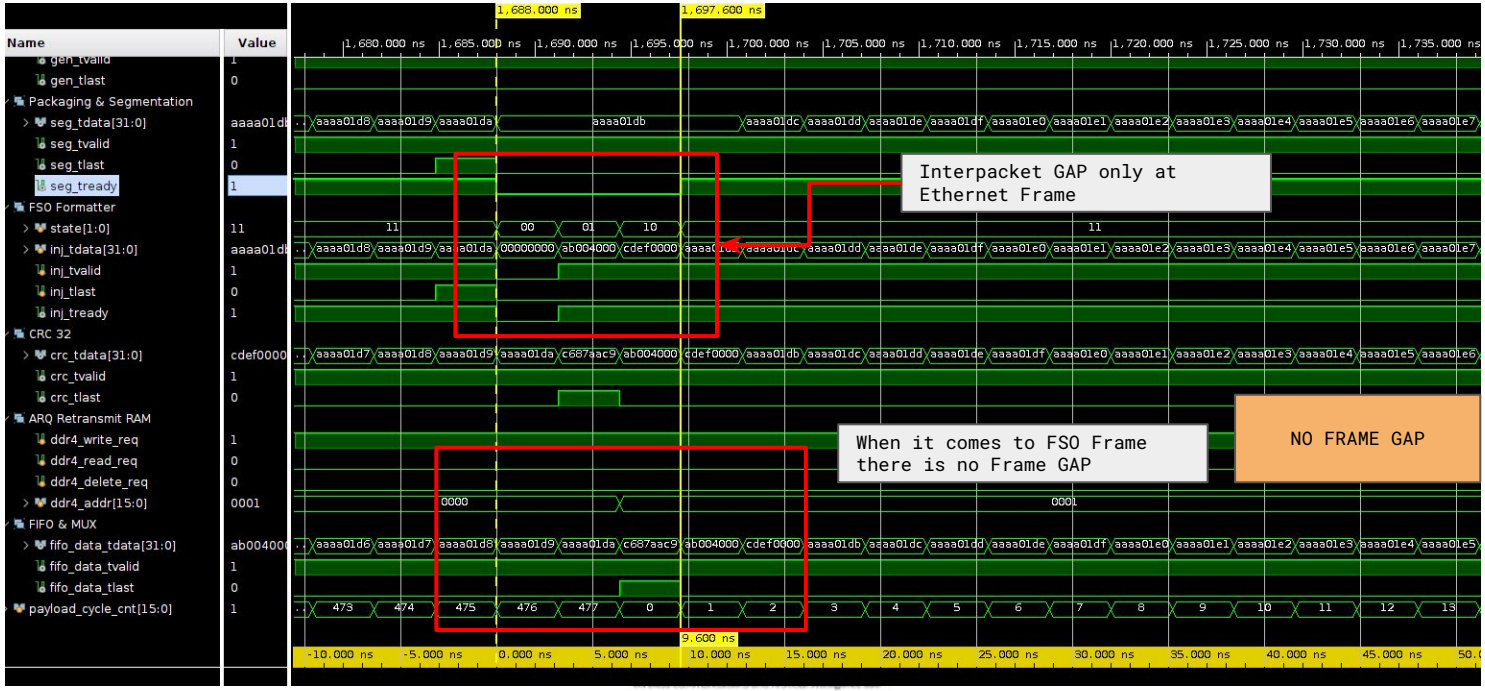


Dual Storage (FIFO & ARQ)

ARQ RAM -> Storage all the Payload Data and repair if NAK
FIFO -> for sending the data to the next FSM (Payload MUX)

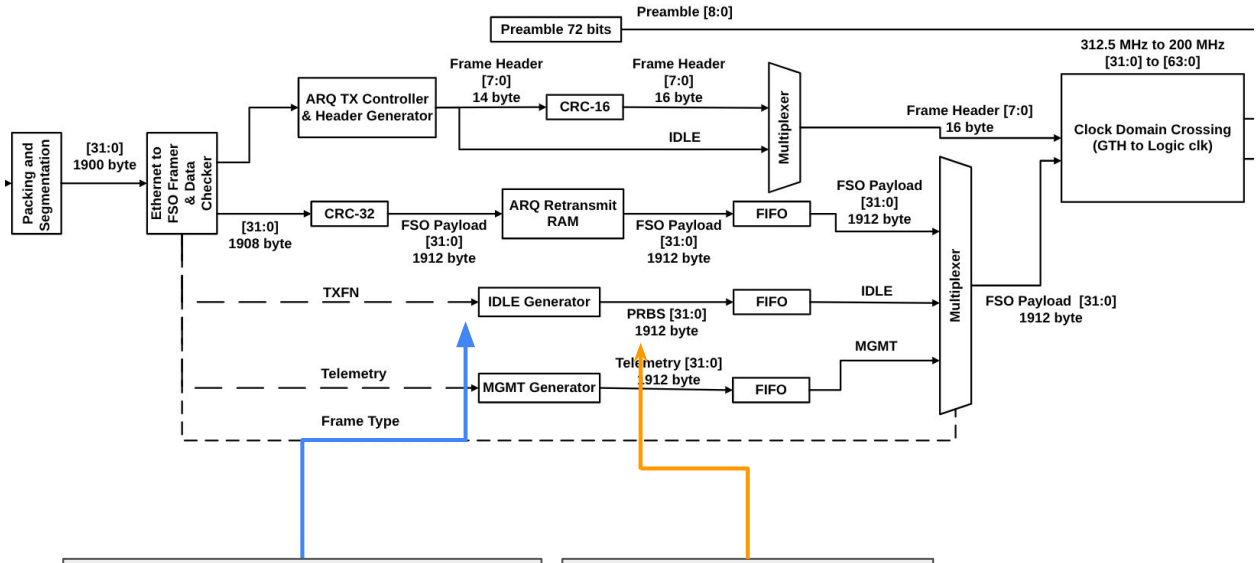
New Module for FSO Frame –
Creating Payload Module, CRC 32, ARQ, FIFO & MUX

Core Engine for FSO Payload (FSO Payload Frame following SDA OISL & ARQ following SDA OCT)
Testing Sending 2000 Byte data
(Divide into 2 data 1900 Byte and 100+1800ZeroPedding Byte Data)



New Module for FSO Frame –
Creating IDLE Module, CRC 32, FIFO & MUX

Core Engine for FSO IDLE Payload (FSO IDLE Payload Frame following SDA OISL)



IDLE Generator(1908 Byte)

Following SDA OISL 2.6.3.5.1.2 that IDLE frames shall be constructed as a pseudo-random binary sequence(PRBS) using the same generator as the frame scrambler

IDLE GENERATOR WITH CRC 32

1908 Byte from FSO IDLE + 4 Byte CRC 32 = 1912 Byte (SDA OISL 2.6.3.5.1.2)

2.6.3.5.1.2 Payload Construction

Modem IDLE frames shall be subject to the same payload FEC and shall be protected by the same CRC-32 as the data frames.

The information payload in IDLE frames shall be constructed as a pseudo-random binary sequence (PRBS) using the same generator as the frame scrambler (Section 2.6.3.6.4) except with its initial seed equal to the value of the frame header TXFN field (Table 2-9) plus one.

2.6.3.5.1 IDLE Frames

By design the modem always transmits frames with no gap between adjacent frames. In the case that data is not available for transmission the modem shall insert consecutive IDLE frames into the transmitted stream until other types of traffic frames become available. There shall not be a gap between frames under any circumstance.

2.6.3.5.3 DATA/IDLE/MGMT Frame Handling

2.6.3.5.3.1 Multiplexing

The modem shall multiplex the modem frame types (FRAME_TYPE, Table 2-9) for transmission as illustrated in Figure 2-6.

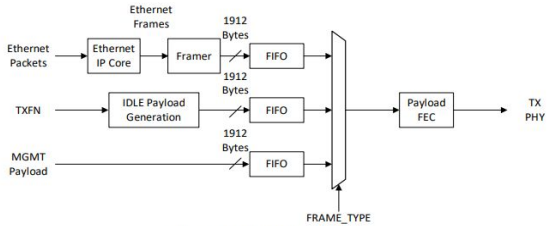
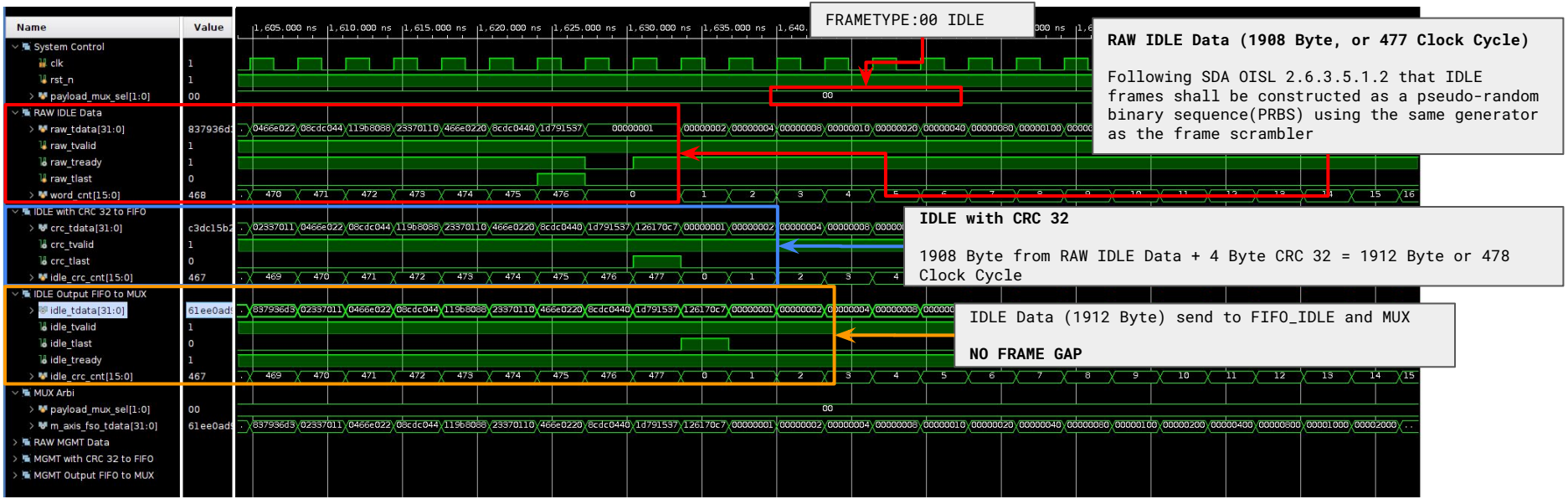


Figure 2-6: Multiplexing Frame Types

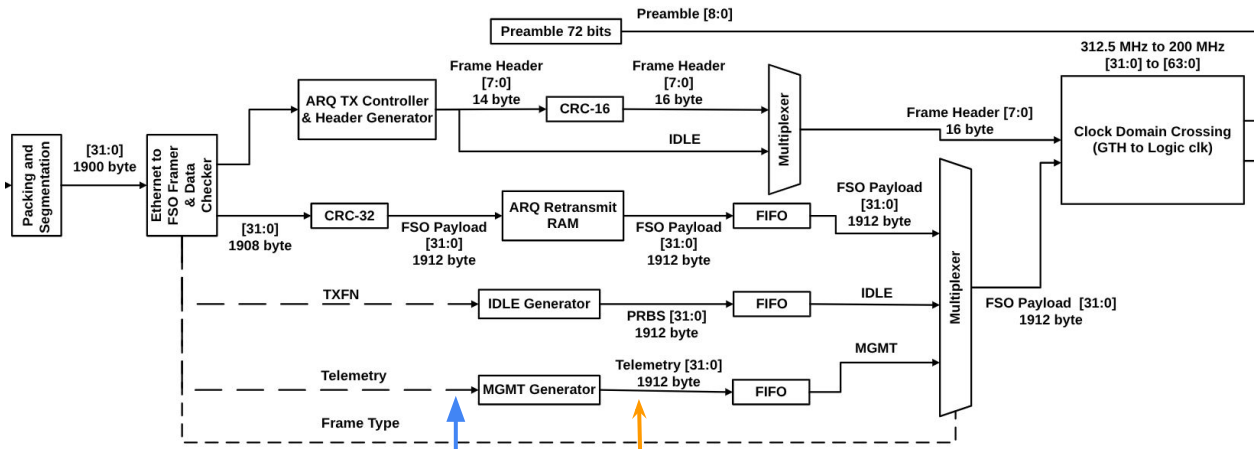
New Module for FSO Frame –
Creating IDLE Module, CRC 32, FIFO & MUX

Core Engine for FSO IDLE Payload



New Module for FSO Frame –
Creating MGMT Module, CRC 32, FIFO & MUX

Core Engine for FSO MGMT Payload (FSO MGMT Payload Frame following SDA OISL)



MGMT Generator(1908 Byte)

Following SDA OISL 2.6.3.5.2.3 that The MGMT/PNT Frames are used to communicate OISL frame timestamps and calculated range and range timestamp between the OISLs. Table 2-14 defines the MGMT/PNT payload.

MGMT GENERATOR WITH CRC 32

Starting from Padding 229 bits to 256 bits (32 bytes) MGMT + 1876 Byte Zero Padding + 4 Byte CRC 32 = 1912 Byte

2.6.3.5.2.2 Payload Construction

Modem MGMT frames shall be subject to the same payload FEC and are protected by the same CRC-32 as the data frames. The size of the MGMT information payload shall be the same as for data frames.

The bytes comprising the MGMT frame payload shall be written into the modem frame in the (common) order described in Section 2.6.3.5.3.2 and illustrated in Figure 2-7.

2.6.3.5.2.3 Payload Frame Definition

The MGMT/PNT Frames are used to communicate OISL frame timestamps and calculated range and range timestamp between the OISLs. Table 2-14 defines the MGMT/PNT payload.

Table 2-14: MGMT/PNT Frame Payload Definition for Ranging Timestamps.

Field	Bits	Bit Positions	Description
RX_PHY_Timestamp_sec	6	0-5	RX PHY timestamp for frame receipt Seconds of the whole GPS minute (0 Hour, January 6, 1980)
RX_PHY_Timestamp_picosec	40	6-45	RX PHY timestamp for frame receipt Picoseconds of the whole second
TX_PHY_Timestamp_sec	6	46-51	TX PHY timestamp for the same frame associated with the RX PHY Timestamp Seconds of the whole GPS minute (0 Hour, January 6, 1980)

2.6.3.5.3 DATA/IDLE/MGMT Frame Handling

2.6.3.5.3.1 Multiplexing

The modem shall multiplex the modem frame types (FRAME_TYPE, Table 2-9) for transmission as illustrated in Figure 2-6.

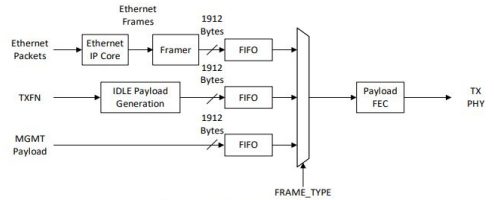


Figure 2-6: Multiplexing Frame Types

New Module for FSO Frame –
Creating MGMT Module, CRC 32, FIFO & MUX

Core Engine for FSO MGMT Payload
(FSO MGMT Payload Frame following SDA OISL)

2.6.3.5.2 MGMT/PNT Frames

The MGMT frame type is provided for management frames. These frames shall be used for inter-OISL communications.

MGMT frames **shall** be sent at 1Hz, each holding 1 time-stamp measurement set.

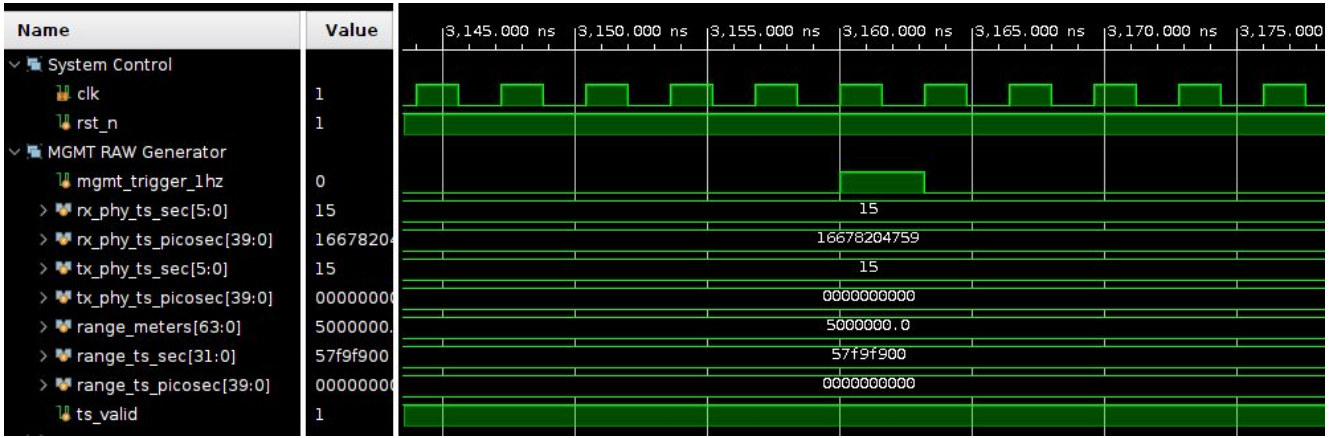


Table 2-14: MGMT/PNT Frame Payload Definition for Ranging Timestamps.

Field	Bits	Bit Positions	Description
RX_PHY_Timestamp_sec	6	0-5	RX PHY timestamp for frame receipt Seconds of the whole GPS minute (0 Hour, January 6, 1980)
RX_PHY_Timestamp_picosec	40	6-45	RX PHY timestamp for frame receipt Picoseconds of the whole second
TX_PHY_Timestamp_sec	6	46-51	TX PHY timestamp for the same frame associated with the RX PHY Timestamp Seconds of the whole GPS minute (0 Hour, January 6, 1980)
TX_PHY_Timestamp_picosec	40	52-91	TX PHY timestamp for the same frame associated with the RX PHY Timestamp Picoseconds of the whole second
Range_Meters	64	92-155	Computed and provided TX OISL aperture to RX OISL aperture range in meters, IEEE 754 double precision floating point format. This field must be populated. Calculation of this field is optional - If not calculated, this field shall be set to 0 (zero).
Range_TimeStamp_sec	32	156-187	Computed range timestamp: whole seconds of GPS time from epoch 0hour, Jan 6, 1980. Calculation of this field is optional - If not calculated, this field shall be set to 0 (zero).
Range_TimeStamp_picosec	40	188-227	Computed range timestamp: picoseconds of the whole second of GPS time Calculation of this field is optional - If not calculated, this field shall be set to 0 (zero).
Timestamp_Valid	1	228	1 = range estimate is valid; 0 = range estimate is not valid.

New Module for FSO Frame –
Creating MGMT Module, CRC 32, FIFO & MUX

Core Engine for FSO MGMT Payload
(FSO MGMT Payload Frame following SDA OISL)



Time of Flight Mathematics

To reach a receiving satellite 5,000 km away, the laser beam requires an absolute travel time (Time of Flight / ToF).

- Speed of Light (c) = 299.792.458 meter/second
- Transmission Distance (d) = 5.000.000 meter
- Counting Seconds:

  $ToF = \frac{d}{c} = \frac{5.000.000}{299.792.458} \approx 0,016678204759\ Second$

- Resolution Conversion Picoseconds $1\ second = 10^{12}\ Picoseconds$

  $ToF = 0,016678204759\ x\ 10^{12} = 16.678.204.759\ Picoseconds$

Range Meter = 5000 KM

confirms that the IEEE 754 Double Precision (64-bit) format is perfectly captured by our architecture. The simulated optical link (Space-to-Space) distance is set at an extreme distance of 5,000 kilometers (5,000,000 meters), which is the target specification for the Cross-Plane orbit SDA constellation.

New Module for FSO Frame –
Creating MGMT Module, CRC 32, FIFO & MUX

Core Engine for FSO MGMT Payload
(FSO MGMT Payload Frame following SDA OISL)

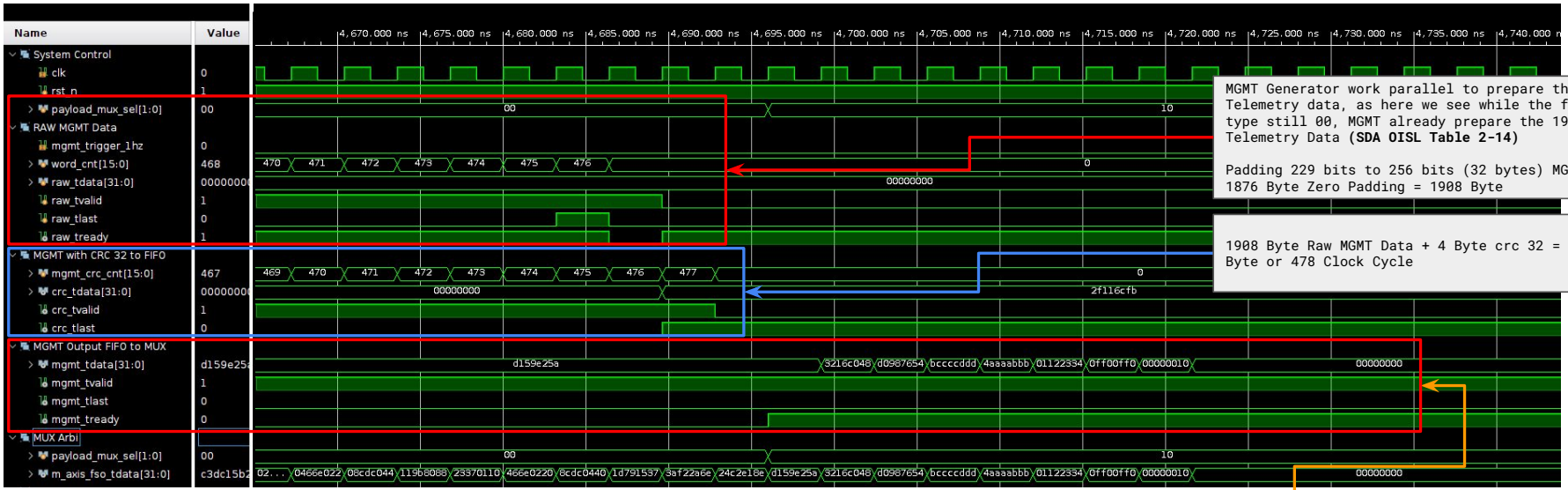


Range_ts_sec = 57f9f900 (Hexadecimal) or 1.476.000.000 in decimal

The satellite system uses the GPS Epoch (starting on January 6, 1980). The number 1,476,000,000 seconds represents an increase of approximately 46.77 years since that Epoch on this simulation test (October 9,2026)

New Module for FSO Frame –
Creating MGMT Module, CRC 32, FIFO & MUX

Core Engine for FSO MGMT Payload



MGMT Generator work parallel to prepare the Telemetry data, as here we see while the frame type still 00, MGMT already prepare the 1908 Byte Telemetry Data **(SDA OISL Table 2-14)**

Padding 229 bits to 256 bits (32 bytes) MGMT + 1876 Byte Zero Padding = 1908 Byte

1908 Byte Raw MGMT Data + 4 Byte crc 32 = 1912 Byte or 478 Clock Cycle

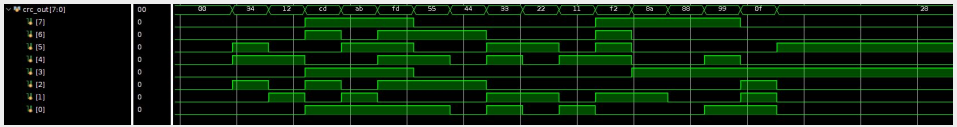
As we can see MGMT Store to FIFO_MGMT.

Starting from Padding 229 bits to 256 bits (32 bytes) MGMT + 1876 Byte Zero Padding + 4 Byte CRC 32 = 1912 Byte

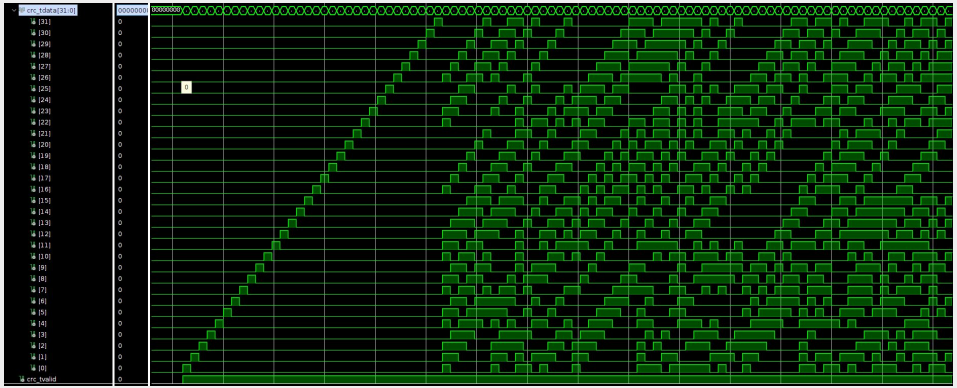
Update Module for FSO Frame –
Update Module for Cyclic Redundancy Check

Core Engine for CRC 16 & CRC 32

CRC 16 [7:0] -> Location Frame Header. 14 Byte Data from Header Generator + 2 Byte CRC 16 = 16 Byte



CRC 32 [31:0] -> Location Frame Payload. 1908 Byte Data from FSO Payload + 4 Byte CRC 32 = 1912 Byte



We follow SDA OISL 2.6.3.6.1

2.6.3.6.1 CRC

Two CRC's are required to generate the modem frame: a CRC-16 protects the frame header bits while a CRC-32 protects the payload bits. A functional description of a generic L-bit CRC encoder circuit is shown in Figure 2-8. The circuit consists of an L-bit register (L = 16 for CRC-16 and L = 32 for CRC-32). The connection polynomials (i.e., values g_k) are defined in Table 2-15 for each of the two CRC's required to create the modem frame.

Calculation of the CRC for a block of k payload bits shall be performed as follows:

- Initialize the L-bit encoder register state to zero at the start of each new CRC calculation
- Clock in the k payload bits with switches S_1 and S_2 both in the down position
- After the last payload bit has been loaded the encoder register contains the L-bit CRC value. It can be clocked-out with switches S_1 and S_2 in the up position.

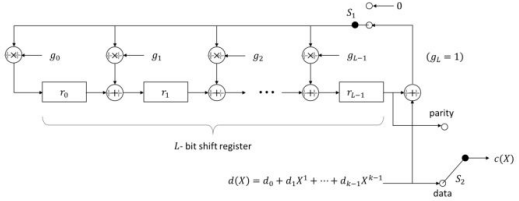


Figure 2-8: L-bit CRC Calculation Circuit

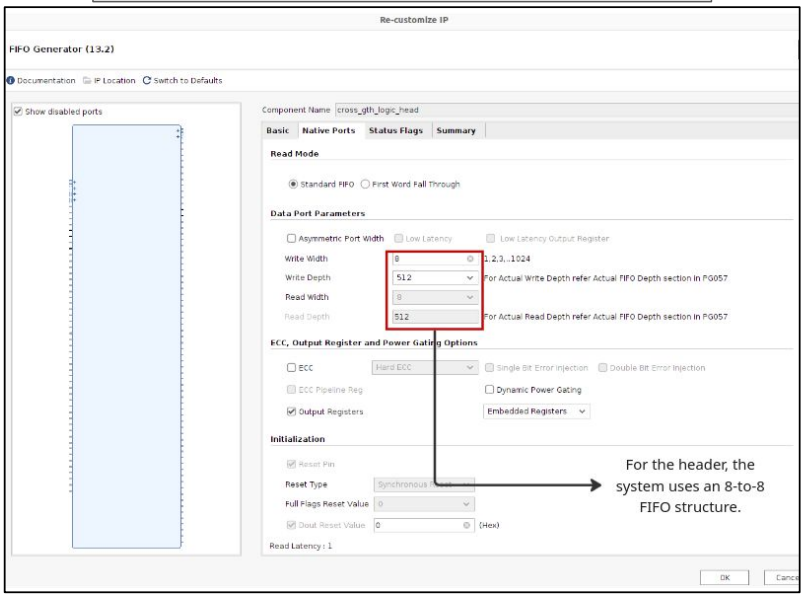
Table 2-15: CRC Polynomials

Location	Length	Polynomial	Source
Frame Header	16 bits	$g(x) = x^{16} + x^{12} + x^5 + 1$	1
Frame Payload	32 bits	$g(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	1

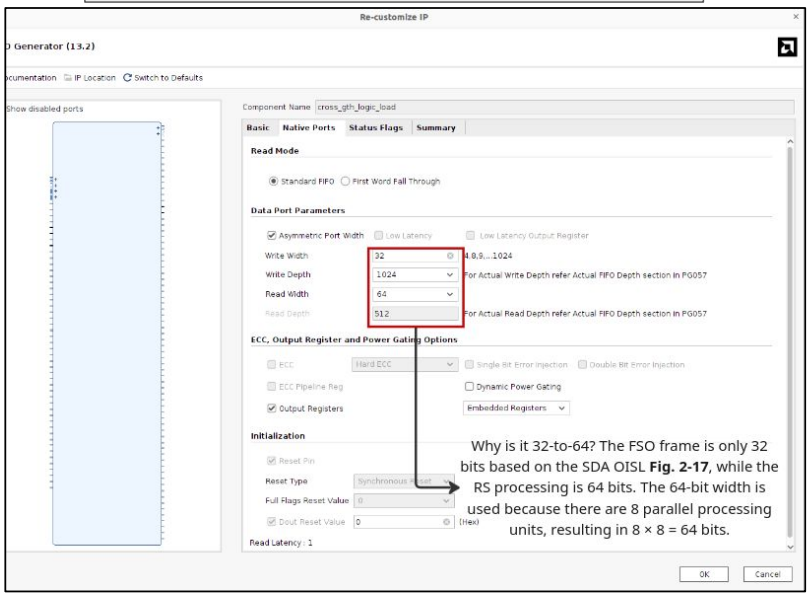
Update Module for Clock Domain Crossing 312.5 MHz to 200 MHz – Update Module for CDC Header and CDC Payload

FIFO and Clock Domain Crossing

FIFO and Clock Domain Crossing for FSO Header



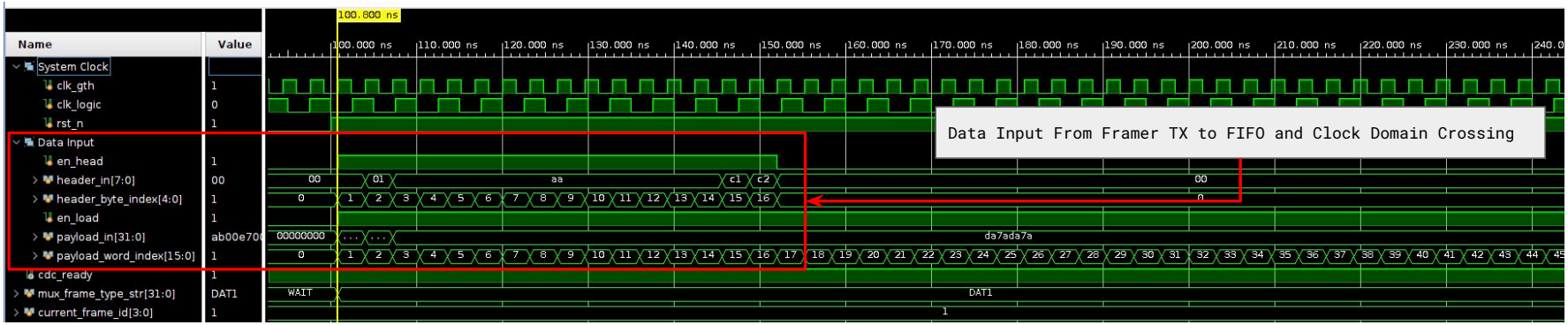
FIFO and Clock Domain Crossing for FSO Payload

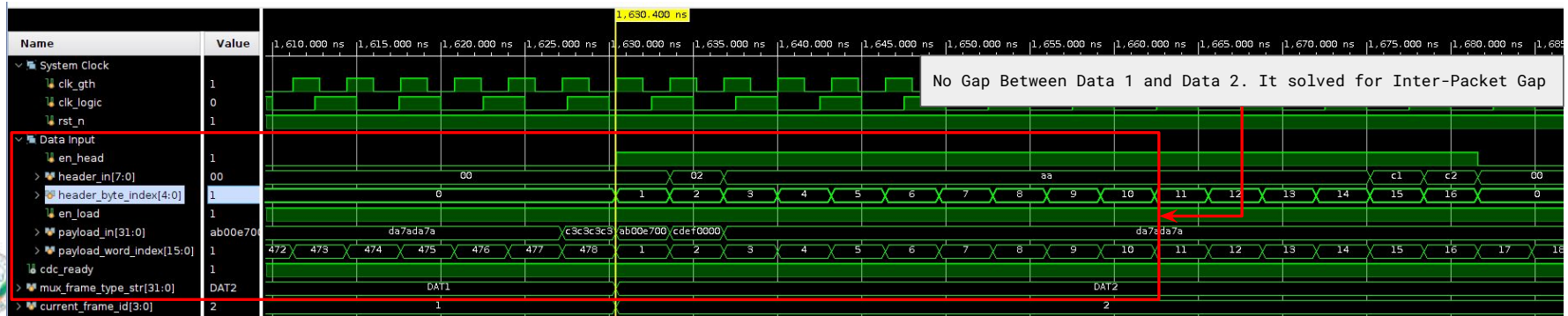


Input Side (Framer to CDC): The speed is 312.5 MHz × 32 bit = 10 Gbps 312.5 MHz × 32-bit = 10 Gbps.
Output Side (CDC to FEC): The speed is 200.0 MHz × 64 bit = 12.8 Gbps 200.0 MHz × 64-bit = 12.8 Gbps.

Update Module for FSO Frame –
Update Module for CRC

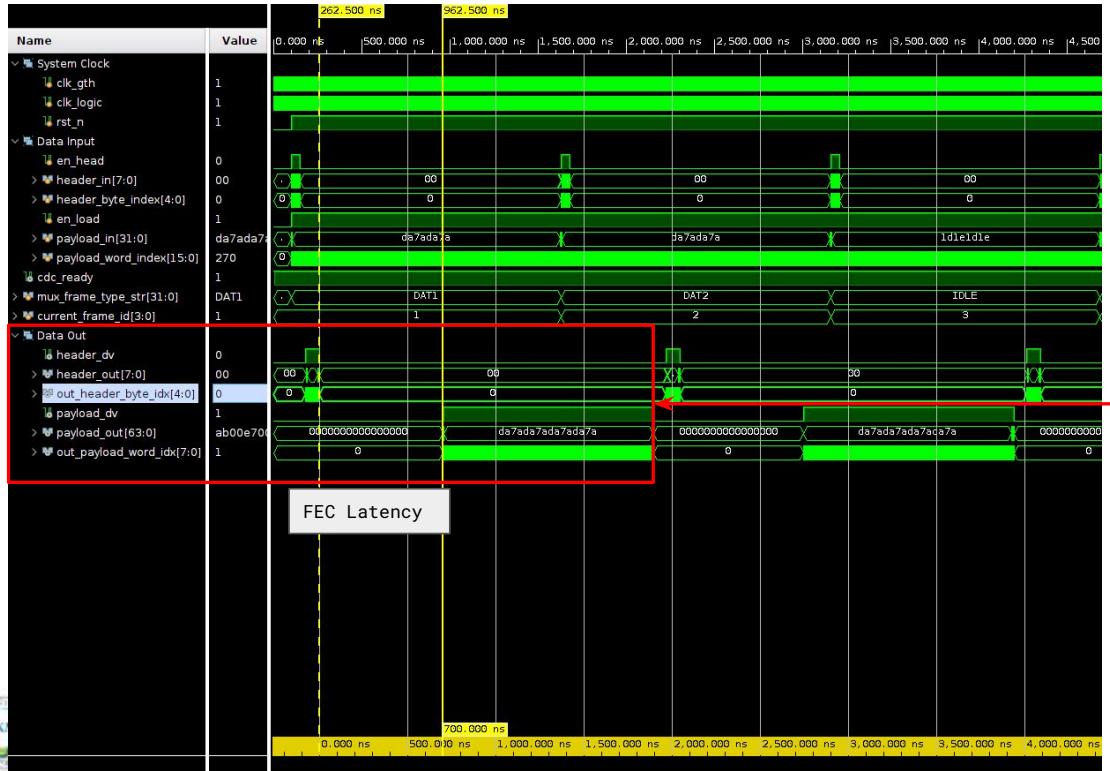
FIFO and Clock Domain Crossing (Testing Sending Jumbo Frame 2000 Byte)
2000 Byte -> Divide to 2 Data. Data 1 (1912 Byte) + Data 2 (112 Byte Data+1800 Byte Zero Padding)





Update Module for FSO Frame –
Update Module for CRC

FIFO and Clock Domain Crossing (Testing Sending Jumbo Frame 2000 Byte)
2000 Byte -> Divide to 2 Data. Data 1 (1912 Byte) + Data 2 (112 Byte Data+1800 Byte Zero Padding)



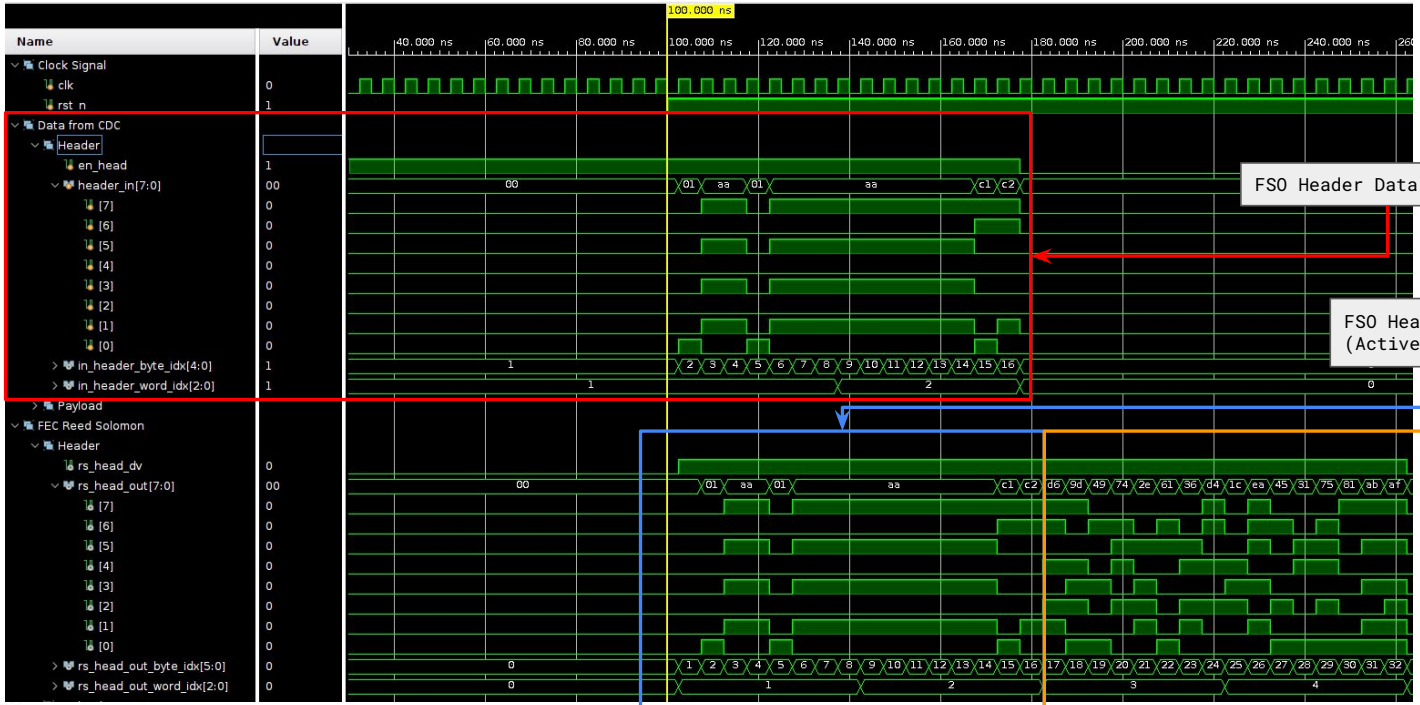
The FSM in the CDC **intentionally holds the payload for 164 cycles to wait for the FEC (Reed-Solomon)** to finish calculating the parity of the header. In the final MUX Gearbox, **this 164-cycle gap will be precisely filled by the 72-bit preamble and parity RS**, so that the final flow to the optical laser (GTH) returns to **continuous without any gap.**

Module for FEC Reed Solomon –
Module for RS Header

Reed Solomon for Header Data. RS (32,16)

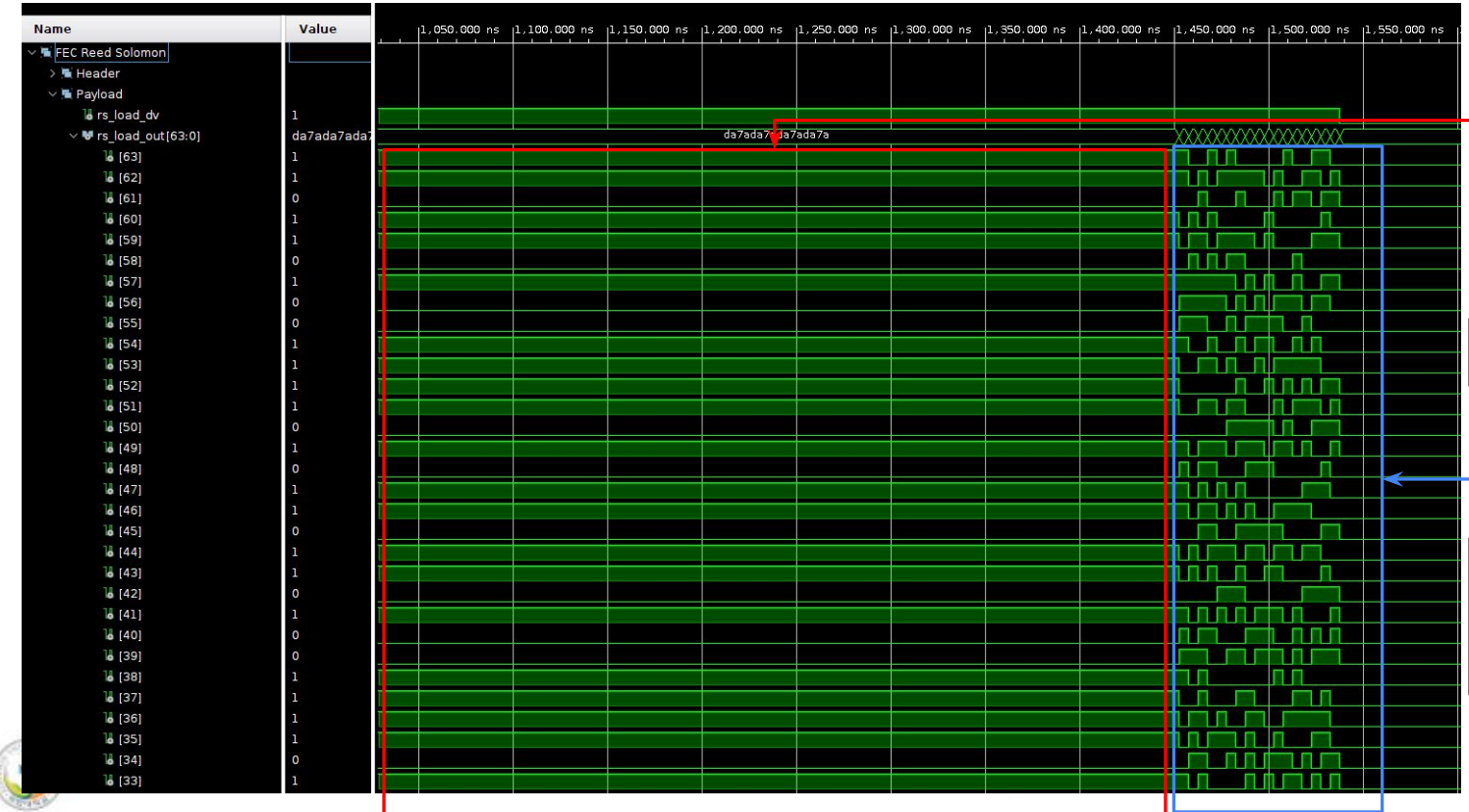
- Input Header : 16 Cycles x 1 Byte = 16 Bytes or 2 Words

- RS Head Out : 32 Cycles x 1 Byte = 32 Bytes or 4 Words



Module for FEC Reed Solomon –
Module for RS Payload

Reed Solomon for Payload Data. RS (255,239)



FSO Payload Data 1912 Byte (Active Phase)

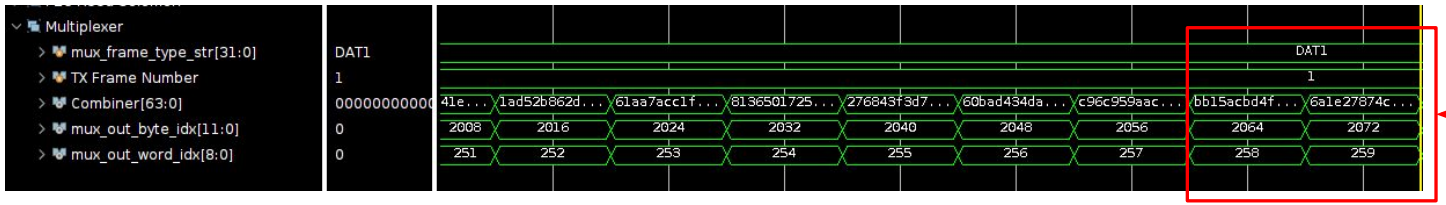
FSO Payload Parity Data 128 Byte (Flush Phase)

- Input Payload : 239 Cycles x 8 Byte = 1912 Bytes or 239 Words
- RS Load Out : 255 Cycles x 8 Byte = 2040 Bytes or 255 Words

Module for FEC Multiplexer –
Module for FEC Multiplexer

Multiplexer from RS Solomon. Merge FSO Header Data and FSO Payload Data





Combiner

FSO Header with RS Parity Data:
32 Byte (4 Words)

FSO Payload with RS Parity Data:
2040 Byte (255 Words)

Data Out: 32+2040=2072 Byte(259 Words)

Module for Scrambler –
Module for Scrambler

Scrambler Data

The scrambler randomizes the bit sequence before transmission. Its purpose is to avoid long, consecutive strings of '0' or '1' bits, maintaining Direct Current balance and ensuring successful clock recovery on the receiving device (GTH Transceiver).

scrambler.v

/home/williams/MYFPGA/FPGA Final/SDAOISL/sfp_int/sfp_int_0/gt_ex/gt_ex.srcs/sources_1/new/encoder/scrambler.v

```
120  assign B[1]   = {B[0][63:0], B[0][64] ^ B[0][61]};
121  assign B[2]   = {B[1][62:0], B[1][64] ^ B[1][61], B[1][63] ^ B[1][60]};
122  assign B[3]   = {B[2][61:0], B[2][64] ^ B[2][61], B[2][63] ^ B[2][60], B[2][62] ^ B[2][59]};
123  assign B[4]   = {B[3][60:0], B[3][64] ^ B[3][61], B[3][63] ^ B[3][60], B[3][62] ^ B[3][59], B[3][61] ^ B[3][58]};
124  assign B[5]   = {B[4][59:0], B[4][64] ^ B[4][61], B[4][63] ^ B[4][60], B[4][62] ^ B[4][59], B[4][61] ^ B[4][58], B[4][60] ^ B[4][57]};
125  assign B[6]   = {B[5][58:0], B[5][64] ^ B[5][61], B[5][63] ^ B[5][60], B[5][62] ^ B[5][59], B[5][61] ^ B[5][58], B[5][60] ^ B[5][57], B[5][59] ^ B[5][56]};
126  assign B[7]   = {B[6][57:0], B[6][64] ^ B[6][61], B[6][63] ^ B[6][60], B[6][62] ^ B[6][59], B[6][61] ^ B[6][58], B[6][60] ^ B[6][57], B[6][59] ^ B[6][56], B[6][58] ^ B[6][55]};
127  assign B[8]   = {B[7][56:0], B[7][64] ^ B[7][61], B[7][63] ^ B[7][60], B[7][62] ^ B[7][59], B[7][61] ^ B[7][58], B[7][60] ^ B[7][57], B[7][59] ^ B[7][56], B[7][58] ^ B[7][55], B[7][57] ^ B[7][54]};
128  assign B[9]   = {B[8][55:0], B[8][64] ^ B[8][61], B[8][63] ^ B[8][60], B[8][62] ^ B[8][59], B[8][61] ^ B[8][58], B[8][60] ^ B[8][57], B[8][59] ^ B[8][56], B[8][58] ^ B[8][55], B[8][57] ^ B[8][54], B[8][56] ^ B[8][53]};
```

The randomization process is carried out through a pure XOR logic operation between the 64-bit input data (data_in_d0) and the active PRBS state vector (S). For each pair of bytes entering the scrambler, the bits shall enter the scrambler in the order byte 0 (MSB) to byte 1 (LSB).

2.6.3.6.7 Scrambler Parallel Implementation Overview and Bit Ordering

The scrambler defined in [2] refers to the bit-serial implementation of the scrambler. The implementer may use a bit-serial representation or unroll it in a parallel implementation. Bit- and byte-ordering are defined using the bit-serial reference implementation of the standard to avoid ambiguity.

The scrambler is applied to all data bytes from both the Header and Payload Reed-Solomon encoders prior to LPC encoding. Only the 72-bit preamble sequence is not scrambled. The scrambler shall be applied as illustrated in Figure 2-12. For each pair of bytes entering the scrambler, the bits shall enter the scrambler in the order byte 0 (MSB) to byte 1 (LSB) as illustrated in the figure. Bytes shall enter the scrambler in natural order from the encoded header followed by the encoded payload.

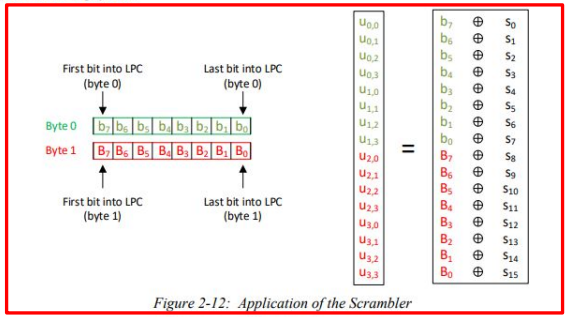


Figure 2-12: Application of the Scrambler

The ingress bits are scrambled as the exclusive or of the bits produced by the scrambler PRBS in natural order. Per Figure 2-12, the scrambler PRBS shall be constructed with the polynomial from ([2] Section 3.3.2.3.3) where, after initialization to the state from Table 2-17 at the start of every header, the bits s_k represent the natural (in-order) outputs from the scrambler PRBS, advancing without reset of its state while the totality of the header and payload bytes are scrambled.

Module for Scrambler –
Module for Scrambler

Scrambler Data



Data from FEC Multiplexer

2072 Byte

Scrambler Data

2072 Byte

Module for Line Product Code –
Module for LPC (24,16)

LPC Data

LPC serves as a secondary layer Forward Error Correction (FEC) mechanism after Reed-Solomon. This compression method detects and corrects additional bit errors that may occur during optical transmission.

lpc_top.v

/home/williams/MyFPGA/FPGA Final/SDAOISL/sfp_int/sfp_int_0/gt_ex/gt_ex.srcs/sources_1/new/encoder/lpc_top.v

```
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////

module lpc_top(
    input               clk,
    input               rst_n,
    input               en,
    input   [63:0]      data_in,
    output              flag,
    output  [95:0]      data_out
);

lpc lpc_3 (
    .clk            (clk)
    ..rst_n         (rst_n)
    ..en            (en)
    ..data_in       (data_in    [63:48])
    ..flag          (flag)
    ..data_out      (data_out   [95:72])
);

lpc lpc_2 (
    .clk            (clk)
    ..rst_n         (rst_n)
    ..en            (en)
    ..data_in       (data_in    [47:32])
    ..flag          ()
    ..data_out      (data_out   [71:48])
);

lpc lpc_1 (
    .clk            (clk)
    ..rst_n         (rst_n)
    ..en            (en)
    ..data_in       (data_in    [31:16])
    ..flag          ()
    ..data_out      (data_out   [47:24])
);

lpc lpc_0 (
    .clk            (clk)
    ..rst_n         (rst_n)
    ..en            (en)
    ..data_in       (data_in    [15:0])
    ..flag          ()
    ..data_out      (data_out   [23:0])
);

endmodule
```

Distributes the 64-bit bus of the Scrambler into **four parallel lines of 16 bits each**. These four lines are processed simultaneously by four instances of the lpc submodule **(lpc_3 to lpc_0)**.

In each lpc.v sub-module, the 16-bit input is first mapped into the encryption component of the matrix E[0:15]. Then, the module computes the Horizontal (H[0:3]) and Vertical (V[0:3]) vectors using high-speed cascaded linear XOR equations.

- u_lpc_top : lpc_top (lpc_top.v) (4)
  - lpc_3 : lpc (lpc.v)
  - lpc_2 : lpc (lpc.v)
  - lpc_1 : lpc (lpc.v)
  - lpc_0 : lpc (lpc.v)

The ingress bits to the LPC encoder shall be first scrambled (Section 2.6.3.6.6). After scrambling, the bits shall be differentially encoded as illustrated in Figure 2-13 where the notation for the differentially encoded bits has been adapted from ([2] Section 3.3.2.3.4.4).



Figure 2-13: Differentially Encoding in LPC Encoder

After differentially encoding, horizontal and parity bits are computed (per [2] Sections 3.3.2.3.4.4.2 and 3.3.2.3.4.4.3, respectively). Active disparity management shall be applied ([2] Section 3.3.2.3.4.5). The LPC-encoded bits are represented as shown in Figure 2-14.

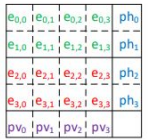


Figure 2-14: LPC Codeword

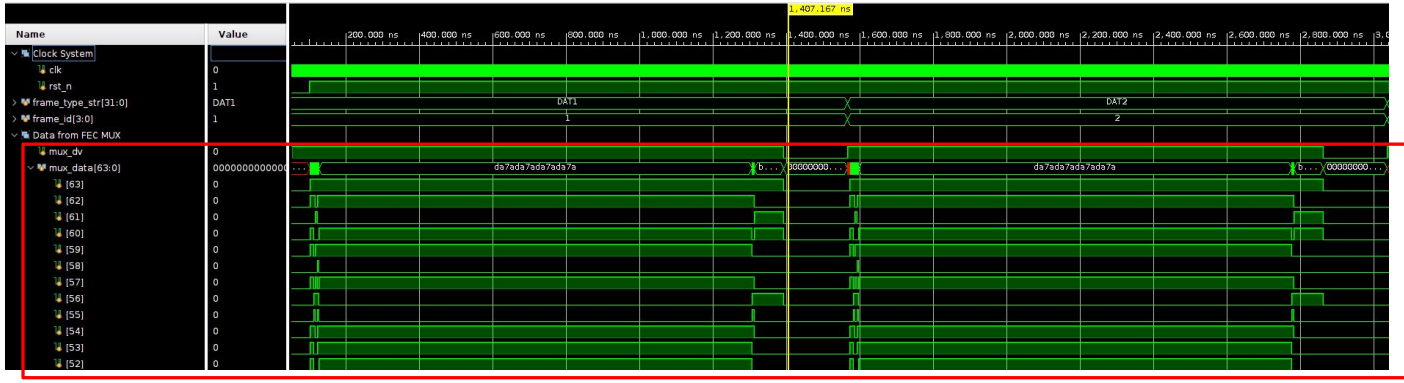
The transmission order of the LPC-encoded bits, after active disparity management, shall be: $\{e_{0,0}, e_{0,1}, e_{0,2}, e_{0,3}, e_{1,0}, e_{1,1}, e_{1,2}, e_{1,3}, e_{2,0}, e_{2,1}, e_{2,2}, e_{2,3}, e_{3,0}, e_{3,1}, e_{3,2}, e_{3,3},$

$$ph_0, ph_1, ph_2, ph_3, pv_0, pv_1, pv_2, pv_3\}$$

with bit $e_{0,0}$ entering the channel first and bit pv_3 transmitted last of the LPC codeword.

Module for Line Product Code –
Module for LPC (23,16)

LPC Data



Data from FEC Multiplexer

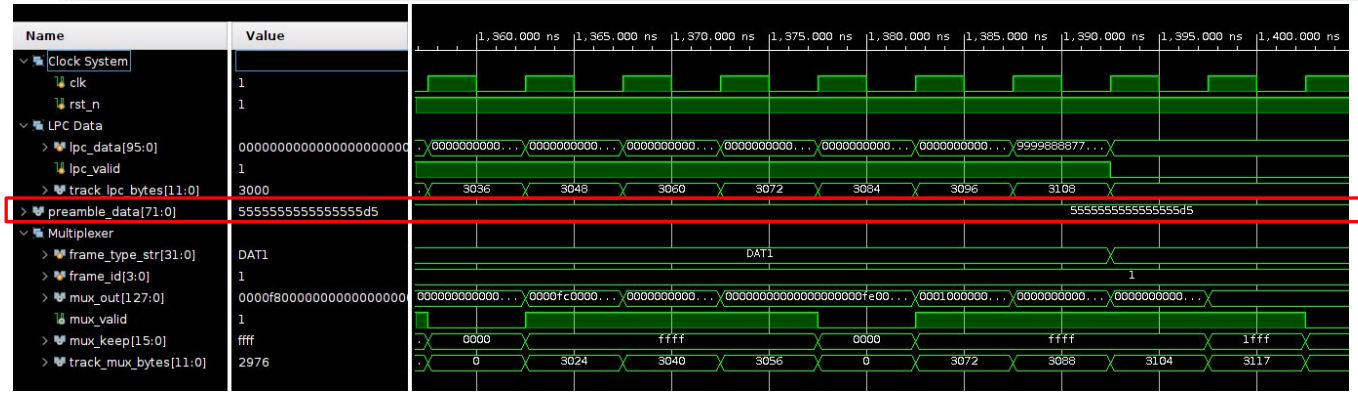
2072 Byte



Scrambler Data

2072 Byte

New Module for Preamble Generator –
Module for Preamble Sequence (24,16)



5555555555555555d5 -> 72 bit or 9 byte

Show that preamble data 72 bit (9 Byte) already been transfer without scrambled and LPC

2.6.3.6.9 Frame Generation Summary

Construction of the OISL frame and its integration with the line product code is summarized in Figure 2-15.

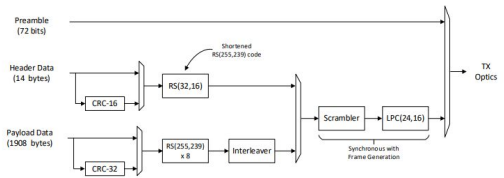


Figure 2-15: Frame Encoding Summary

2.6.3.2 Preamble Sequence

Every frame shall begin with a Preamble Sequence (PS), which is used by the receiving modem for frame synchronization. The preamble sequence shall be identical for all OISL modem frames and shall take the value shown in Table 2-8.

Table 2-8: Modem Frame Summary

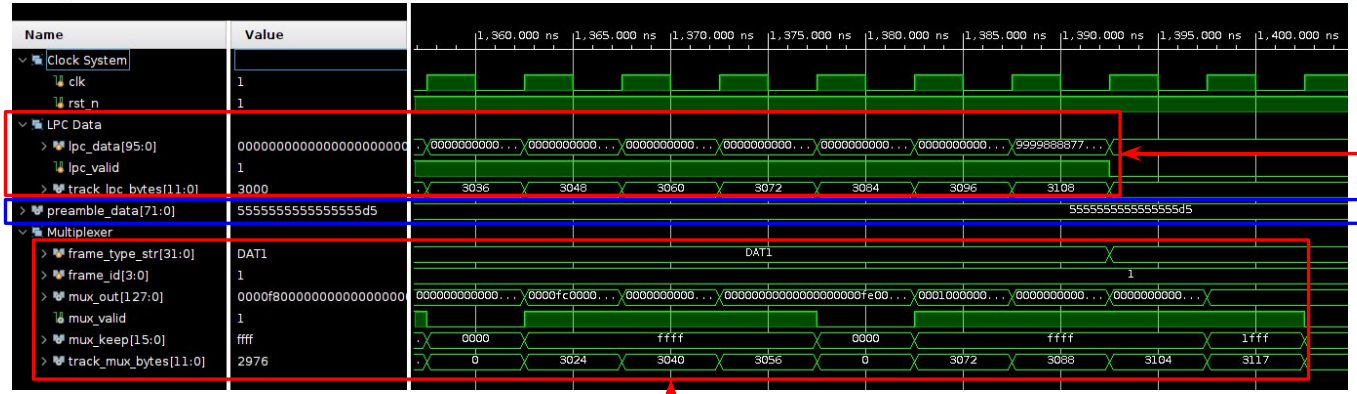
Field	Number of bits	Number of LPC Code Words	Total Number of bits	Comments
Preamble	72	N/A	72	Preamble: 7 2 ' 77AD5B584364 1E2E26. The MSB (72nd) bit is transmitted first followed in order down to the LSB (1st)

2.6.3.6.4 Scrambling and Line Product Code

The scrambler and Line Product Code (LPC) shall be implemented and shall be adopted from [2]. The scrambler follows ([2] Section 3.3.2.3.3.1) with details of its initial seed provided in Section 2.6.3.6.5 of this Standard.

- The scrambler shall reset to a fixed value (Table 2-17) at the start of every modem header.
- The scrambler shall advance naturally after scrambling the header to also scramble the payload bits.
- The preamble bits shall not be scrambled.

New Module for Preamble –
Module for Multiplexer for LPC Data and Preamble



Data from LPC (24,16) -> 3108 Byte

Preamble Data -> 9 Byte

Total = 3108 Byte (LPC) + 9 Byte (Preamble) = 3117 Byte or 24936 bits.

Byte 1 to Byte 9 (9 bytes / 72 bits): Preamble (Pure, unscrambled, used by the Frame Synchronizer at the receiver as a reference for the frame zero point).

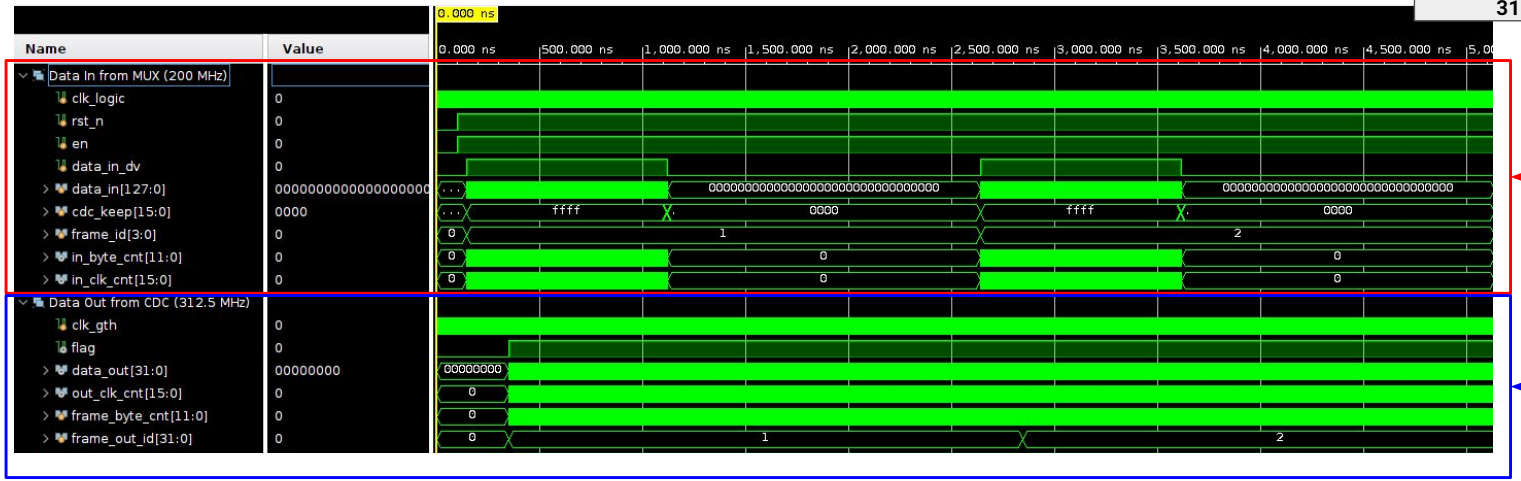
Byte 10 to Byte 57 (48 bytes / 384 bits): Encoded Header (derived from the 32-byte RS Header that has been scrambled and expanded by LPC 16/24 to 48 bytes).

Byte 58 to Byte 3117 (3060 bytes / 24,480 bits): Encoded Payload (derived from the 2040-byte RS Payload that has been scrambled and expanded by LPC 16/24 to 3060 bytes).

Update Module for Clock Domain Crossing –
Module for CDC Logic Clk to GTH

We put same condition sending 2000 Byte.

2000 Byte -> Divide to 2 Data Frame.



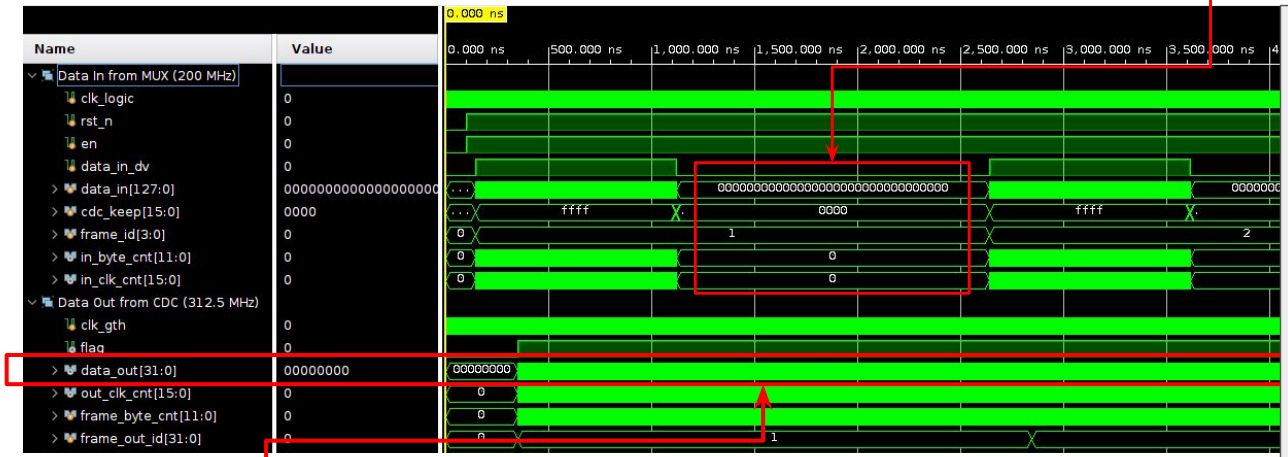
Data In from Multiplexer to CDC
3117 Byte or 24936 bits

Data Out from CDC to GTH
3117 Byte or 24936 bits

Update Module for Clock Domain Crossing –
Module for CDC Logic Clk to GTH

We put same condition sending 2000 Byte.
2000 Byte -> Divide to 2 Data Frame.

This interval is not an optical transmission pause, but a Bandwidth Rate-Matching mechanism.



NO GAP.
synchronous with no pauses between frames and no pauses

Write Domain (200 MHz x 128 bit)
Read Domain (312.5 MHz x 32 bit)

At the 10.0 Gbps optical line rate, transmitting one complete 24,936-bit OISL frame requires 2.4936 µs for frame duration (24,936 bit / 10 Gbps= 2.4936 µs or 2493.6 in nanosecond)

However, because the 128-bit write bus operates at a 25.6 Gbps burst rate, the MUX Gearbox finishes writing all 3117 Bytes into the CDC FIFO in just 975.0 ns.

To match the 10.0 Gbps average line rate and prevent FIFO overflow, the write interface remains idle for the remaining 304 clock cycle

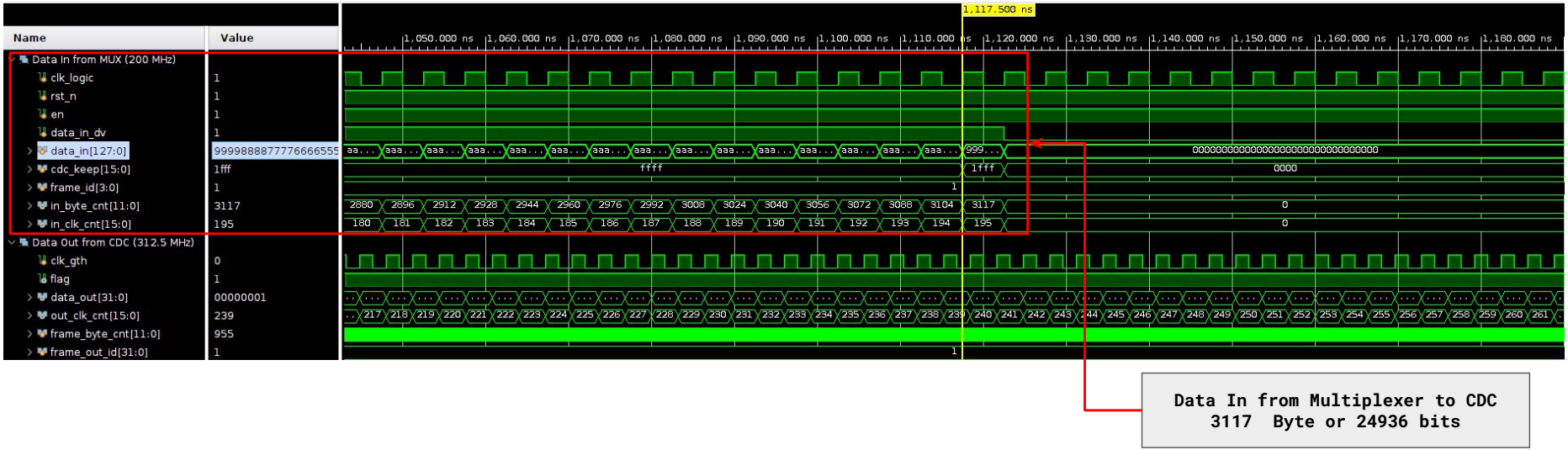
(24933.6 ns-975.0 ns ≈ 1520.0 ns or 304 clock cycle)

while the upstream FEC/LPC pipeline processes the next frame.

Update Module for Clock Domain Crossing –
Module for CDC Logic Clk to GTH

We put same condition sending 2000 Byte.

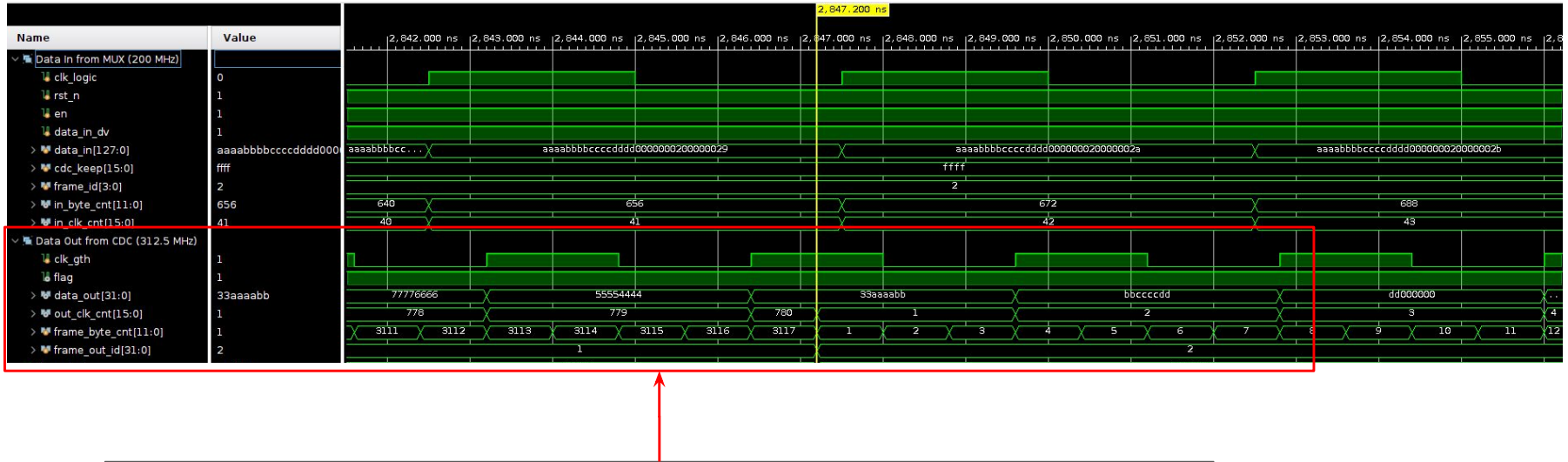
2000 Byte -> Divide to 2 Data Frame.



Update Module for Clock Domain Crossing –
Module for CDC Logic Clk to GTH

We put same condition sending 2000 Byte.

2000 Byte -> Divide to 2 Data Frame.



Data Out from CDC to GTH
3117 Byte or 24936 bits

Sending continuously without any gap. Data 1 and Data 2

Following SDA OISL "Transmission of modem frames shall be synchronous with no pauses between frames and no pauses between any of the bits comprising the frame components in Table 2-8. From the values in Table 2-8, the total frame size shall be 72+384+24480=24936 bits."

Update Module for Clock Domain Crossing –
Module for CDC Logic Clk to GTH



2.4 Effective Data Rate

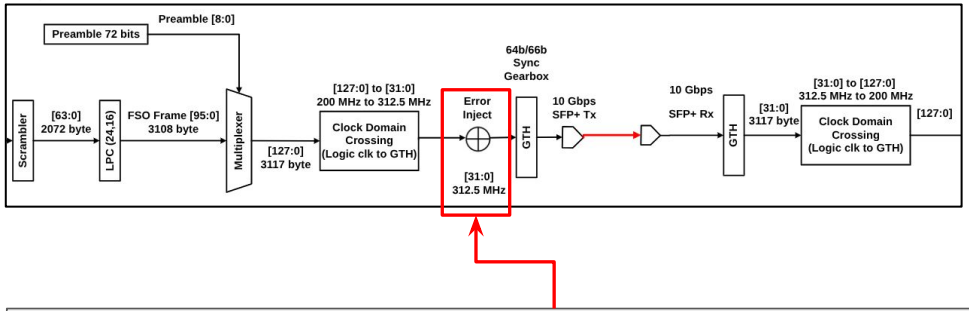
The effective data rates for the primary OISL modem frame configurations are summarized in Table 2-4 as a function of the OISL baud rate. The last column of the table indicates the required support for each baud rate.

Table 2-4: Maximum Effective Data Rates for Frame Configurations

Baud Rate (Mbaud)	Frame Duration (microseconds)	Effective Data Rate (Mbps)	Frame Rate (kHz)	Fast Control Channel (Mbps)	Requirement
312.5	79.795	191.290	12.532	0.201	Shall be supported
625	39.898	382.579	25.064	0.401	Should be supported
1250	19.949	765.159	50.128	0.802	Should be supported
2500	9.974	1530.318	100.257	1.604	May be supported
5000	4.987	3060.635	200.513	3.208	May be supported
10000	2.494	6121.270	401.027	6.416	May be supported

Baud Rate	Frame Duration	Effective Data Rate	Frame Rate	Fast Control Channel
10000 Mbaud (312.5 MHz x 32 Bit = 10 Gbps)	2.4936 us (In this slide show 2,493.600 ns)	6121.270 Mbps (6,121 Gbps)	401.02 66 kHz	6.416 Mbps

New Module for Error Inject –
Module for Error Inject for Simulation Mode and Hardware Mode



In Here we make 2 mode error injector

1. **Simulation Mode -> using Digital Memory Injection Active (error.mem)**
   The error.mem file contains 48 rows of 32-bit words (index 0 to 47, total 48×4 Bytes=192 Bytes).
   XOR the 32 bit data stream with hexadecimal data from error.mem

2. **Hardware Mode -> Digital Bypass + Active VOA Manual Hardware (Single Mode 0.9-1m /t49G/51 LC/PC)**
   3117 bytes of data pass through the GTH TX and SFP+ TX with the same pipe latency. Bit errors are then physically generated in the real world by turning the damping screw on the Manual Variable Optical Attenuator (LC/PC /t49G/51)

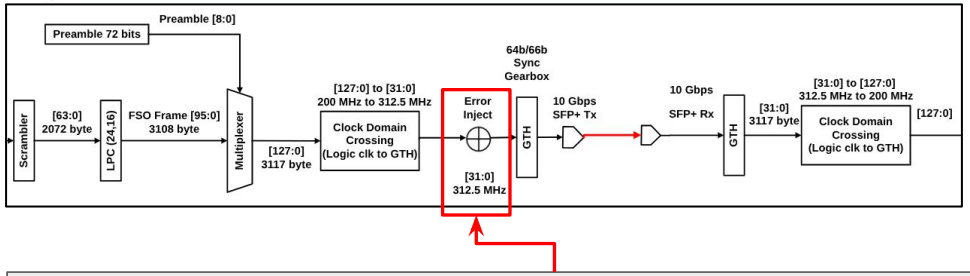
/home/williams/MYFPGA/FPGA Final/SDAOISL/sfp_int/sfp_int_0/gt_ex/gt_ex.srcs/sources_1/new/error.mem

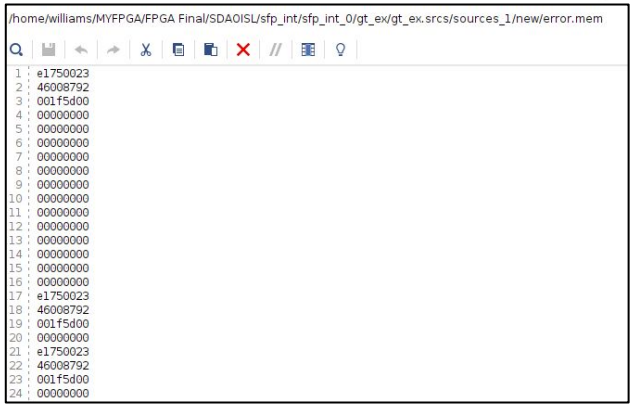
```
1  e1750023
2  46008792
3  001f5d00
4  00000000
5  00000000
6  00000000
7  00000000
8  00000000
9  00000000
10 00000000
11 00000000
12 00000000
13 00000000
14 00000000
15 00000000
16 00000000
17 e1750023
18 46008792
19 001f5d00
20 00000000
21 e1750023
22 46008792
23 001f5d00
24 00000000
```



Manual Variable Attenuator (Single Mode)-0.9-1m /t49G/51
Connector type: **LC/PC**. Brand : **Fiberpia**

New Module for Error Inject –
Module for Error Inject for Simulation Mode





Simulation Mode -> using Digital Memory Injection Active (error.mem)
The error.mem file contains 48 rows of 32-bit words (index 0 to 47, total 48×4 Bytes=192 Bytes).

XOR the 32 bit data stream with hexadecimal data from error.mem

3 Operational Zones:

Zone 1 – Preamble & Header Protection -> For the first 15 words (60 bytes = Byte 1 to Byte 60), which includes 9 bytes of preamble + 48 bytes of LPC header + 3 bytes of margin

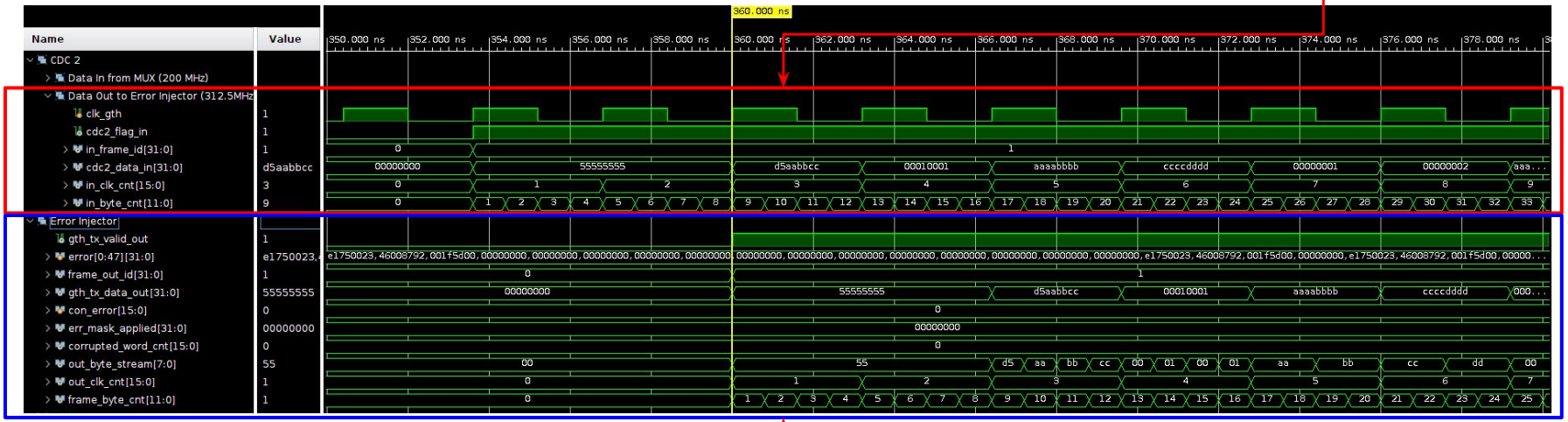
Zone 2 – Payload Error Injection -> From word 16 to word 63 (48 words = 192 bytes, i.e., Byte 61 to Byte 252), the module reads 48 lines of the garbled pattern from error.mem and XORs them with the payload data.

Zone 3 – Payload Remainder Pass-Through -> From word 64 to byte 3117 (the remaining 2865 bytes, i.e., Byte 253 to Byte 3117), the data Pass-Through. Right on reaching Byte 3117, the FSM immediately rolls back to Zone 1 (IDLE) to protect the Preamble of the next frame without ever dropping the gth_tx_valid_out signal.

New Module for Error Inject –
Module for Error Inject for Simulation Mode

We put same condition sending 2000 Byte.
2000 Byte -> Divide to 2 Data Frame.

Data In from CDC transfer 3117 Byte to Error Inject



Data Out From transfer 3117 Byte to GTH

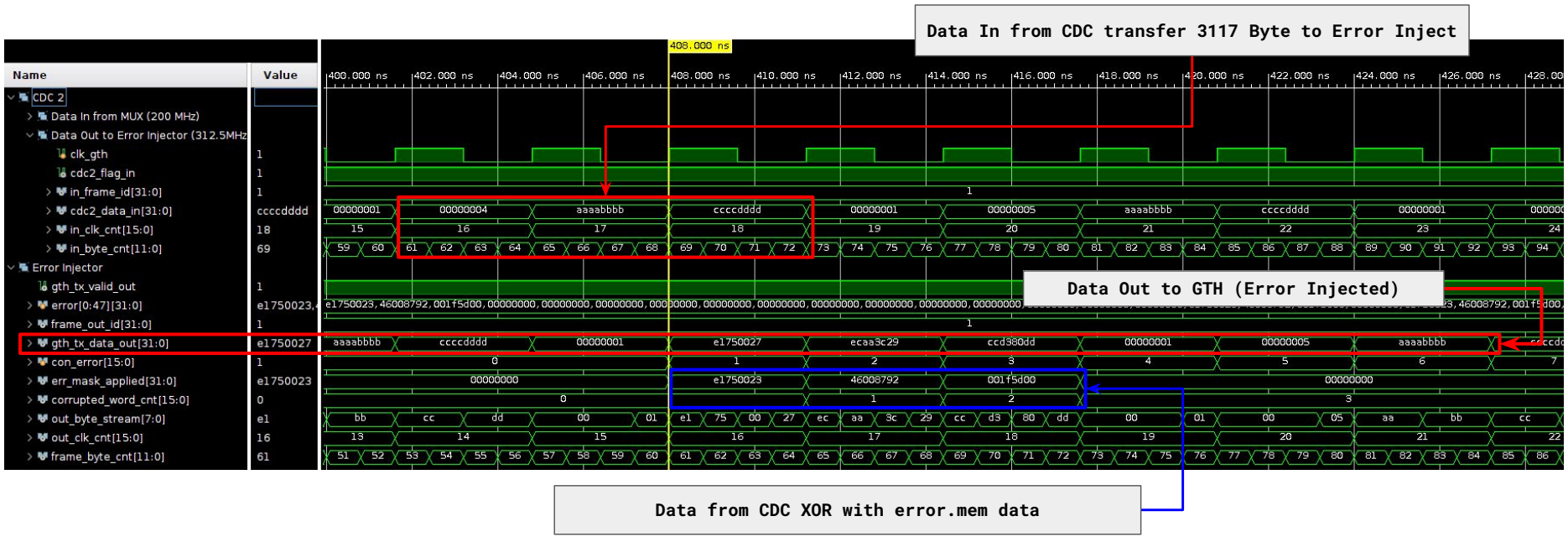
Zone 1 – Preamble & Header Protection -> Byte 1 to Byte 60

Zone 2 – Payload Error Injection -> Byte 61 to Byte 252

Zone 3 – Payload Remainder Pass-Through -> Byte 253 to Byte 3117

New Module for Error Inject –
Module for Error Inject for Simulation Mode

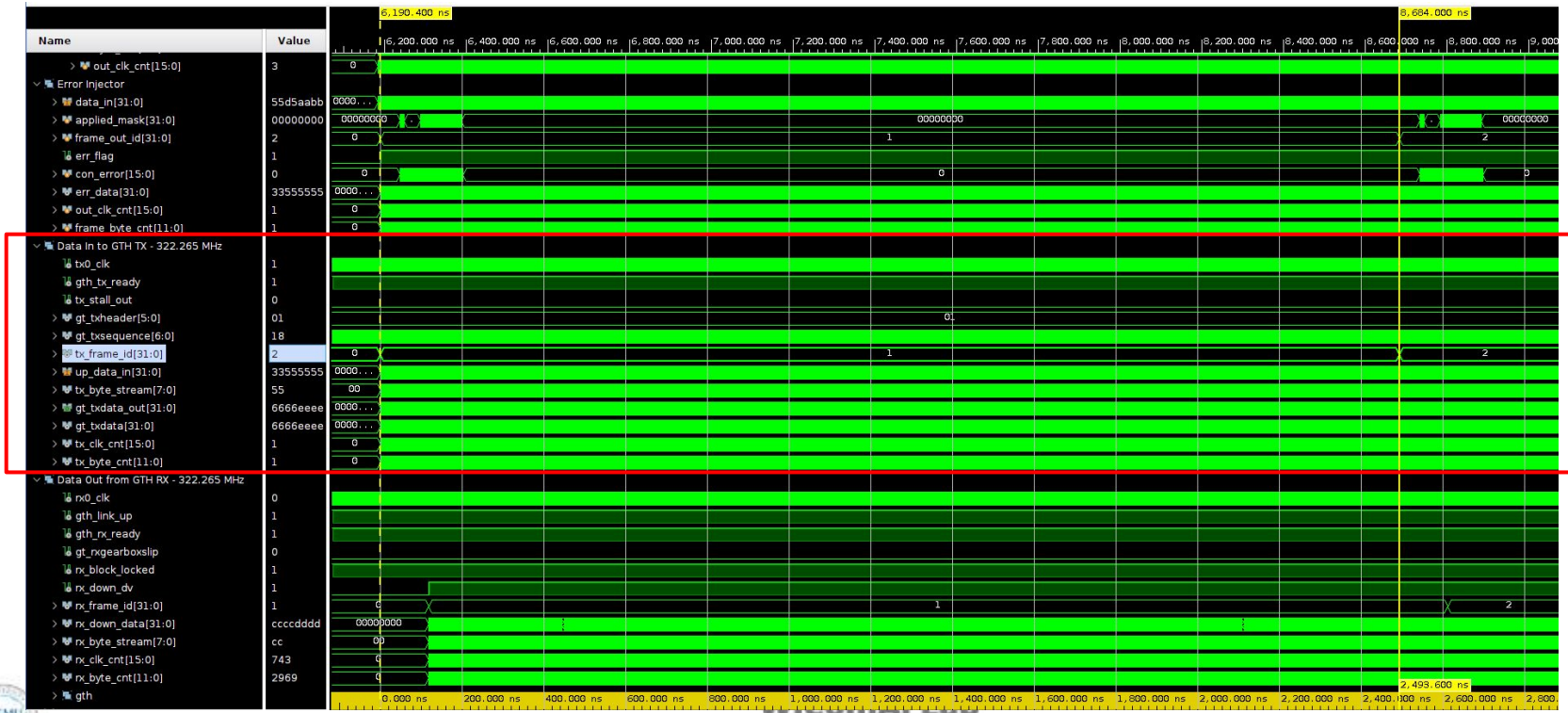
Error Inject with Error.mem for Simulation Mode



New Module for Gigabit Transceiver High Performance (GTH) – Module for GTH TX

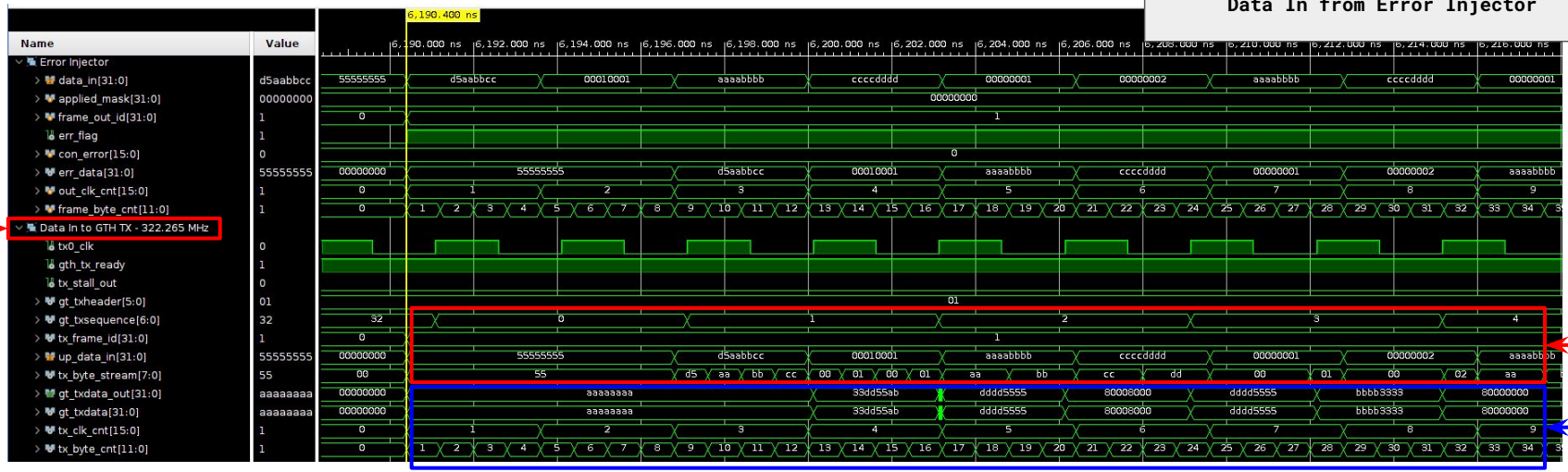
GTH TX from FPGA 1 to GTH RX FPGA 2

Data transfer from FPGA 1 -> all the frame duration, data rate still same



New Module for Gigabit Transceiver High Performance (GTH) – Module for GTH TX

GTH TX from FPGA 1 to GTH RX FPGA 2



Data In from Error Injector

(322.265625 MHz = 312.5 MHz x (66/64))

64B/66B Gearbox Clock Frequency Ratio:

$$f_{line} = 10Gbps \times \frac{66}{64} = 10.3125Gbps$$

$$f_{usrclk2} = \frac{10.3125Gbps}{32bit} = 322.265625\,MHz\ (T_{usrclk2} = 3.10303ns)$$

Since every 66 tx0_clk cycles there are 64 active data cycles and 2 pause cycles (txsequence == 32), the average effective data rate is:

$$R_{eff_GTH} = 322.265625MHz \times \frac{64}{66} \times 32bit = 312.5MHz \times 32bit = 10.000Gbps$$

Data Out from GTH TX FPGA 1 (bit-reversal so that the MSB is sent first optically)

New Module for Gigabit Transceiver High Performance (GTH) – Module for GTH RX

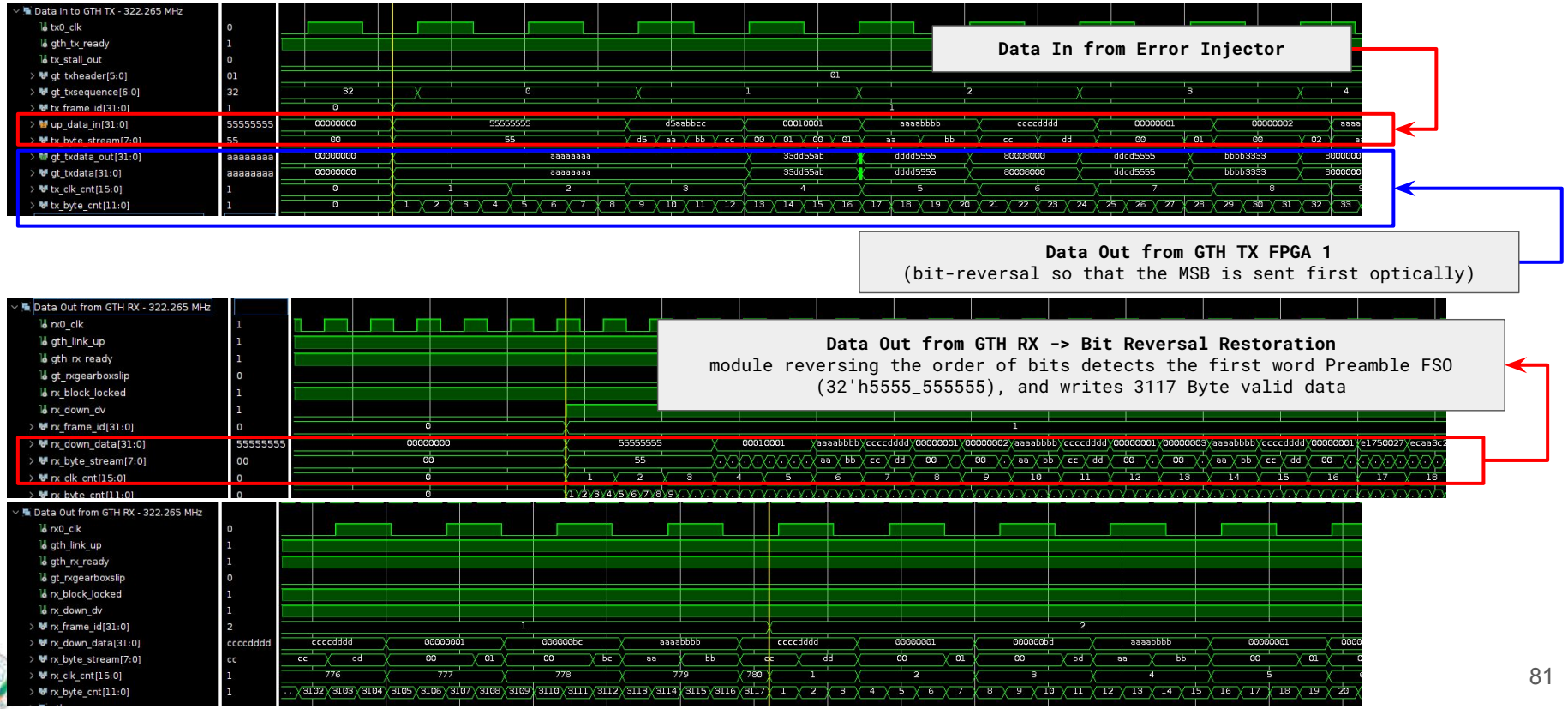
GTH RX FPGA 2



Data Receive from FPGA 1 -> all the frame duration, data rate still same

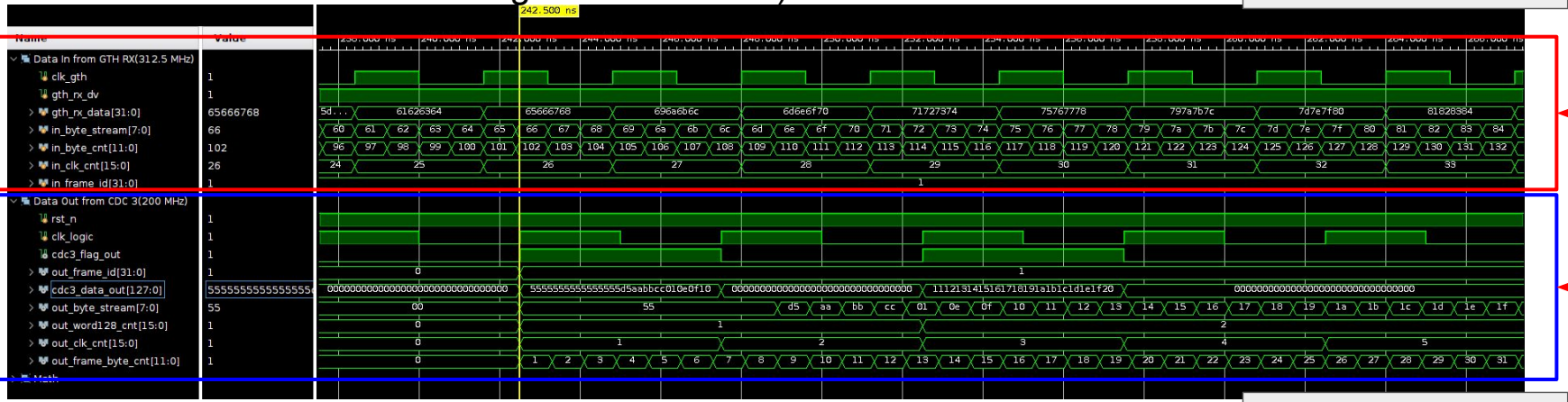
New Module for Gigabit Transceiver High Performance (GTH) – Module for GTH RX

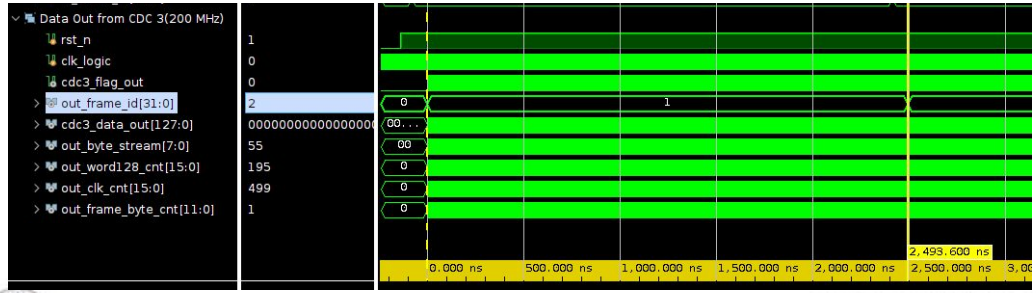
GTH RX FPGA 2



Update Module for Clock Domain Crossing –
Module CDC GTH to Logic Clk

CDC for GTH 312.5 MHz to Logic Clk 200 MHz)



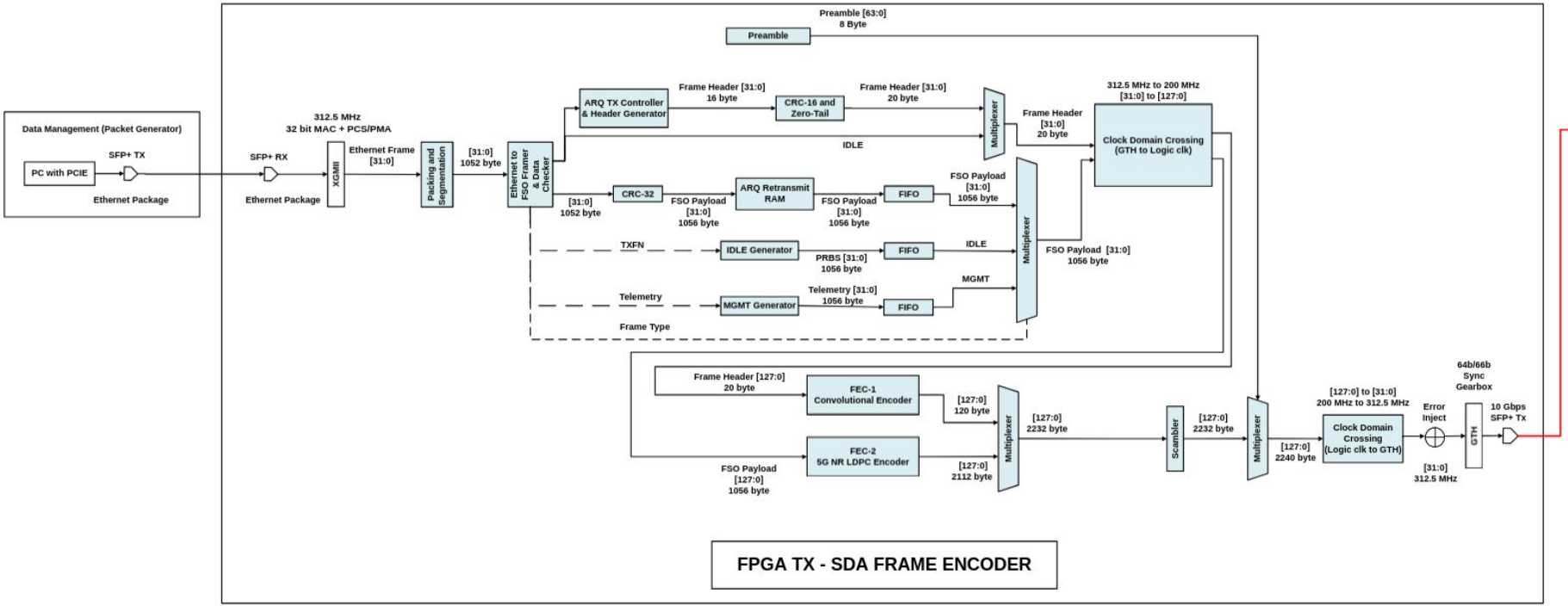


all the frame duration, data rate still same

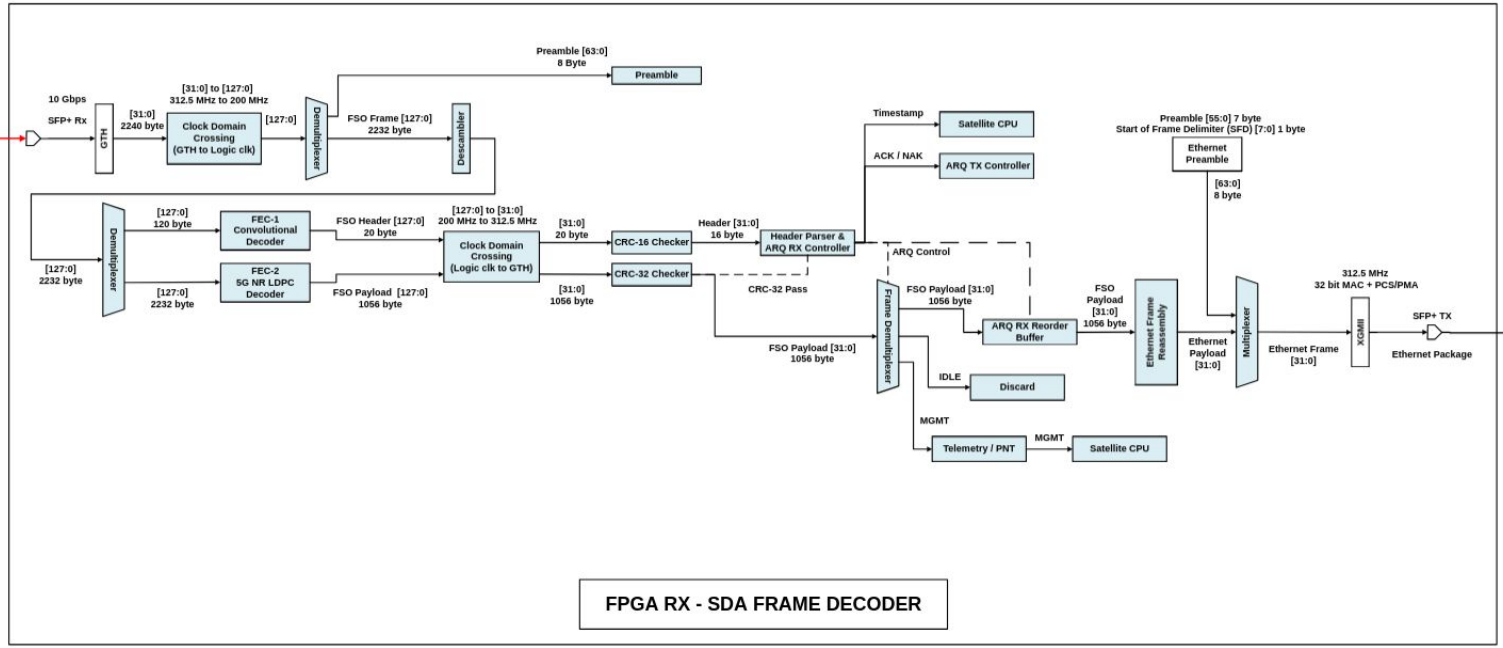
WiComAI Lab
Wireless Communications and Artificial Intelligence Lab

System Architecture following SDA OCT (2027)

System Architecture following SDA OCT(2027)

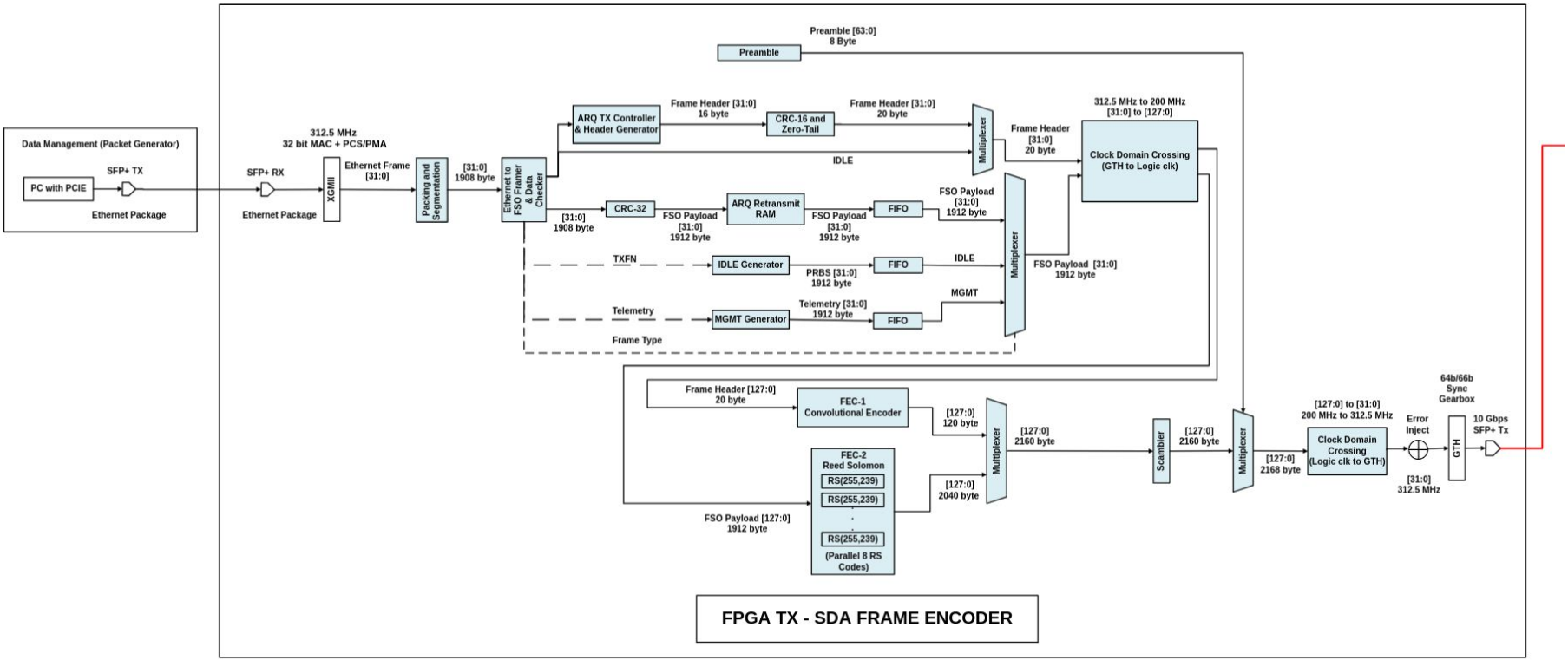


System Architecture following SDA OCT(2027)



System Architecture following ESA ESTOL(2028)

System Architecture following ESA ESTOL(2028)



System Architecture following ESA ESTOL(2028)

